

Квест-Курс С# для продвинутых

Модуль 1 - Продвинутое возможности языка

Урок 1 - Делегаты и события

Шаг 01:

Миссия 1

Алекс создает программу уведомлений для команды. Какой тип делегата лучше использовать для передачи сообщений?

Подсказка: Делегаты позволяют передавать методы как параметры.

Ответ:

- **Action**
- int
- string
- bool

Шаг 02:

Миссия 2

Боб хочет, чтобы его программа автоматически уведомляла пользователей о событиях. Какое ключевое слово используется для объявления события?

Подсказка: Событие связано с делегатом и обычно использует специальное ключевое слово.

Ответ:

- **event**
- delegate
- void
- notify

Шаг 03:

Миссия 3

Клара создает делегат для логирования ошибок. Какой тип делегата лучше использовать, если метод не возвращает значение, но принимает строку?

Подсказка: Action подходит для методов без возвращаемого значения.

Ответ:

- **Action**
- Func
- Predicate
- EventHandler

Шаг 04:

Миссия 4

Даниэль хочет объединять несколько методов для обработки событий в одну цепочку. Какой оператор поможет объединять делегаты?

Подсказка: Делегаты можно складывать и вычитать.

Ответ:

- +
- *
- -
- /

Шаг 05:

Миссия 5

 Задача: Сопоставьте тип делегата с его назначением.

Типы делегатов:

Ответ:

- Action
- Func
- Predicate
- EventHandler

Назначение:

1. Метод без возвращаемого значения
2. Метод с возвращаемым значением
3. Метод для проверки условия, возвращает bool
4. Метод для обработки событий

 **Подсказка:**

Подумайте о количестве и типах параметров и возвращаемого значения у делегатов.

- **Action** — Метод без возвращаемого значения
- **Func** — Метод с возвращаемым значением
- **Predicate** — Метод для проверки условия, возвращает bool
- **EventHandler** — Метод для обработки событий

Шаг 06:

Миссия 6

 Задача: Сопоставьте событие с правильным описанием.

События:

Ответ:

- Click
- KeyPress
- MouseEnter
- DataReceived

Описание:

1. Нажатие кнопки мыши
2. Нажатие клавиши на клавиатуре
3. Наведение мыши на элемент
4. Получение данных от внешнего источника

 **Подсказка:**

Вспомните, какие события относятся к интерфейсу, а какие к данным.

- **Click** — Нажатие кнопки мыши
- **KeyPress** — Нажатие клавиши на клавиатуре
- **MouseEnter** — Наведение мыши на элемент
- **DataReceived** — Получение данных от внешнего источника

🔗 Шаг 07:

Миссия 7

📖 Сопоставьте действия с правильным методом делегата:

Действия:

Ответ:

- Вывод текста на экран
- Проверка числа на четность
- Получение длины строки
- Обработка клика кнопки

Методы делегатов:

1. Action<string>
2. Predicate<int>
3. Func<string,int>
4. EventHandler

🔗 Подсказка:

Смотрите на количество параметров и возвращаемое значение.

- **Вывод текста на экран** — Action
- **Проверка числа на четность** — Predicate
- **Получение длины строки** — Func
- **Обработка клика кнопки** — EventHandler

🔗 Шаг 08:

Миссия 8

📖 Сопоставьте делегат с примером использования в проекте:

Делегаты:

Ответ:

- Action
- Func
- Predicate
- EventHandler

Примеры:

1. Метод проверки, является ли пользователь администратором
2. Метод вывода уведомления
3. Метод вычисления суммы двух чисел
4. Метод обработки клика кнопки

🔗 Подсказка:

Смотрите на возвращаемое значение и тип параметров.

- **Predicate** — Метод проверки, является ли пользователь администратором
- **Action** — Метод вывода уведомления
- **Func** — Метод вычисления суммы двух чисел
- **EventHandler** — Метод обработки клика кнопки

🔮 Шаг 09:

Миссия 9

Расположите этапы работы с событиями в правильном порядке:

1. Создание делегата
2. Подписка методов на событие
3. Определение события в классе
4. Вызов события

🔮 Подсказка:

Сначала объявляем делегат, затем событие, подписываем методы, и только потом вызываем событие.

Ответ:

- Создание делегата
- Определение события в классе
- Подписка методов на событие
- Вызов события

🔮 Шаг 10:

Миссия 10

Расположите шаги объединения нескольких методов в один делегат:

1. Создание делегата
2. Присвоение первого метода
3. Добавление дополнительных методов с помощью +
4. Вызов делегата

🔮 Подсказка:

Сначала делегат создается, потом присваивается первый метод, затем добавляются остальные.

Ответ:

- Создание делегата
- Присвоение первого метода
- Добавление дополнительных методов с помощью +
- Вызов делегата

🔮 Шаг 11:

Миссия 11

Расположите этапы создания цепочки событий для формы:

1. Определение события
2. Создание обработчиков
3. Подписка обработчиков на событие
4. Вызов события

Подсказка:

Сначала событие определяется, потом создаются обработчики, затем подписка и вызов.

Ответ:

- Определение события
- Создание обработчиков
- Подписка обработчиков на событие
- Вызов события

Шаг 12:

Миссия 12

Расположите шаги создания делегата с возвратом значения:

1. Объявление делегата с типом возвращаемого значения
2. Создание метода с соответствующей сигнатурой
3. Присвоение метода делегату
4. Вызов делегата

Подсказка:

Сначала объявляется делегат, затем создается метод, присваивается делегату и вызывается.

Ответ:

- Объявление делегата с типом возвращаемого значения
- Создание метода с соответствующей сигнатурой
- Присвоение метода делегату
- Вызов делегата

Шаг 13:

Миссия 13

Что такое делегат в C#?

Подсказка: Подумайте о типе, который может хранить ссылку на метод.

Ответ:

- **Тип, который хранит ссылку на метод**
- Интерфейс для работы с коллекциями
- Класс для организации событий
- Модификатор доступа для методов

Шаг 14:

Миссия 14

Что нужно сделать, чтобы подписаться на событие в C#?

Подсказка: Вспомните оператор += для добавления обработчиков.

Ответ:

- **Использовать оператор += и указать метод-обработчик**
- Присвоить событию значение null
- Вызвать событие напрямую как метод
- Использовать оператор -=

🔗 Шаг 15:

Миссия 15

📖 Сопоставьте понятие с его описанием:

Ответ:

- Делегат
- Событие
- Мультикаст делегат
- Подписка на событие

1. Тип, который может ссылаться на один или несколько методов
2. Механизм уведомления объектов о произошедших действиях
3. Делегат, который может хранить несколько методов одновременно
4. Добавление метода в список обработчиков события

Подсказка: Вспомните, как делегаты и события работают вместе.

- **Делегат** — Тип, который может ссылаться на один или несколько методов
- **Событие** — Механизм уведомления объектов о произошедших действиях
- **Мультикаст делегат** — Делегат, который может хранить несколько методов одновременно
- **Подписка на событие** — Добавление метода в список обработчиков события

🔗 Шаг 16:

Миссия 16

Что произойдет, если вызвать событие без подписчиков?

Подсказка: Подумайте, что происходит при null значении.

Ответ:

- **Произойдет исключения null не будет, если проверить на null**
- Событие вызовет все методы из сборки
- Компилятор автоматически добавит пустой обработчик
- Программа завершится аварийно

📖 Урок 2 - Лямбда-выражения и локальные функции

🔗 Шаг 01:

Миссия 1

Алекс создает программу для фильтрации списка задач. Какой синтаксис используется для создания лямбда-выражения?

Подсказка: Лямбда-выражение выглядит как параметры => выражение или блок кода.

Ответ:

- **x => x > 0**
- **def x(x) { return x > 0; }**
- **function(x) x > 0**
- **lambda x: x > 0**

🔗 Шаг 02:

Миссия 2

Боб хочет использовать лямбда-выражение для сортировки списка имен по длине. Какой метод LINQ ему подойдет?

Подсказка: Обратите внимание на методы, которые принимают функцию проекции.

Ответ:

- **OrderBy**
- Where
- Select
- Aggregate

Шаг 03:

Миссия 3

Клара хочет объявить локальную функцию внутри метода для вычисления факториала. Какое ключевое слово нужно использовать?

Подсказка: Локальная функция объявляется внутри метода как обычный метод.

Ответ:

- **локальная функция объявляется как обычный метод внутри метода**
- using
- delegate
- func

Шаг 04:

Миссия 4

Даниэль создает лямбда-выражение, которое суммирует два числа. Какой тип делегата лучше использовать?

Подсказка: Лямбда возвращает значение, значит нужен Func.

Ответ:

- **Func**
- Action
- Predicate
- EventHandler

Шаг 05:

Миссия 5

 Сопоставьте лямбда-выражение с его назначением:

Лямбда:

Ответ:

- $x \Rightarrow x * 2$
- $(a,b) \Rightarrow a + b$
- $str \Rightarrow str.Length$
- $n \Rightarrow n \% 2 == 0$

Назначение:

1. Проверка, является ли число четным
2. Умножение числа на 2
3. Вычисление длины строки

4. Суммирование двух чисел

Подсказка:

Смотрите на количество параметров и возвращаемое значение.

- `x => x * 2` — Умножение числа на 2
- `(a,b) => a + b` — Суммирование двух чисел
- `str => str.Length` — Вычисление длины строки
- `n => n % 2 == 0` — Проверка, является ли число четным

Шаг 06:

Миссия 6

 Сопоставьте локальные функции с их использованием:

Функции:

Ответ:

- `int Factorial(int n)`
- `bool IsPrime(int n)`
- `double Power(double x, int y)`
- `string Reverse(string s)`

Использование:

1. Проверка, является ли число простым
2. Вычисление факториала числа
3. Возведение числа в степень
4. Разворот строки

Подсказка:

Название функции обычно отражает её действие.

- **Factorial** — Вычисление факториала числа
- **IsPrime** — Проверка, является ли число простым
- **Power** — Возведение числа в степень
- **Reverse** — Разворот строки

Шаг 07:

Миссия 7

 Сопоставьте тип лямбды с примером:

Тип:

Ответ:

- `Expression Lambda`
- `Statement Lambda`
- `Func`
- `Action`

Пример:

1. `x => x + 1`
2. `x => { return x * 2; }`
3. `x => x * x`
4. `x => Console.WriteLine(x)`

Подсказка:

Expression Lambda — одно выражение, Statement Lambda — блок кода.

- **Expression Lambda** — $x \Rightarrow x + 1$
- **Statement Lambda** — $x \Rightarrow \{ \text{return } x * 2; \}$
- **Func** — $x \Rightarrow x * x$
- **Action** — $x \Rightarrow \text{Console.WriteLine}(x)$

Шаг 08:

Миссия 8

Расположите шаги применения лямбда-выражений для обработки списка:

1. Определение лямбды
2. Применение метода LINQ с лямбдой
3. Обработка результата
4. Вывод на экран

Подсказка:

Сначала создаём лямбду, потом используем её в LINQ, далее обрабатываем и выводим.

Ответ:

- Определение лямбды
- Применение метода LINQ с лямбдой
- Обработка результата
- Вывод на экран

Шаг 09:

Миссия 9

Расположите шаги создания локальной функции для рекурсивного вычисления факториала:

1. Объявление локальной функции внутри метода
2. Определение базового случая
3. Определение рекурсивного вызова
4. Вызов локальной функции и возврат результата

Подсказка:

Сначала создаём функцию, определяем базовый случай, затем рекурсивный вызов и возвращаем результат.

Ответ:

- Объявление локальной функции внутри метода
- Определение базового случая
- Определение рекурсивного вызова
- Вызов локальной функции и возврат результата

Шаг 10:

Миссия 10

Расположите шаги использования лямбда-выражения в фильтрации списка чисел:

1. Определение лямбды для условия фильтрации
2. Применение метода Where с лямбдой

3. Сбор результатов в новый список
4. Вывод результатов

 Подсказка:

Сначала задаем условие, фильтруем, собираем результаты и выводим.

Ответ:

- Определение лямбды для условия фильтрации
- Применение метода Where с лямбдой
- Сбор результатов в новый список
- Вывод результатов

 Шаг 11:

Миссия 11

Расположите шаги применения Statement Lambda с блоком кода:

1. Объявление лямбды с фигурными скобками
2. Включение нескольких операторов внутри блока
3. Возврат результата с помощью return
4. Присвоение лямбды делегату и вызов

 Подсказка:

Statement Lambda использует фигурные скобки для блока кода с return.

Ответ:

- Объявление лямбды с фигурными скобками
- Включение нескольких операторов внутри блока
- Возврат результата с помощью return
- Присвоение лямбды делегату и вызов

 Шаг 12:

Миссия 12

Что такое лямбда-выражение в C#?

Подсказка: Представьте компактный способ описания функции прямо в коде.

Ответ:

- **Анонимная функция, которую можно передать как значение**
- Класс для работы с коллекциями
- Специальный тип исключения
- Модификатор доступа для методов

 Шаг 13:

Миссия 13

Для чего используют локальные функции в C#?

Подсказка: Подумайте о способе скрыть вспомогательную логику внутри метода.

Ответ:

- **Чтобы определить функцию внутри метода и ограничить её область видимости**
- Чтобы определить глобальную функцию в проекте
- Чтобы ускорить выполнение метода

- Чтобы автоматически генерировать события

🔗 Шаг 14:

Миссия 14

📖 Сопоставьте понятие с правильным объяснением:

Ответ:

- Лямбда-выражение
 - Локальная функция
 - Expression-bodied метод
 - Func делегат
1. Компактная анонимная функция, используемая как значение
 2. Функция, определённая внутри другого метода
 3. Метод с сокращённой записью через =>
 4. Тип делегата для функций с возвращаемым значением

Подсказка: Вспомните, как компактно и локально можно определить логику в C#.

- **Лямбда-выражение** — Компактная анонимная функция, используемая как значение
- **Локальная функция** — Функция, определённая внутри другого метода
- **Expression-bodied метод** — Метод с сокращённой записью через =>
- **Func делегат** — Тип делегата для функций с возвращаемым значением

🔗 Шаг 15:

Миссия 15

Что является преимуществом лямбда-выражений?

Подсказка: Подумайте о сокращении кода и передаче функций как аргументов.

Ответ:

- **Позволяют передавать функциональность как значение и сокращают код**
- Ускоряют работу CPU автоматически
- Создают новый тип данных
- Автоматически проверяют ошибки компиляции

🔗 Шаг 16:

Миссия 16

📖 Сопоставьте синтаксис с назначением:

Ответ:

- `(x, y) => x + y`
 - `int Sum(int a, int b) => a + b;`
 - `void LocalPrint(string msg) { Console.WriteLine(msg); }`
 - `Func adder = (x,y) => x+y;`
1. Лямбда-выражение для сложения
 2. Expression-bodied метод
 3. Локальная функция внутри метода
 4. Использование Func делегата

Подсказка: Сравните синтаксис и контекст использования.

- **(x, y) => x + y** — Лямбда-выражение для сложения
- **int Sum(int a, int b) => a + b;** — Expression-bodied метод
- **void LocalPrint(string msg) { Console.WriteLine(msg); }** — Локальная функция внутри метода
- **Func adder = (x,y) => x+y;** — Использование Func делегата

Урок 3 - Pattern matching и null-операторы

Шаг 01:

Миссия 1

Алекс хочет быстро проверить, все ли числа в списке положительные. Какой синтаксис лямбда-выражения ему подходит?

Подсказка: Лямбда-выражение обычно имеет вид параметр => выражение.

Ответ:

- **x => x > 0**
- (x) { return x > 0 }
- lambda x: x > 0
- def Check(x): return x > 0

Шаг 02:

Миссия 2

Боб сортирует список имен по длине. Какой LINQ-метод ему нужен с лямбдой?

Подсказка: Обратите внимание на методы, которые принимают функцию проекции.

Ответ:

- **OrderBy**
- Where
- Select
- Aggregate

Шаг 03:

Миссия 3

Клара создает локальную функцию внутри метода для вычисления факториала. Как правильно её объявить?

Подсказка: Локальная функция выглядит как обычный метод, но находится внутри метода.

Ответ:

- **int Factorial(int n) { if(n<=1) return 1; return n*Factorial(n-1); }**
- delegate int Factorial(int n);
- lambda int Factorial(int n) => n*Factorial(n-1);
- Func Factorial = n => n*Factorial(n-1);

Шаг 04:

Миссия 4

Даниэль хочет создать лямбду, суммирующую два числа. Какой делегат использовать?

Подсказка: Если лямбда возвращает значение, используйте Func.

Ответ:

- **Func**
- Action
- Predicate
- EventHandler

Шаг 05:

Миссия 5

 Сопоставьте лямбда-выражение с его назначением:

Ответ:

- $x \Rightarrow x * 2$
- $(a,b) \Rightarrow a + b$
- $str \Rightarrow str.Length$
- $n \Rightarrow n \% 2 == 0$

1. Проверка, является ли число четным
2. Умножение числа на 2
3. Вычисление длины строки
4. Суммирование двух чисел

 **Подсказка:**

Смотрите на количество параметров и возвращаемое значение.

- $x \Rightarrow x * 2$ — Умножение числа на 2
- $(a,b) \Rightarrow a + b$ — Суммирование двух чисел
- $str \Rightarrow str.Length$ — Вычисление длины строки
- $n \Rightarrow n \% 2 == 0$ — Проверка, является ли число четным

Шаг 06:

Миссия 6

 Сопоставьте локальные функции с их назначением:

Ответ:

- Factorial(int n)
- IsPrime(int n)
- Power(double x,int y)
- Reverse(string s)

1. Вычисление факториала числа
2. Проверка, является ли число простым
3. Возведение числа в степень
4. Разворот строки

 **Подсказка:**

Название функции обычно отражает её действие.

- **Factorial** — Вычисление факториала числа
- **IsPrime** — Проверка, является ли число простым
- **Power** — Возведение числа в степень
- **Reverse** — Разворот строки

🔗 Шаг 07:

Миссия 7

Расположите шаги применения лямбда-выражения к списку чисел:

1. Определение лямбды
2. Применение метода LINQ с лямбдой
3. Обработка результата
4. Вывод на экран

🔗 Подсказка:

Сначала создаём лямбду, затем используем её, обрабатываем и выводим.

Ответ:

- Определение лямбды
- Применение метода LINQ с лямбдой
- Обработка результата
- Вывод на экран

🔗 Шаг 08:

Миссия 8

Расположите шаги создания локальной функции для рекурсивного вычисления факториала:

1. Объявление локальной функции внутри метода
2. Определение базового случая
3. Определение рекурсивного вызова
4. Вызов локальной функции и возврат результата

🔗 Подсказка:

Сначала создаём функцию, затем базовый случай, рекурсивный вызов и возвращаем результат.

Ответ:

- Объявление локальной функции внутри метода
- Определение базового случая
- Определение рекурсивного вызова
- Вызов локальной функции и возврат результата

🔗 Шаг 09:

Миссия 9

Что делает оператор null-объединения ?? в C#?

Подсказка: Он позволяет подставить значение по умолчанию, если переменная равна null.

Ответ:

- **Возвращает левый операнд, если он не null, иначе правый**
- Сравнивает два значения на null
- Вызывает исключение, если значение null
- Присваивает null всегда

🔗 Шаг 10:

Миссия 10

Что делает оператор ?. (null-условный оператор) в C#?

Подсказка: Он позволяет безопасно обращаться к члену объекта, который может быть null.

Ответ:

- **Вызывает метод или обращается к свойству, если объект не null, иначе возвращает null**
- Присваивает объекту значение null
- Проверяет тип объекта на null
- Создает новый объект, если старый null

Шаг 11:

Миссия 11

 Сопоставьте оператор с его назначением:

Ответ:

- ??
 - ?.
 - is
 - switch с паттернами
1. Возвращает левый операнд, если он не null, иначе правый
 2. Безопасное обращение к члену объекта, который может быть null
 3. Проверяет соответствие объекта типу
 4. Позволяет проверять значения на соответствие различным шаблонам

Подсказка: Вспомните, как операторы null и pattern matching упрощают проверки.

- **??** — Возвращает левый операнд, если он не null, иначе правый
- **?.** — Безопасное обращение к члену объекта, который может быть null
- **is** — Проверяет соответствие объекта типу
- **switch с паттернами** — Позволяет проверять значения на соответствие различным шаблонам

Шаг 12:

Миссия 12

Что делает оператор null-условного присваивания ??.= в C#?

Подсказка: Он присваивает значение только если переменная равна null.

Ответ:

- **Присваивает значение только если текущая переменная равна null**
- Всегда присваивает новое значение
- Проверяет значение на null, но не присваивает
- Удаляет переменную, если она null

Шаг 13:

Миссия 13

 Сопоставьте тип проверки с примером pattern matching:

Ответ:

- Типовая проверка
- Сопоставление с константой
- Сопоставление с типом

- Switch pattern matching

1. if (x != null)
2. if (x is 5)
3. if (x is string s)
4. switch(x) { case int i: ... }

Подсказка: Смотрите на структуру условий и тип проверки.

- **Типовая проверка** — if (x != null)
- **Сопоставление с константой** — if (x is 5)
- **Сопоставление с типом** — if (x is string s)
- **Switch pattern matching** — switch(x) { case int i: ... }

Шаг 14:

Миссия 14

Почему null-операторы упрощают работу с объектами?

Подсказка: Подумайте о сокращении проверок на null и предотвращении исключений.

Ответ:

- **Позволяют безопасно работать с объектами, которые могут быть null, сокращая код**
- Всегда вызывают исключение при null
- Присваивают значение по умолчанию независимо от условия
- Автоматически конвертируют null в пустую строку

Шаг 15:

Миссия 15

Расположите шаги проверки переменной с использованием null-операторов и pattern matching:

1. Используем ?. чтобы безопасно обратиться к члену объекта
2. Применяем ?? для подстановки значения по умолчанию
3. Используем is для проверки типа переменной
4. Применяем switch с паттернами для сложной логики

Подсказка: Сначала нужно безопасно обратиться к объекту, потом обработать null, а затем проверять тип и сложные условия.

Ответ:

- Используем ?. чтобы безопасно обратиться к члену объекта
- Применяем ?? для подстановки значения по умолчанию
- Используем is для проверки типа переменной
- Применяем switch с паттернами для сложной логики

Урок 4 - Атрибуты и Reflection

Шаг 01:

Миссия 1

Алиса обрабатывает объект типа Shape и хочет проверить, является ли он Circle с помощью pattern matching. Какой синтаксис верный?

Подсказка: Используйте ключевое слово 'is' для проверки типа и привязки переменной.

Ответ:

- **if (shape is Circle c) { ... }**
- if (shape as Circle c) { ... }
- if (shape == Circle c) { ... }
- if (shape ? Circle c) { ... }

Шаг 02:

Миссия 2

Боб хочет безопасно получить длину строки, которая может быть null, используя null-оператор. Какой вариант правильный?

Подсказка: Смотрите на оператор ?. и ?? для безопасной работы с null.

Ответ:

- **int len = str?.Length ?? 0;**
- int len = str.Length;
- int len = str!.Length;
- int len = str?.Length;

Шаг 03:

Миссия 3

Клара использует switch с pattern matching для обработки фигур. Как правильно проверить, что shape — Rectangle?

Подсказка: Каждое case может содержать проверку типа с присвоением переменной.

Ответ:

- **switch(shape) { case Rectangle r: ... break; }**
- switch(shape) { case shape is Rectangle r: ... break; }
- switch(shape) { case Rectangle: ... break; }
- switch(shape) { case r = Rectangle: ... break; }

Шаг 04:

Миссия 4

Даниэль хочет использовать null-условный оператор для вызова метода объекта, который может быть null. Как правильно?

Подсказка: ?. позволяет безопасно вызвать метод только если объект не null.

Ответ:

- **obj?.DoSomething();**
- obj.DoSomething();
- obj?DoSomething();
- obj!.DoSomething();

Шаг 05:

Миссия 5

 Сопоставьте pattern matching с его действием:

Ответ:

- x is int i

- x is string s
- x is null
- x is double d

1. Привязка объекта к переменной типа double
2. Проверка на null
3. Привязка объекта к переменной типа string
4. Привязка объекта к переменной типа int

Подсказка:

Обратите внимание на ключевое слово 'is' и тип переменной.

- **x is int i** — Привязка объекта к переменной типа int
- **x is string s** — Привязка объекта к переменной типа string
- **x is null** — Проверка на null
- **x is double d** — Привязка объекта к переменной типа double

Шаг 06:

Миссия 6

 Сопоставьте null-оператор с его функцией:

Ответ:

- ?.
- ??
- ??=
- !

1. Присвоение значения переменной только если она равна null
2. Безопасный вызов метода или свойства, если объект не null
3. Значение по умолчанию, если выражение null
4. Подавление предупреждения компилятора о возможном null

Подсказка:

Смотрите на назначение каждого оператора в работе с null.

- **?.** — Безопасный вызов метода или свойства, если объект не null
- **??** — Значение по умолчанию, если выражение null
- **??=** — Присвоение значения переменной только если она равна null
- **!** — Подавление предупреждения компилятора о возможном null

Шаг 07:

Миссия 7

 Сопоставьте конструкции C# с их назначением:

Ответ:

- switch(x)
- if(x is int i)
- str?.Length
- value ?? 10

1. Фильтрация по типу с привязкой переменной
2. Выбор действия по значению переменной

3. Безопасное обращение к свойству nullable объекта
4. Использование значения по умолчанию при null

🔗 Подсказка:

Каждая конструкция имеет своё специфическое поведение с типами и null.

- **switch(x)** — Выбор действия по значению переменной
- **if(x is int i)** — Фильтрация по типу с привязкой переменной
- **str?.Length** — Безопасное обращение к свойству nullable объекта
- **value ?? 10** — Использование значения по умолчанию при null

🔗 Шаг 08:

Миссия 8

Расположите шаги безопасной работы с nullable объектом:

1. Проверка объекта на null или использование ?.
2. Вызов метода или свойства объекта
3. Использование значения по умолчанию при необходимости (??)
4. Присвоение результата переменной

🔗 Подсказка:

Сначала безопасно получаем доступ, затем обрабатываем результат.

Ответ:

- Проверка объекта на null или использование ?.
- Вызов метода или свойства объекта
- Использование значения по умолчанию при необходимости (??)
- Присвоение результата переменной

🔗 Шаг 09:

Миссия 9

Расположите шаги использования pattern matching в switch:

1. Определение switch по объекту
2. Добавление case с проверкой типа и привязкой переменной
3. Выполнение действий внутри case
4. Добавление default на случай других типов

🔗 Подсказка:

Сначала выбираем объект, затем проверяем тип, выполняем действия и добавляем default.

Ответ:

- Определение switch по объекту
- Добавление case с проверкой типа и привязкой переменной
- Выполнение действий внутри case
- Добавление default на случай других типов

🔗 Шаг 10:

Миссия 10

Для чего используются атрибуты в C#?

Подсказка: Подумайте, как можно добавить метаданные к классам и методам.

Ответ:

- **Для добавления метаданных к классам, методам или свойствам**
- Для выполнения арифметических операций
- Для управления памятью объектов
- Для компиляции в другой язык программирования

Шаг 11:

Миссия 11

Что делает класс System.Reflection в C#?

Подсказка: Подумайте о динамическом исследовании структуры программы во время выполнения.

Ответ:

- **Позволяет исследовать метаданные и получать информацию о типах во время выполнения**
- Автоматически компилирует проект в DLL
- Сохраняет состояние объекта на диск
- Обеспечивает многопоточную синхронизацию объектов

Шаг 12:

Миссия 12

 Сопоставьте понятие с его назначением в контексте атрибутов и Reflection:

Ответ:

- Атрибут
 - Reflection
 - MethodInfo
 - PropertyInfo
1. Метаданные, добавляемые к классам или методам
 2. Позволяет исследовать типы и члены во время выполнения
 3. Представление метода для анализа через Reflection
 4. Представление свойства для анализа через Reflection

Подсказка: Смотрите на роли элементов: что описывает, а что исследует.

- **Атрибут** — Метаданные, добавляемые к классам или методам
- **Reflection** — Позволяет исследовать типы и члены во время выполнения
- **MethodInfo** — Представление метода для анализа через Reflection
- **PropertyInfo** — Представление свойства для анализа через Reflection

Шаг 13:

Миссия 13

Как получить список всех методов класса через Reflection?

Подсказка: Сначала получите объект Type класса.

Ответ:

- **type.GetMethods()**
- class.GetProperties()
- type.Methods()

- `Reflection.GetMethods(class)`

Шаг 14:

Миссия 14

 Сопоставьте класс `Reflection` с его функционалом:

Ответ:

- `Type`
 - `MethodInfo`
 - `PropertyInfo`
 - `Attribute`
1. Представление типа и его структуры
 2. Метаданные метода
 3. Метаданные свойства
 4. Метаданные, добавленные программистом

Подсказка: Смотрите на различия между типами метаданных и объектами для анализа.

- **`Type`** — Представление типа и его структуры
- **`MethodInfo`** — Метаданные метода
- **`PropertyInfo`** — Метаданные свойства
- **`Attribute`** — Метаданные, добавленные программистом

Шаг 15:

Миссия 15

Что позволяет делать `Reflection` с приватными членами класса?

Подсказка: Думайте о получении информации о членах, доступ к которым обычно ограничен.

Ответ:

- **Получать информацию и вызывать методы/свойства даже если они приватные**
- Только видеть публичные методы
- Автоматически делает их публичными
- Удаляет приватные методы из класса

Урок 5 - Indexers и Operator Overloading

Шаг 01:

Миссия 1

Алекс создает класс `Matrix` и хочет реализовать индексатор для доступа к элементам по строкам и столбцам. Какой синтаксис правильный?

Подсказка: Используйте ключевое слово `'this'` для создания индексатора.

Ответ:

- **`public int this[int row, int col] { get { ... } set { ... } }`**
- `public int Matrix[int row, int col] { get; set; }`
- `public int this[row, col] { get; set; }`
- `public int this[int r, c] { get; set; }`

Шаг 02:

Миссия 2

Бетти хочет перегрузить оператор '+' для класса Vector, чтобы складывать два вектора. Какой правильный синтаксис?

Подсказка: Оператор должен быть статическим и возвращать новый объект.

Ответ:

- **public static Vector operator +(Vector a, Vector b) { ... }**
- public Vector operator +(Vector a, Vector b) { ... }
- public static Vector + (Vector a, Vector b) { ... }
- public Vector + (Vector a, Vector b) { ... }

Шаг 03:

Миссия 3

Крис хочет запретить использование оператора '-' для своего класса Money. Какой подход верный?

Подсказка: Просто не перегружайте оператор, и он будет недоступен.

Ответ:

- **Не перегружать оператор '-'**
- Сделать оператор приватным
- Использовать 'delete operator -'
- Перегрузить оператор с throw Exception

Шаг 04:

Миссия 4

Дэвид реализует класс Table с индексатором. Он хочет, чтобы попытка обратиться к несуществующему индексу возвращала 0. Как это сделать?

Подсказка: Можно использовать проверку индекса в get-блоке.

Ответ:

- **get { return (row >=0 && row < rows) ? data[row] : 0; }**
- get { return data[row]; }
- get { return null; }
- get { return default; }

Шаг 05:

Миссия 5

 Сопоставьте индексаторы и перегрузки операторов с их функцией:

Ответ:

- this[int i]
 - operator +
 - operator ==
 - this[string key]
1. Сравнение двух объектов на равенство
 2. Доступ к элементу по целочисленному индексу
 3. Сложение двух объектов одного типа

4. Доступ к элементу по строковому ключу

Подсказка:

Помните, что индексаторы используют 'this', а операторы перегружаются через 'operator'.

- **this[int i]** — Доступ к элементу по целочисленному индексу
- **operator +** — Сложение двух объектов одного типа
- **operator ==** — Сравнение двух объектов на равенство
- **this[string key]** — Доступ к элементу по строковому ключу

Шаг 06:

Миссия 6

 Сопоставьте тип перегрузки оператора с примером:

Ответ:

- Унарный оператор
- Бинарный оператор
- Логический оператор
- Сравнения

1. +a или -a
2. a + b
3. a && b
4. a == b

Подсказка:

Унарный оператор работает с одним объектом, бинарный — с двумя.

- **Унарный оператор** — +a или -a
- **Бинарный оператор** — a + b
- **Логический оператор** — a && b
- **Сравнения** — a == b

Шаг 07:

Миссия 7

 Сопоставьте сценарий с подходящей функцией:

Ответ:

- Объект ListWrapper
- Объект DictionaryWrapper
- Объект Money
- Объект Vector

1. Используем индексатор с int
2. Используем индексатор с string
3. Перегрузка оператора '+' и '-'
4. Перегрузка оператора '*' и '/'

Подсказка:

Подумайте, какие типы данных удобно индексировать числами или строками.

- **Объект ListWrapper** — Используем индексатор с int
- **Объект DictionaryWrapper** — Используем индексатор с string

- **Объект Money** — Перегрузка оператора '+' и '-'
- **Объект Vector** — Перегрузка оператора '*' и '/'

🔮 Шаг 08:

Миссия 8

Для чего используются индексаторы (Indexers) в C#?

Подсказка: Подумайте о доступе к элементам объекта как к элементам массива.

Ответ:

- **Для предоставления доступа к элементам объекта через синтаксис массива**
- Для определения арифметических операций с объектами
- Для обработки исключений
- Для создания статических методов класса

🔮 Шаг 09:

Миссия 9

Что позволяет делать перегрузка операторов (Operator Overloading) в C#?

Подсказка: Подумайте о возможности определить поведение стандартных операторов для своих классов.

Ответ:

- **Определять поведение стандартных операторов для пользовательских классов**
- Создавать новые ключевые слова языка
- Автоматически оптимизировать память объектов
- Обеспечивать многопоточную безопасность класса

🔮 Шаг 10:

Миссия 10

📄 Сопоставьте понятие с его назначением:

Ответ:

- Индексатор
- Перегрузка оператора
- `get`
- `set`

1. Позволяет обращаться к объекту как к массиву
2. Определяет поведение оператора для пользовательского класса
3. Метод для получения значения элемента индексатора
4. Метод для установки значения элемента индексатора

Подсказка: Смотрите на роли: доступ к данным vs изменение данных, операторы vs массивоподобный доступ.

- **Индексатор** — Позволяет обращаться к объекту как к массиву
- **Перегрузка оператора** — Определяет поведение оператора для пользовательского класса
- **`get`** — Метод для получения значения элемента индексатора
- **`set`** — Метод для установки значения элемента индексатора

Шаг 11:

Миссия 11

Как объявляется индексатор в классе C#?

Подсказка: Используется ключевое слово `this` с параметром.

Ответ:

- `public int this[int index] { get; set; }`
- `public int Indexer() { get; set; }`
- `public int array[int index] { get; set; }`
- `public void this[int index]() { get; set; }`

Шаг 12:

Миссия 12

Что происходит, если не реализовать `set` в индексаторе?

Подсказка: Подумайте, можно ли изменять элементы.

Ответ:

- **Индексатор становится только для чтения**
- Индексатор нельзя использовать вообще
- Произойдет ошибка компиляции в любом случае
- Элементы можно изменять напрямую через поле

Шаг 13:

Миссия 13

Расположите шаги для создания перегрузки арифметического оператора для класса `Vector`:

1. Определить сигнатуру оператора с ключевым словом `operator`
2. Реализовать логику сложения или вычитания внутри метода
3. Вернуть новый объект класса с результатом операции
4. Использовать оператор с объектами класса в коде

Подсказка: Сначала создаем метод, потом реализуем логику, возвращаем объект и только потом применяем оператор.

Ответ:

- Определить сигнатуру оператора с ключевым словом `operator`
- Реализовать логику сложения или вычитания внутри метода
- Вернуть новый объект класса с результатом операции
- Использовать оператор с объектами класса в коде

Шаг 14:

Миссия 14

Расположите шаги для создания индексатора в классе:

1. Создать приватное поле для хранения данных
2. Объявить индексатор с `get` и/или `set`
3. Реализовать логику доступа к элементам через индексатор
4. Использовать индексатор в коде для чтения или записи элементов

Подсказка: Сначала данные, потом объявление, затем логика доступа и использование.

Ответ:

- Создать приватное поле для хранения данных
- Объявить индексатор с get и/или set
- Реализовать логику доступа к элементам через индексатор
- Использовать индексатор в коде для чтения или записи элементов

Модуль 2 - Работа с памятью и оптимизация

Урок 1 - IDisposable и управление ресурсами

Шаг 01:

Миссия 1

Аня открывает файл с помощью FileStream. Как правильно освободить ресурс после работы?

Подсказка: Используйте конструкцию, которая автоматически закрывает ресурс.

Ответ:

- **Использовать 'using' блок**
- Вызывать Close() вручную без блока using
- Не освобождать ресурс, GC справится сам
- Вызывать Dispose() после каждого чтения

Шаг 02:

Миссия 2

Борис реализует класс DatabaseConnection и хочет, чтобы ресурсы закрывались корректно. Какой интерфейс нужно реализовать?

Подсказка: Он нужен для явного освобождения ресурсов.

Ответ:

- **IDisposable**
- IEnumerable
- ICloneable
- IComparable

Шаг 03:

Миссия 3

Виктор создает класс с сетевым соединением. Какой метод нужно обязательно реализовать для IDisposable?

Подсказка: Этот метод вызывается для очистки ресурсов.

Ответ:

- **Dispose()**
- Finalize()
- Close()
- Clear()

Шаг 04:

Миссия 4

Галина использует FileStream без блока using. Что может произойти?

Подсказка: Ресурс может остаться открытым до вызова сборщика мусора.

Ответ:

- **Файл может остаться заблокированным**
- Файл сразу закроется автоматически
- Ничего не изменится
- Программа выбросит ошибку компиляции

Шаг 05:

Миссия 5

 Сопоставьте тип ресурса с правильным действием при использовании IDisposable:

Ответ:

- Файл
 - Сетевое соединение
 - Память unmanaged
 - База данных
1. Закрыть FileStream с помощью Dispose
 2. Закрыть соединение методом Dispose или using
 3. Освободить память с помощью Dispose и финализатора
 4. Закрыть соединение с БД через Dispose

 **Подсказка:**

Подумайте, какие ресурсы требуют явного освобождения.

- **Файл** — Закрыть FileStream с помощью Dispose
- **Сетевое соединение** — Закрыть соединение методом Dispose или using
- **Память unmanaged** — Освободить память с помощью Dispose и финализатора
- **База данных** — Закрыть соединение с БД через Dispose

Шаг 06:

Миссия 6

 Сопоставьте сценарий с методом освобождения ресурсов:

Ответ:

- Чтение файла
 - Работа с SQL соединением
 - Сетевой сокет
 - Объект с unmanaged памятью
1. using
 2. Dispose()
 3. using или Dispose()
 4. Dispose() + финализатор

 **Подсказка:**

Подумайте, какие объекты можно закрывать через using, а какие требуют дополнительной очистки.

- **Чтение файла** — using
- **Работа с SQL соединением** — Dispose()

- **Сетевой сокет** — using или Dispose()
- **Объект с unmanaged памятью** — Dispose() + финализатор

Шаг 07:

Миссия 7

 Сопоставьте ключевое слово с его действием при управлении ресурсами:

Ответ:

- using
 - Dispose()
 - finalize
 - try-finally
1. Автоматическое вызовы Dispose в конце блока
 2. Явный вызов метода освобождения ресурсов
 3. Очистка unmanaged ресурсов при сборке мусора
 4. Гарантированное освобождение ресурса независимо от исключений

Подсказка:

Подумайте, как управлять ресурсами безопасно и гарантированно.

- **using** — Автоматическое вызовы Dispose в конце блока
- **Dispose()** — Явный вызов метода освобождения ресурсов
- **finalize** — Очистка unmanaged ресурсов при сборке мусора
- **try-finally** — Гарантированное освобождение ресурса независимо от исключений

Шаг 08:

Миссия 8

Расположите действия для корректного использования IDisposable в правильном порядке:

1. Создать объект ресурса
2. Выполнить операции с ресурсом
3. Закрыть ресурс через Dispose или using
4. Обработка исключений при работе с ресурсом

Подсказка:

Сначала создаем ресурс, потом работаем, не забывая об исключениях, и в конце освобождаем.

Ответ:

- Создать объект ресурса
- Выполнить операции с ресурсом
- Обработка исключений при работе с ресурсом
- Закрыть ресурс через Dispose или using

Шаг 09:

Миссия 9

Что делает интерфейс IDisposable в C#?

Подсказка: Подумайте о правильном освобождении ресурсов, таких как файлы и соединения с базой данных.

Ответ:

- **Позволяет освободить неуправляемые ресурсы вручную**
- Автоматически делает объект потокобезопасным
- Отслеживает использование памяти объектами
- Создает резервную копию объекта

Шаг 10:

Миссия 10

Что делает конструкция using в C#?

Подсказка: Смотрите на ресурсы, которые нужно обязательно освободить после использования.

Ответ:

- **Автоматически вызывает Dispose после окончания блока кода**
- Запускает объект в отдельном потоке
- Перегружает операторы для ресурса
- Сохраняет объект в кэш памяти

Шаг 11:

Миссия 11

 Сопоставьте понятие с его назначением:

Ответ:

- IDisposable
 - using
 - Dispose()
 - Неуправляемые ресурсы
1. Интерфейс для освобождения ресурсов вручную
 2. Блок, который автоматически вызывает Dispose()
 3. Метод для освобождения ресурсов объекта
 4. Ресурсы, не управляемые сборщиком мусора

Подсказка: Обратите внимание на то, кто что освобождает: код, сборщик мусора или объект.

- **IDisposable** — Интерфейс для освобождения ресурсов вручную
- **using** — Блок, который автоматически вызывает Dispose()
- **Dispose()** — Метод для освобождения ресурсов объекта
- **Неуправляемые ресурсы** — Ресурсы, не управляемые сборщиком мусора

Шаг 12:

Миссия 12

Почему важно реализовывать Dispose в классе, который использует файловые потоки?

Подсказка: Подумайте о том, что произойдет, если поток останется открытым.

Ответ:

- **Чтобы освободить системные ресурсы и закрыть файлы**
- Чтобы ускорить сборку мусора
- Чтобы автоматически сохранять файл в облаке
- Чтобы исключить создание потоков в памяти

Шаг 13:

Миссия 13

 Сопоставьте понятие с примером использования:

Ответ:

- using
 - Dispose()
 - Неуправляемый ресурс
 - GC.Collect()
1. Блок, который автоматически закрывает StreamReader после использования
 2. Метод, который закрывает соединение с базой данных
 3. Файловый дескриптор или сетевой сокет
 4. Принудительный запуск сборщика мусора

Подсказка: Смотрите на действие: автоматическое vs ручное, ресурсы vs сборка мусора.

- **using** — Блок, который автоматически закрывает StreamReader после использования
- **Dispose()** — Метод, который закрывает соединение с базой данных
- **Неуправляемый ресурс** — Файловый дескриптор или сетевой сокет
- **GC.Collect()** — Принудительный запуск сборщика мусора

Шаг 14:

Миссия 14

Расположите шаги правильного использования IDisposable для класса с сетевым подключением:

1. Создать объект подключения
2. Использовать подключение внутри блока using
3. После выхода из блока using автоматически вызовется Dispose
4. Ресурс освобождается и соединение закрывается

Подсказка: Сначала создаем, потом используем, потом автоматически закрываем, затем ресурс освобождается.

Ответ:

- Создать объект подключения
- Использовать подключение внутри блока using
- После выхода из блока using автоматически вызовется Dispose
- Ресурс освобождается и соединение закрывается

Шаг 15:

Миссия 15

Что может произойти, если забыть вызвать Dispose для объекта, использующего неуправляемый ресурс?

Подсказка: Подумайте о ресурсах операционной системы.

Ответ:

- **Может произойти утечка ресурсов и проблемы с производительностью**
- Объект автоматически удалится без последствий
- Компилятор выдаст ошибку
- Программа станет потокобезопасной

Шаг 16:

Миссия 16

Расположите шаги правильного освобождения ресурсов в классе с файловыми потоками:

1. Открытие файла
2. Работа с файлом (чтение/запись)
3. Вызов Dispose или выход из блока using
4. Заккрытие файла и освобождение ресурсов

Подсказка: Думайте логично: сначала открытие, работа, освобождение ресурсов.

Ответ:

- Открытие файла
- Работа с файлом (чтение/запись)
- Вызов Dispose или выход из блока using
- Заккрытие файла и освобождение ресурсов

Урок 2 - Структуры, классы и ref struct

Шаг 01:

Миссия 1

Вы пишете игру, и нужно выбрать между struct и class для небольшого объекта данных. Что правильнее выбрать для лёгкой копируемой сущности?

Подсказка: Структуры хорошо подходят для маленьких объектов без сложной логики.

Ответ:

- **struct**
- class
- interface
- enum

Шаг 02:

Миссия 2

Вы заметили, что класс занимает много памяти из-за большого количества небольших полей. Что поможет уменьшить использование памяти?

Подсказка: Иногда стоит использовать структуры или упорядочивать поля для выравнивания.

Ответ:

- **Использовать struct для маленьких объектов**
- **Оптимизировать порядок полей для уменьшения выравнивания**
- Добавить больше свойств и методов
- Игнорировать память и продолжать использовать class

Шаг 03:

Миссия 3

Вы используете ref struct для работы с большим массивом данных. Какое ограничение нужно учитывать?

Подсказка: ref struct нельзя хранить в куче, они должны оставаться на стеке.

Ответ:

- **ref struct нельзя хранить в куче**
- ref struct можно наследовать
- ref struct может иметь конструктор без параметров
- ref struct можно использовать с async/await

Шаг 04:

Миссия 4

Вы создаёте игровой объект, который должен часто копироваться. Какой тип данных снизит нагрузку на GC?

Подсказка: Меньше объектов в куче — меньше работы сборщика мусора.

Ответ:

- **struct**
- class
- interface
- delegate

Шаг 05:

Миссия 5

Вы пишете код для высокопроизводительной игры. Что важно для ref struct?

Подсказка: Они идеально подходят для временных данных и буферов.

Ответ:

- **ref struct всегда на стеке**
- **ref struct нельзя передавать в async методы**
- ref struct можно хранить в class как поле
- ref struct поддерживает наследование

Шаг 06:

Миссия 6

 Сопоставьте понятие с его описанием:

Понятия:

Ответ:

- Struct
- Class
- Ref struct
- Boxing

Определения:

1. Тип значения, который хранится на стеке.
2. Тип ссылки, который хранится в куче.
3. Структура, которая всегда остаётся на стеке и не может быть в куче.
4. Превращение struct в объект для хранения в куче.

Подсказка:

Подумайте, где находятся данные: стек или куча, и что происходит при копировании и преобразовании.

- **Struct** — Тип значения, который хранится на стеке.
- **Class** — Тип ссылки, который хранится в куче.
- **Ref struct** — Структура, которая всегда остаётся на стеке и не может быть в куче.
- **Boxing** — Превращение struct в объект для хранения в куче.

Шаг 07:

Миссия 7

 Сопоставьте понятие с примером:

Понятия:

Ответ:

- Immutable struct
- Mutable struct
- Value type copy
- Reference type copy

Определения:

1. Структура, поля которой нельзя изменить после создания.
2. Структура, поля которой можно менять.
3. При копировании создаётся новая независимая копия.
4. При копировании создаётся ссылка на один объект.

Подсказка:

Подумайте о поведении struct и class при присваивании и изменении данных.

- **Immutable struct** — Структура, поля которой нельзя изменить после создания.
- **Mutable struct** — Структура, поля которой можно менять.
- **Value type copy** — При копировании создаётся новая независимая копия.
- **Reference type copy** — При копировании создаётся ссылка на один объект.

Шаг 08:

Миссия 8

 Сопоставьте понятие с его особенностью:

Понятия:

Ответ:

- Stack
- Heap
- Garbage Collector
- Ref struct lifetime

Определения:

1. Место, где хранятся временные объекты и локальные переменные.
2. Место, где хранятся объекты ссылочного типа.
3. Система автоматического управления памятью для объектов в куче.
4. Срок жизни ограничен контекстом стека, нельзя хранить в куче.

Подсказка:

Подумайте, какие объекты живут на стеке, какие — в куче и как это связано с GC.

- **Stack** — Место, где хранятся временные объекты и локальные переменные.
- **Heap** — Место, где хранятся объекты ссылочного типа.
- **Garbage Collector** — Система автоматического управления памятью для объектов в куче.
- **Ref struct lifetime** — Срок жизни ограничен контекстом стека, нельзя хранить в куче.

🔗 Шаг 09:

Миссия 9

📄 Сопоставьте сценарий с типом данных:

Понятия:

Ответ:

- Игровая точка (Vector2)
- Большой объект с данными игрока
- Временный буфер для сетевого пакета
- Списки объектов на уровне

Определения:

1. Struct, маленький и часто копируемый
2. Class, хранится в куче, большие данные
3. Ref struct, только на стеке, быстро использовать и удалить
4. Class, множество ссылок и управление через GC

🔗 Подсказка:

Подумайте, где объекты будут часто создаваться и уничтожаться, а где — долго жить.

- **Игровая точка (Vector2)** — Struct, маленький и часто копируемый
- **Большой объект с данными игрока** — Class, хранится в куче, большие данные
- **Временный буфер для сетевого пакета** — Ref struct, только на стеке, быстро использовать и удалить
- **Списки объектов на уровне** — Class, множество ссылок и управление через GC

🔗 Шаг 10:

Миссия 10

Расположите этапы использования struct для оптимизации памяти в правильном порядке:

1. Определение маленьких объектов, часто копируемых
2. Замена class на struct
3. Проверка производительности и использования памяти
4. При необходимости вернуть class для больших объектов

🔗 Подсказка:

Сначала выбираем кандидатов для struct, потом меняем тип, проверяем результат и решаем, где struct не подходит.

Ответ:

- Определение маленьких объектов, часто копируемых
- Замена class на struct
- Проверка производительности и использования памяти
- При необходимости вернуть class для больших объектов

🔮 Шаг 11:

Миссия 11

Расположите шаги оптимизации ref struct для буфера:

1. Определить объём данных и частоту использования
2. Создать ref struct для хранения данных на стеке
3. Использовать временно и удалять сразу после использования
4. Проверить, что объект не хранится в куче и нет ошибок компиляции

🔮 Подсказка:

Подумайте, как ref struct работает: сначала проектируем, потом используем, потом проверяем.

Ответ:

- Определить объём данных и частоту использования
- Создать ref struct для хранения данных на стеке
- Использовать временно и удалять сразу после использования
- Проверить, что объект не хранится в куче и нет ошибок компиляции

🔮 Шаг 12:

Миссия 12

Расположите шаги при оптимизации больших классов с множеством полей:

1. Проанализировать размер и типы полей
2. Разбить класс на несколько меньших структур/классов
3. Перенести временные данные в ref struct
4. Проверить использование памяти и производительность

🔮 Подсказка:

Сначала анализ, потом рефакторинг, потом перенос временных данных, потом контроль результата.

Ответ:

- Проанализировать размер и типы полей
- Разбить класс на несколько меньших структур/классов
- Перенести временные данные в ref struct
- Проверить использование памяти и производительность

🔮 Шаг 13:

Миссия 13

Расположите шаги работы с immutable struct:

1. Создать struct с readonly полями
2. Инициализировать все поля через конструктор
3. Использовать struct без изменения полей
4. При необходимости создавать новые копии с изменёнными данными

🔮 Подсказка:

Immutable struct нельзя менять, поэтому создаём копии для изменений.

Ответ:

- Создать struct с readonly полями

- Инициализировать все поля через конструктор
- Использовать struct без изменения полей
- При необходимости создавать новые копии с изменёнными данными

🔗 Шаг 14:

Миссия 14

Чем отличается структура (struct) от класса (class) в C#?

Подсказка: Обратите внимание на хранение данных и способ передачи объектов.

Ответ:

- **Структуры хранятся в стеке и передаются по значению, классы** — в куче и по ссылке
- **Классы не могут иметь методы, структуры** — могут
- **Структуры поддерживают наследование, классы** — нет
- **Классы хранятся в стеке, структуры** — в куче

🔗 Шаг 15:

Миссия 15

Для чего используют ref struct в C#?

Подсказка: Подумайте о ситуациях, когда нужно ограничить объект стеком.

Ответ:

- **Чтобы объект существовал только в стеке и не попадал в кучу**
- Чтобы объект мог наследовать другие классы
- Чтобы объект был сериализуемым по умолчанию
- Чтобы объект автоматически реализовывал IDisposable

🔗 Шаг 16:

Миссия 16

📄 Сопоставьте понятие с его характеристикой:

Ответ:

- Класс
 - Структура
 - ref struct
 - Передача по значению
1. Объект хранится в куче, передается по ссылке
 2. Хранится в стеке, передается по значению
 3. Объект ограничен стеком и не может быть частью кучи
 4. Создает копию объекта при передаче в метод

Подсказка: Смотрите на место хранения и способ передачи объекта.

- **Класс** — Объект хранится в куче, передается по ссылке
- **Структура** — Хранится в стеке, передается по значению
- **ref struct** — Объект ограничен стеком и не может быть частью кучи
- **Передача по значению** — Создает копию объекта при передаче в метод

🔗 Шаг 17:

Миссия 17

Почему стоит использовать структуры для маленьких неизменяемых объектов?

Подсказка: Обратите внимание на производительность и копирование данных.

Ответ:

- Они быстрее создаются и передаются по значению без нагрузки на кучу
- Они поддерживают наследование и полиморфизм
- Они автоматически управляют памятью через GC
- Они могут быть null по умолчанию

Шаг 18:

Миссия 18

 Сопоставьте тип с особенностью поведения:

Ответ:

- Класс
 - Структура
 - ref struct
 - readonly struct
1. Можно создавать наследников и интерфейсы
 2. Передача всегда копией, изменения не влияют на оригинал
 3. Объект существует только на стеке
 4. Все поля неизменяемы после инициализации

Подсказка: Подумайте о наследовании, копировании и неизменяемости.

- **Класс** — Можно создавать наследников и интерфейсы
- **Структура** — Передача всегда копией, изменения не влияют на оригинал
- **ref struct** — Объект существует только на стеке
- **readonly struct** — Все поля неизменяемы после инициализации

Шаг 19:

Миссия 19

Расположите шаги создания и использования ref struct:

1. Определить ref struct с необходимыми полями
2. Создать экземпляр внутри метода
3. Использовать объект внутри метода
4. После выхода из метода объект удаляется из стека

Подсказка: Думайте, что ref struct живет только в стеке.

Ответ:

- Определить ref struct с необходимыми полями
- Создать экземпляр внутри метода
- Использовать объект внутри метода
- После выхода из метода объект удаляется из стека

Урок 3 - Span и Memory

Шаг 01:

Миссия 1

Вы работаете с большим массивом чисел и хотите быстро получить срез данных без копирования. Что использовать?

Подсказка: Ищите тип, который позволяет работать с частью массива без аллокаций.

Ответ:

- **Span**
- `Array.Copy`
- `List`
- `Queue`

Шаг 02:

Миссия 2

Вы создаёте сервер для обработки сетевых пакетов. Что выбрать для безопасного управления памятью при больших объёмах данных?

Подсказка: `Memory` позволяет хранить данные на куче, но с безопасным доступом к срезам.

Ответ:

- **Memory**
- **Span для локальных срезов**
- `new int[]` каждый раз копировать данные
- `string[]` для бинарных данных

Шаг 03:

Миссия 3

Вы используете `Span` в методе, который должен работать асинхронно. Что нужно помнить?

Подсказка: `Span` живёт только на стеке и не может пересекать `await`.

Ответ:

- **Span нельзя использовать через `await`**
- `Span` можно хранить в классе
- `Span` автоматически копируется в кучу
- `Span` наследует `IDisposable`

Шаг 04:

Миссия 4

Вы хотите обработать большой файл без аллокаций большого массива. Что подходит?

Подсказка: `Memory` позволяет работать с участками данных на куче и передавать между методами.

Ответ:

- **Memory**
- **Span для локальных буферов**
- Создание нового массива на каждый кусок
- Использование `List` и копирование

Шаг 05:

Миссия 5

Вы хотите передавать сегменты массива между методами безопасно и эффективно. Какой тип выбрать?

Подсказка: Memory безопасен для передачи за пределы стека.

Ответ:

- **Memory**
- Span
- ArraySegment
- List

Шаг 06:

Миссия 6

 Сопоставьте тип с его особенностью:

Понятия:

Ответ:

- Span
- Memory
- ReadOnlySpan
- ArraySegment

Определения:

1. Стековый срез данных, нельзя хранить в куче
2. Срез данных, безопасный для хранения в куче и передачи между методами
3. Нельзя изменять данные через этот срез
4. Старый способ среза массива, безопасен, но менее гибок

 **Подсказка:**

Подумайте, что живёт на стеке, что можно хранить, и где нужны изменения.

- **Span** — Стековый срез данных, нельзя хранить в куче
- **Memory** — Срез данных, безопасный для хранения в куче и передачи между методами
- **ReadOnlySpan** — Нельзя изменять данные через этот срез
- **ArraySegment** — Старый способ среза массива, безопасен, но менее гибок

Шаг 07:

Миссия 7

 Сопоставьте метод с его действием:

Понятия:

Ответ:

- Memory.Slice
- Span.ToArray
- ReadOnlySpan.Length
- Memory.Pin

Определения:

1. Создание среза без копирования
2. Создание нового массива с копированием данных
3. Получение длины среза
4. Закрепление памяти для безопасного доступа через указатель

 **Подсказка:**

Подумайте, что создаёт копию, а что просто срез или фиксирует память.

- **Memory.Slice** — Создание среза без копирования
- **Span.ToArray** — Создание нового массива с копированием данных
- **ReadOnlySpan.Length** — Получение длины среза
- **Memory.Pin** — Закрепление памяти для безопасного доступа через указатель

🔗 Шаг 08:

Миссия 8

📄 Сопоставьте тип с его особенностью работы с GC:

Понятия:

Ответ:

- Span
- Memory
- ArraySegment
- ReadOnlySpan

Определения:

1. Не участвует в сборке мусора, живёт на стеке
2. Хранится в куче, GC может управлять памятью
3. Хранится в куче, GC управляет массивом
4. Не изменяемый срез, не влияет на GC

🔗 Подсказка:

Подумайте, какие объекты на стеке, какие на куче, а какие только читаются.

- **Span** — Не участвует в сборке мусора, живёт на стеке
- **Memory** — Хранится в куче, GC может управлять памятью
- **ArraySegment** — Хранится в куче, GC управляет массивом
- **ReadOnlySpan** — Не изменяемый срез, не влияет на GC

🔗 Шаг 09:

Миссия 9

Расположите шаги работы с большим массивом через Span и Memory в правильном порядке:

1. Создать массив данных
2. Создать Span для локальной обработки
3. Использовать Memory для передачи между методами
4. Обработать данные и при необходимости создать копию через ToArray

🔗 Подсказка:

Сначала создаём данные, потом локально обрабатываем, потом безопасно передаём, потом создаём копии при необходимости.

Ответ:

- Создать массив данных
- Создать Span для локальной обработки
- Использовать Memory для передачи между методами
- Обработать данные и при необходимости создать копию через ToArray

🔗 Шаг 10:

Миссия 10

Расположите шаги безопасного использования Memory.Pin:

1. Создать Memory объекта
2. Вызвать Pin для закрепления памяти
3. Использовать указатель внутри блока using
4. Разблокировать память автоматически после выхода из using

🔗 Подсказка:

Сначала выделяем память, потом фиксируем, используем, затем отпускаем автоматически.

Ответ:

- Создать Memory объекта
- Вызвать Pin для закрепления памяти
- Использовать указатель внутри блока using
- Разблокировать память автоматически после выхода из using

🔗 Шаг 11:

Миссия 11

Расположите шаги при обработке больших файлов без лишних аллокаций:

1. Создать буфер Memory для чтения файла
2. Создать Span для локальной обработки части буфера
3. Обработать данные и сохранить результат
4. Повторить для следующей части файла

🔗 Подсказка:

Сначала создаём память, потом срез, обрабатываем, затем повторяем для новых сегментов.

Ответ:

- Создать буфер Memory для чтения файла
- Создать Span для локальной обработки части буфера
- Обработать данные и сохранить результат
- Повторить для следующей части файла

🔗 Шаг 12:

Миссия 12

Расположите шаги оптимизации парсинга строк с использованием Span:

1. Получить ReadOnlySpan для строки
2. Разделить строку на подстроки без копирования
3. Обработать подстроки
4. При необходимости создать новые строки через ToString

🔗 Подсказка:

Сначала получаем срез, затем разделяем, обрабатываем, и только если нужны полноценные строки — копируем.

Ответ:

- Получить ReadOnlySpan для строки

- Разделить строку на подстроки без копирования
- Обработать подстроки
- При необходимости создать новые строки через ToString

🔗 Шаг 13:

Миссия 13

Расположите этапы работы с буферами сетевых пакетов через Span и Memory:

1. Получить Memory из входящего пакета
2. Создать Span для обработки локальной части
3. Обработать данные пакета
4. При необходимости передать Memory дальше без копирования

🔗 Подсказка:

Сначала выделяем память, затем создаём срез, обрабатываем и передаём безопасно.

Ответ:

- Получить Memory из входящего пакета
- Создать Span для обработки локальной части
- Обработать данные пакета
- При необходимости передать Memory дальше без копирования

🔗 Шаг 14:

Миссия 14

Что такое Span в C#?

Подсказка: Подумайте о том, как он работает с памятью и массивами без копирования.

Ответ:

- **Тип, предоставляющий безопасный доступ к непрерывной области памяти без копирования**
- Ссылочный тип, хранящийся в куче
- Структура для работы только с файлами
- Класс для потокового ввода/вывода данных

🔗 Шаг 15:

Миссия 15

Для чего используют Memory вместо Span?

Подсказка: Подумайте о долгоживущих объектах и хранении на куче.

Ответ:

- **Чтобы работать с данными в куче и передавать между методами, сохраняя доступ к памяти**
- Чтобы хранить объекты только в стеке
- Чтобы автоматически управлять потоками ввода/вывода
- Чтобы создавать массивы фиксированной длины на стеке

🔗 Шаг 16:

Миссия 16

📄 Сопоставьте тип с его характеристикой:

Ответ:

- `Span<T>`
- `Memory<T>`
- `ReadOnlySpan<T>`
- `ReadOnlyMemory<T>`

1. Стековый тип, обеспечивает доступ к памяти без копирования
2. Кучевой тип, можно передавать между методами и хранить дольше
3. Стековый тип, доступ только для чтения
4. Кучевой тип, доступ только для чтения, можно передавать между методами

Подсказка: Смотрите на место хранения и возможность изменения данных.

- **Span** — Стековый тип, обеспечивает доступ к памяти без копирования
- **Memory** — Кучевой тип, можно передавать между методами и хранить дольше
- **ReadOnlySpan** — Стековый тип, доступ только для чтения
- **ReadOnlyMemory** — Кучевой тип, доступ только для чтения, можно передавать между методами

Урок 4 - WeakReference и кеширование объектов

Шаг 01:

Миссия 1

Программист создает кеш для часто используемых объектов, но не хочет удерживать их в памяти навсегда. Какой тип лучше использовать?

Подсказка: Ищите тип, который позволяет объекту быть удаленным сборщиком мусора при необходимости.

Ответ:

- **WeakReference**
- StrongReference
- List
- Dictionary

Шаг 02:

Миссия 2

Вы хотите кешировать объекты, которые могут быть удалены сборщиком мусора. Как проверить, доступен ли объект в WeakReference?

Подсказка: WeakReference имеет способ проверить, существует ли целевой объект.

Ответ:

- **Использовать TryGetTarget()**
- Использовать объект напрямую без проверки
- Проверять объект через null-условие
- Вызывать сборку мусора вручную

Шаг 03:

Миссия 3

Разработчик хранит большое количество объектов в кешах и хочет предотвратить OutOfMemoryException. Что поможет?

Подсказка: Слабые ссылки позволяют сборщику мусора удалять объекты при нехватке памяти.

Ответ:

- **Использовать WeakReference для редко используемых объектов**
- **Периодически проверять доступность объектов через TryGetTarget**
- Всегда хранить сильные ссылки на все объекты
- Вызывать GC.Collect после каждого создания объекта

Шаг 04:

Миссия 4

Вы используете кеширование изображений в приложении. Что произойдет, если использовать WeakReference без проверки объекта?

Подсказка: Слабая ссылка может указывать на объект, который уже удален сборщиком мусора.

Ответ:

- **Может возникнуть NullReferenceException**
- Объект всегда будет доступен
- Сборщик мусора игнорирует WeakReference
- Кеш автоматически очищается через WeakReference

Шаг 05:

Миссия 5

Разработчик хочет кешировать данные и обновлять их при удалении объекта сборщиком мусора. Что использовать для уведомления?

Подсказка: WeakReference не предоставляет прямого события удаления, но можно отслеживать доступность объекта.

Ответ:

- **Проверять TryGetTarget и обновлять кеш**
- **Удалять устаревшие данные вручную**
- Ожидать автоматическое уведомление WeakReference
- Сильно ссылаться на объекты, чтобы избежать удаления

Шаг 06:

Миссия 6

 Сопоставьте понятие с правильным объяснением:

Ответ:

- WeakReference
 - StrongReference
 - Кеширование объектов
 - TryGetTarget
1. Ссылка, которая позволяет объекту быть удалённым сборщиком мусора
 2. Обычная ссылка, удерживающая объект в памяти
 3. Механизм хранения часто используемых объектов для быстрого доступа
 4. Метод проверки доступности объекта WeakReference

Подсказка:

Подумайте, какая ссылка позволяет объекту исчезнуть, а какая удерживает его.

- **WeakReference** — Ссылка, которая позволяет объекту быть удалённым сборщиком мусора
- **StrongReference** — Обычная ссылка, удерживающая объект в памяти
- **Кеширование объектов** — Механизм хранения часто используемых объектов для быстрого доступа
- **TryGetTarget** — Метод проверки доступности объекта WeakReference

Шаг 07:

Миссия 7

 Сопоставьте сценарий с правильным подходом:

Ответ:

- Часто используемые большие объекты в приложении
 - Редко используемые объекты
 - Передача объекта между методами
 - Обновление устаревших данных
1. StrongReference и кеширование
 2. WeakReference
 3. Сильная ссылка или Memory
 4. Проверка TryGetTarget и обновление

 **Подсказка:**

Подумайте, где нужна безопасность от удаления, а где наоборот, экономия памяти.

- **Часто используемые большие объекты в приложении** — StrongReference и кеширование
- **Редко используемые объекты** — WeakReference
- **Передача объекта между методами** — Сильная ссылка или Memory
- **Обновление устаревших данных** — Проверка TryGetTarget и обновление

Шаг 08:

Миссия 8

 Сопоставьте проблему с подходом WeakReference:

Ответ:

- Утечка памяти при хранении кеша
 - Доступ к объекту после удаления сборщиком мусора
 - Необходимость обновления устаревших данных
 - Обработка часто используемых объектов
1. Использовать WeakReference для редко используемых объектов
 2. Всегда проверять TryGetTarget
 3. Обновлять данные после проверки доступности
 4. Сильно ссылаться и кешировать

 **Подсказка:**

Подумайте, как избежать утечки памяти и обработать исчезнувшие объекты.

- **Утечка памяти при хранении кеша** — Использовать WeakReference для редко используемых объектов

- **Доступ к объекту после удаления сборщиком мусора** — Всегда проверять TryGetTarget
- **Необходимость обновления устаревших данных** — Обновлять данные после проверки доступности
- **Обработка часто используемых объектов** — Сильно ссылаться и кешировать

🔧 Шаг 09:

Миссия 9

📖 Сопоставьте метод WeakReference с его назначением:

Ответ:

- SetTarget
- TryGetTarget
- IsAlive
- Reset

1. Установить объект для WeakReference
2. Попытаться получить объект, если он не удален сборщиком мусора
3. Проверка, существует ли объект
4. Сбросить ссылку

🔧 Подсказка:

Подумайте, какой метод задаёт объект, а какой проверяет его существование.

- **SetTarget** — Установить объект для WeakReference
- **TryGetTarget** — Попытаться получить объект, если он не удален сборщиком мусора
- **IsAlive** — Проверка, существует ли объект
- **Reset** — Сбросить ссылку

🔧 Шаг 10:

Миссия 10

Расположите шаги безопасного использования WeakReference в правильном порядке:

1. Создать WeakReference на объект
2. Проверить доступность через TryGetTarget
3. Использовать объект, если он доступен
4. При необходимости пересоздать объект и обновить WeakReference

🔧 Подсказка:

Сначала создаём ссылку, потом проверяем, используем, затем обновляем при удалении.

Ответ:

- Создать WeakReference на объект
- Проверить доступность через TryGetTarget
- Использовать объект, если он доступен
- При необходимости пересоздать объект и обновить WeakReference

🔧 Шаг 11:

Миссия 11

Расположите шаги обновления кеша с использованием WeakReference:

1. Итерировать по WeakReference в коллекции

2. Проверять TryGetTarget
3. Удалять устаревшие ссылки
4. Добавлять новые объекты при необходимости

 Подсказка:

Сначала проверяем ссылки, затем удаляем устаревшие, добавляем новые объекты.

Ответ:

- Итерировать по WeakReference в коллекции
- Проверять TryGetTarget
- Удалять устаревшие ссылки
- Добавлять новые объекты при необходимости

 Шаг 12:

Миссия 12

Расположите этапы реализации кеша объектов с WeakReference:

1. Создать коллекцию WeakReference
2. Добавлять объекты в коллекцию через WeakReference
3. При доступе проверять доступность объектов
4. При необходимости пересоздать удалённые объекты

 Подсказка:

Сначала создаём коллекцию, потом добавляем, проверяем и пересоздаём объекты при удалении сборщиком мусора.

Ответ:

- Создать коллекцию WeakReference
- Добавлять объекты в коллекцию через WeakReference
- При доступе проверять доступность объектов
- При необходимости пересоздать удалённые объекты

 Шаг 13:

Миссия 13

Расположите этапы обработки устаревших данных в кеш-системе:

1. Итерировать по коллекции WeakReference
2. Проверить доступность каждого объекта
3. Удалить ссылки на объекты, которых нет в памяти
4. Обновить данные или создать новые объекты при необходимости

 Подсказка:

Сначала проверяем, потом удаляем, затем обновляем устаревшие данные.

Ответ:

- Итерировать по коллекции WeakReference
- Проверить доступность каждого объекта
- Удалить ссылки на объекты, которых нет в памяти
- Обновить данные или создать новые объекты при необходимости

Шаг 14:

Миссия 14

Расположите этапы эффективного использования WeakReference для кеша больших объектов:

1. Создать WeakReference для объекта
2. Использовать TryGetTarget перед доступом к объекту
3. Если объект удалён, пересоздать и обновить ссылку
4. Регулярно очищать устаревшие ссылки из коллекции

Подсказка:

Сначала создаём ссылку, затем проверяем, пересоздаём при удалении, затем очищаем коллекцию.

Ответ:

- Создать WeakReference для объекта
- Использовать TryGetTarget перед доступом к объекту
- Если объект удалён, пересоздать и обновить ссылку
- Регулярно очищать устаревшие ссылки из коллекции

Шаг 15:

Миссия 15

Что такое WeakReference в C#?

Подсказка: Подумайте о ссылках на объекты, которые могут быть удалены сборщиком мусора.

Ответ:

- **Слабая ссылка, которая не предотвращает сборку объекта сборщиком мусора**
- Сильная ссылка на объект, сохраняющая его в памяти
- Ссылка только на статические объекты
- Механизм для кеширования файлов на диске

Шаг 16:

Миссия 16

Зачем использовать WeakReference при кешировании объектов?

Подсказка: Подумайте о том, как избежать переполнения памяти и при этом хранить часто используемые объекты.

Ответ:

- **Чтобы объекты могли быть удалены сборщиком мусора при нехватке памяти**
- Чтобы объекты всегда оставались в памяти независимо от сборщика мусора
- Чтобы предотвратить доступ к объектам из других потоков
- Чтобы ускорить сериализацию объектов

Шаг 17:

Миссия 17

 Сопоставьте понятие с его характеристикой:

Ответ:

- WeakReference
- StrongReference

- Кеш с слабой ссылкой
- Кеш с сильной ссылкой

1. Слабая ссылка, позволяющая сборщику мусора удалять объект
2. Сильная ссылка, препятствующая удалению объекта сборщиком мусора
3. Хранит объекты, которые могут быть удалены при нехватке памяти
4. Хранит объекты, которые всегда остаются в памяти

Подсказка: Смотрите на силу ссылки и влияние на сборку мусора.

- **WeakReference** — Слабая ссылка, позволяющая сборщику мусора удалять объект
- **StrongReference** — Сильная ссылка, препятствующая удалению объекта сборщиком мусора
- **Кеш с слабой ссылкой** — Хранит объекты, которые могут быть удалены при нехватке памяти
- **Кеш с сильной ссылкой** — Хранит объекты, которые всегда остаются в памяти

Урок 5 - ValueTask vs Task

Шаг 01:

Миссия 1

Разработчик заметил, что асинхронный метод почти всегда возвращает готовый результат. Что будет эффективнее использовать?

Подсказка: Подумайте, какой тип минимизирует выделение памяти при готовом результате.

Ответ:

- **ValueTask**
- Task
- void
- IAsyncResult

Шаг 02:

Миссия 2

Программист создает метод для редких сетевых запросов. Что лучше использовать, чтобы не усложнять код?

Подсказка: Если результат почти всегда требует ожидания, обычный Task подходит лучше.

Ответ:

- **Task**
- ValueTask
- void
- IEnumerable

Шаг 03:

Миссия 3

Метод часто возвращает уже готовый результат, но иногда выполняется асинхронно. Какой тип оптимален?

Подсказка: ValueTask может использоваться для уменьшения аллокаций при готовом результате.

Ответ:

- **ValueTask**
- **Task (для упрощения кода при сложных сценариях)**
- void
- IEnumerable

Шаг 04:

Миссия 4

Разработчик хочет избежать лишних аллокаций для метода, который почти всегда возвращает синхронный результат. Какой подход выбрать?

Подсказка: ValueTask помогает не создавать новый Task для готового значения.

Ответ:

- **ValueTask**
- Task
- Action
- Func

Шаг 05:

Миссия 5

При использовании ValueTask важно помнить об одной особенности при многократном ожидании результата. Что это?

Подсказка: ValueTask нельзя await более одного раза без преобразования в Task.

Ответ:

- **ValueTask нельзя await более одного раза**
- **Для многократного использования нужно вызвать AsTask()**
- ValueTask автоматически поддерживает многократное await
- Task и ValueTask полностью идентичны по многократному await

Шаг 06:

Миссия 6

 Сопоставьте тип с его особенностью:

Ответ:

- Task
 - ValueTask
 - Task с результатом
 - ValueTask с результатом
1. Всегда выделяет объект для асинхронной операции
 2. Не выделяет объект, если результат готов синхронно
 3. Возвращает результат после завершения асинхронной операции
 4. Может быть await один раз или преобразован в Task для многократного await

Подсказка:

Подумайте, какой тип минимизирует аллокации и какие особенности имеет для результата.

- **Task** — Всегда выделяет объект для асинхронной операции
- **ValueTask** — Не выделяет объект, если результат готов синхронно

- **Task с результатом** — Возвращает результат после завершения асинхронной операции
- **ValueTask с результатом** — Может быть await один раз или преобразован в Task для многократного await

Шаг 07:

Миссия 7

 Сопоставьте преимущество с типом:

Ответ:

- Task
- ValueTask
- Task с результатом
- ValueTask с результатом

1. Простота кода и поддержка многократного await
2. Снижение аллокаций при готовом результате
3. Возвращение результата после асинхронной операции
4. Может использоваться один раз без выделения памяти

 **Подсказка:**

Обратите внимание на производительность и особенности await.

- **Task** — Простота кода и поддержка многократного await
- **ValueTask** — Снижение аллокаций при готовом результате
- **Task с результатом** — Возвращение результата после асинхронной операции
- **ValueTask с результатом** — Может использоваться один раз без выделения памяти

Шаг 08:

Миссия 8

Расположите шаги безопасного использования ValueTask с результатом:

1. Вызвать метод, возвращающий ValueTask
2. Await результат
3. Проверить, нужно ли многократное использование
4. При необходимости вызвать AsTask() для повторного await

 **Подсказка:**

Сначала вызываем, потом await, затем смотрим, нужно ли преобразование.

Ответ:

- Вызвать метод, возвращающий ValueTask
- Await результат
- Проверить, нужно ли многократное использование
- При необходимости вызвать AsTask() для повторного await

Шаг 09:

Миссия 9

Расположите шаги оптимизации метода, который чаще возвращает готовый результат:

1. Заменить Task на ValueTask
2. Сократить выделение памяти при готовом результате

3. Проверить совместимость с многократным await
4. При необходимости вызвать AsTask()

Подсказка:

Сначала меняем тип, затем минимизируем аллокации и обеспечиваем безопасность многократного await.

Ответ:

- Заменить Task на ValueTask
- Сократить выделение памяти при готовом результате
- Проверить совместимость с многократным await
- При необходимости вызвать AsTask()

Шаг 10:

Миссия 10

Что из перечисленного правильно описывает ValueTask?

Подсказка: Подумайте о том, как ValueTask работает с асинхронными результатами и памятью.

Ответ:

- **Структура, позволяющая возвращать результат асинхронной операции без выделения объекта Task, если результат уже известен**
- Класс для выполнения долгих синхронных операций
- Объект, всегда выделяемый в куче для асинхронного метода
- Тип, который можно использовать только с потоками UI

Шаг 11:

Миссия 11

Когда рекомендуется использовать Task вместо ValueTask?

Подсказка: Подумайте о методах, которые всегда возвращают асинхронный результат.

Ответ:

- **Когда метод всегда выполняет асинхронную работу и результат еще не известен**
- Когда метод возвращает значение синхронно
- Когда метод работает только с коллекциями
- Когда метод выполняется на UI-потоке исключительно

Шаг 12:

Миссия 12

 Сопоставьте тип с его характеристикой:

Ответ:

- Task<T>
- ValueTask<T>
- Использование Task
- Использование ValueTask

1. Объект, выделяемый в куче для асинхронного метода
2. Структура, минимизирующая выделение памяти при известном результате
3. Применяется, когда метод всегда асинхронный и результат еще не готов

4. Применяется, когда результат может быть сразу известен и копирования Task можно избежать

Подсказка: Обратите внимание на эффективность памяти и частоту вызовов.

- **Task** — Объект, выделяемый в куче для асинхронного метода
- **ValueTask** — Структура, минимизирующая выделение памяти при известном результате
- **Использование Task** — Применяется, когда метод всегда асинхронный и результат еще не готов
- **Использование ValueTask** — Применяется, когда результат может быть сразу известен и копирования Task можно избежать

Шаг 13:

Миссия 13

Какой недостаток ValueTask нужно учитывать?

Подсказка: Подумайте о повторном использовании и ожидании результата.

Ответ:

- **Нельзя многократно ожидать один и тот же ValueTask без преобразования в Task**
- ValueTask всегда выделяется в куче
- Task нельзя использовать с async/await
- ValueTask блокирует поток UI автоматически

Шаг 14:

Миссия 14

Расположите шаги использования ValueTask для оптимизации памяти:

1. Создание ValueTask для метода, который может вернуть результат сразу
2. Проверка, готов ли результат без ожидания
3. Если результат не готов, преобразовать ValueTask в Task
4. Ожидание Task при необходимости

Подсказка: Сначала используем ValueTask, потом только при необходимости создаем Task.

Ответ:

- Создание ValueTask для метода, который может вернуть результат сразу
- Проверка, готов ли результат без ожидания
- Если результат не готов, преобразовать ValueTask в Task
- Ожидание Task при необходимости

Модуль 3 - LINQ и продвинутая работа с данными

Урок 1 - Продвинутый LINQ

Шаг 01:

Миссия 1

Разработчик замечает, что метод почти всегда возвращает готовый результат. Что эффективнее использовать?

Подсказка: Подумайте, какой тип минимизирует выделение памяти при готовом результате.

Ответ:

- **ValueTask**

- Task
- void
- IAsyncResult

🔮 Шаг 02:

Миссия 2

Метод для редких сетевых запросов. Что лучше использовать, чтобы код оставался простым?

Подсказка: Если результат почти всегда требует ожидания, обычный Task подходит лучше.

Ответ:

- **Task**
- ValueTask
- void
- IAsyncEnumerable

🔮 Шаг 03:

Миссия 3

Метод часто возвращает готовый результат, но иногда выполняется асинхронно. Какой тип оптимален?

Подсказка: ValueTask уменьшает выделение памяти при готовом результате.

Ответ:

- **ValueTask**
- **Task (для сложных сценариев)**
- void
- IAsyncEnumerable

🔮 Шаг 04:

Миссия 4

Разработчик хочет избежать лишних аллокаций для метода, который почти всегда возвращает синхронный результат. Какой тип выбрать?

Подсказка: ValueTask не создаёт новый Task для готового значения.

Ответ:

- **ValueTask**
- Task
- Action
- Func

🔮 Шаг 05:

Миссия 5

При использовании ValueTask важно помнить про многократное await. Что нужно учитывать?

Подсказка: ValueTask нельзя await более одного раза без преобразования.

Ответ:

- **ValueTask нельзя await более одного раза**
- **Для многократного использования нужно вызвать AsTask()**
- ValueTask поддерживает многократное await автоматически
- Task и ValueTask идентичны для многократного await

Шаг 06:

Миссия 6

 Сопоставьте тип с особенностью:

Ответ:

- Task
 - ValueTask
 - Task с результатом
 - ValueTask с результатом
1. Всегда выделяет объект для асинхронной операции
 2. Не выделяет объект, если результат готов синхронно
 3. Возвращает результат после завершения операции
 4. Можно await один раз или вызвать AsTask() для многократного await

Подсказка:

Смотрите на аллокации и возможность повторного await.

- **Task** — Всегда выделяет объект для асинхронной операции
- **ValueTask** — Не выделяет объект, если результат готов синхронно
- **Task с результатом** — Возвращает результат после завершения операции
- **ValueTask с результатом** — Можно await один раз или вызвать AsTask() для многократного await

Шаг 07:

Миссия 7

 Сопоставьте преимущество с типом:

Ответ:

- Task
 - ValueTask
 - Task с результатом
 - ValueTask с результатом
1. Простота кода и поддержка многократного await
 2. Снижение аллокаций при готовом результате
 3. Возвращение результата после асинхронной операции
 4. Может использоваться один раз без выделения памяти

Подсказка:

Обратите внимание на производительность и возможности await.

- **Task** — Простота кода и поддержка многократного await
- **ValueTask** — Снижение аллокаций при готовом результате
- **Task с результатом** — Возвращение результата после асинхронной операции
- **ValueTask с результатом** — Может использоваться один раз без выделения памяти

Шаг 08:

Миссия 8

Что из перечисленного правильно описывает оператор LINQ Select?

Подсказка: Подумайте о проекции элементов коллекции на новый вид.

Ответ:

- **Позволяет преобразовать каждый элемент последовательности в новый вид**
- Фильтрует элементы по условию
- Сортирует элементы по ключу
- Объединяет две последовательности

Шаг 09:

Миссия 9

Для чего используется оператор LINQ Where?

Подсказка: Подумайте о том, как выбрать элементы коллекции по условию.

Ответ:

- **Фильтрует элементы последовательности по заданному условию**
- Преобразует элементы последовательности в другой тип
- Объединяет элементы из нескольких последовательностей
- Сортирует элементы по возрастанию или убыванию

Шаг 10:

Миссия 10

 Сопоставьте оператор LINQ с его действием:

Ответ:

- Select
 - Where
 - OrderBy
 - Join
1. Преобразует элементы последовательности в новый вид
 2. Фильтрует элементы по условию
 3. Сортирует элементы по ключу
 4. Объединяет элементы двух последовательностей по ключу

Подсказка: Вспомните, какой оператор отвечает за проекцию, фильтрацию, сортировку и объединение.

- **Select** — Преобразует элементы последовательности в новый вид
- **Where** — Фильтрует элементы по условию
- **OrderBy** — Сортирует элементы по ключу
- **Join** — Объединяет элементы двух последовательностей по ключу

Шаг 11:

Миссия 11

Что из перечисленного описывает оператор GroupBy?

Подсказка: Подумайте о том, как группировать элементы по ключу.

Ответ:

- **Группирует элементы последовательности по указанному ключу**
- Фильтрует элементы по условию
- Создает новую последовательность из элементов двух коллекций
- Сортирует элементы по убыванию

Шаг 12:

Миссия 12

 Сопоставьте LINQ метод с примером использования:

Ответ:

- Select
- Where
- GroupBy
- OrderByDescending

1. Преобразование списка студентов в список их имен
2. Выбор студентов старше 18 лет
3. Группировка студентов по факультету
4. Сортировка списка по убыванию баллов

Подсказка: Смотрите на действие метода и ожидаемый результат.

- **Select** — Преобразование списка студентов в список их имен
- **Where** — Выбор студентов старше 18 лет
- **GroupBy** — Группировка студентов по факультету
- **OrderByDescending** — Сортировка списка по убыванию баллов

Шаг 13:

Миссия 13

Расположите шаги составления LINQ-запроса для выборки и сортировки:

1. Определение источника данных
2. Применение фильтрации с помощью Where
3. Применение проекции с помощью Select
4. Сортировка результата с помощью OrderBy
5. Выполнение запроса и получение результата

Подсказка: Сначала выбираем источник, затем фильтруем, проецируем, сортируем и получаем результат.

Ответ:

- Определение источника данных
- Применение фильтрации с помощью Where
- Применение проекции с помощью Select
- Сортировка результата с помощью OrderBy
- Выполнение запроса и получение результата

Шаг 14:

Миссия 14

Что делает оператор LINQ Any?

Подсказка: Подумайте о проверке существования элементов в коллекции.

Ответ:

- **Проверяет, содержит ли последовательность хотя бы один элемент, удовлетворяющий условию**
- Сортирует элементы по возрастанию

- Группирует элементы по ключу
- Создает новую коллекцию с преобразованными элементами

Шаг 15:

Миссия 15

 Сопоставьте метод LINQ с его функциональностью:

Ответ:

- Any
- All
- FirstOrDefault
- Distinct

1. Проверяет, есть ли хотя бы один элемент, удовлетворяющий условию
2. Проверяет, что все элементы удовлетворяют условию
3. Возвращает первый элемент или значение по умолчанию, если коллекция пуста
4. Удаляет дубликаты из последовательности

Подсказка: Смотрите на смысл проверки условий, извлечения элементов и удаления повторов.

- **Any** — Проверяет, есть ли хотя бы один элемент, удовлетворяющий условию
- **All** — Проверяет, что все элементы удовлетворяют условию
- **FirstOrDefault** — Возвращает первый элемент или значение по умолчанию, если коллекция пуста
- **Distinct** — Удаляет дубликаты из последовательности

Урок 2 - Асинхронные потоки и `IAsyncEnumerable`

Шаг 01:

Миссия 1

Разработчик хочет вернуть последовательность данных асинхронно без загрузки всех элементов в память сразу. Что использовать?

Подсказка: Смотрите на типы, поддерживающие асинхронную итерацию.

Ответ:

- **IAsyncEnumerable**
- IEnumerable
- Task<>
- List

Шаг 02:

Миссия 2

Метод возвращает `IAsyncEnumerable`, но некоторые элементы вычисляются дорого. Как лучше их обрабатывать?

Подсказка: Асинхронная итерация позволяет обрабатывать элементы по мере их готовности.

Ответ:

- **await foreach**
- foreach
- for
- Parallel.ForEach

Шаг 03:

Миссия 3

Вы хотите объединить несколько асинхронных потоков в один. Какой подход оптимален?
Подсказка: Используйте операторы LINQ для потоков, поддерживающие `IEnumerableAsync`.

Ответ:

- **SelectManyAwait**
- **MergeAsync**
- Concat
- Zip

Шаг 04:

Миссия 4

Метод возвращает `IEnumerableAsync`, но нужно получить все элементы в List. Какой способ корректен?

Подсказка: Используйте встроенные методы расширения для асинхронных потоков.

Ответ:

- **await ToListAsync()**
- ToList()
- await ToArray()
- await Collect()

Шаг 05:

Миссия 5

Вы хотите пропускать элементы до тех пор, пока не выполнится определённое условие в асинхронном потоке. Какой оператор LINQ использовать?

Подсказка: Подумайте про фильтрацию с условием в начале потока.

Ответ:

- **SkipWhileAwaitAsync**
- TakeWhileAwaitAsync
- WhereAwait
- SelectAwait

Шаг 06:

Миссия 6

 Сопоставьте тип `IEnumerableAsync` с его особенностью:

Ответ:

- `IEnumerableAsync`
 - `await foreach`
 - `ToListAsync`
 - `SelectAwait`
1. Позволяет асинхронно перечислять элементы
 2. Синтаксис для асинхронного перебора
 3. Превращает поток в List асинхронно
 4. Проецирует элементы с асинхронной функцией

Подсказка:

Смотрите на возможности перебора, проекции и сборки потока.

- **IAsyncEnumerable** — Позволяет асинхронно перечислять элементы
- **await foreach** — Синтаксис для асинхронного перебора
- **ToListAsync** — Превращает поток в List асинхронно
- **SelectAwait** — Проецирует элементы с асинхронной функцией

Шаг 07:

Миссия 7

 Сопоставьте оператор LINQ с его действием для IAsyncEnumerable:

Ответ:

- WhereAwait
 - SkipWhileAwaitAsync
 - TakeWhileAwaitAsync
 - FirstOrDefaultAsync
1. Фильтрует элементы по условию
 2. Пропускает элементы пока условие верно
 3. Берет элементы пока условие верно
 4. Возвращает первый элемент или значение по умолчанию

Подсказка:

Обратите внимание, как изменяются границы выборки элементов.

- **WhereAwait** — Фильтрует элементы по условию
- **SkipWhileAwaitAsync** — Пропускает элементы пока условие верно
- **TakeWhileAwaitAsync** — Берет элементы пока условие верно
- **FirstOrDefaultAsync** — Возвращает первый элемент или значение по умолчанию

Шаг 08:

Миссия 8

 Сопоставьте ситуацию с подходящим методом LINQ для асинхронного потока:

Ответ:

- Пропустить первые элементы
 - Выбрать элементы с условием
 - Собрать все элементы в коллекцию
 - Произвести асинхронную трансформацию
1. SkipAwaitAsync
 2. WhereAwait
 3. ToListAsync
 4. SelectAwait

Подсказка:

Смотрите на фильтрацию, трансформацию и сборку элементов.

- **Пропустить первые элементы** — SkipAwaitAsync
- **Выбрать элементы с условием** — WhereAwait
- **Собрать все элементы в коллекцию** — ToListAsync

- **Произвести асинхронную трансформацию** — SelectAwait

Шаг 09:

Миссия 9

Расположите шаги корректной обработки IEnumerable:

1. Создать IEnumerable
2. Вызвать await foreach для перебора
3. Применить асинхронные операторы LINQ
4. Собрать элементы в коллекцию при необходимости

Подсказка:

Сначала создаём поток, потом перебираем и трансформируем элементы, и только потом собираем в коллекцию.

Ответ:

- Создать IEnumerable
- Вызвать await foreach для перебора
- Применить асинхронные операторы LINQ
- Собрать элементы в коллекцию при необходимости

Шаг 10:

Миссия 10

Что такое IEnumerable в C#?

Подсказка: Подумайте о потоках данных, которые можно перебирать асинхронно.

Ответ:

- **Интерфейс для асинхронного перебора последовательностей элементов**
- Синхронная коллекция с поддержкой многопоточности
- Метод для синхронной обработки событий
- Класс для управления задачами Task

Шаг 11:

Миссия 11

Какой оператор используется для асинхронного перебора элементов IEnumerable?

Подсказка: Смотрите на привычный foreach, но с асинхронной обработкой.

Ответ:

- **await foreach**
- foreach
- for await
- async foreachLoop

Шаг 12:

Миссия 12

 Сопоставьте термин с его определением:

Ответ:

- IEnumerable
- await foreach

- Yield
- CancellationToken

1. Интерфейс для асинхронного перебора последовательностей
2. Оператор для асинхронного цикла перебора
3. Ключевое слово для генерации последовательности элементов по одному
4. Позволяет отменять асинхронные операции

Подсказка: Вспомните, что используется для асинхронного перебора, генерации элементов и управления отменой.

- **IAsyncEnumerable** — Интерфейс для асинхронного перебора последовательностей
- **await foreach** — Оператор для асинхронного цикла перебора
- **Yield** — Ключевое слово для генерации последовательности элементов по одному
- **CancellationToken** — Позволяет отменять асинхронные операции

Шаг 13:

Миссия 13

Зачем использовать IEnumerableAsync вместо List в асинхронной обработке данных?

Подсказка: Подумайте о потоковой обработке больших данных.

Ответ:

- **Позволяет обрабатывать элементы по мере их готовности без загрузки всех данных в память**
- Ускоряет синхронный перебор коллекции
- Гарантирует порядок обработки элементов без асинхронных задержек
- Превращает список в поток задач Task

Шаг 14:

Миссия 14

 Сопоставьте методы с их применением в IEnumerableAsync:

Ответ:

- ToListAsync
 - Where
 - Select
 - Take
1. Преобразует поток в список асинхронно
 2. Фильтрует элементы по условию
 3. Преобразует элементы в новый вид
 4. Берет первые N элементов последовательности

Подсказка: Вспомните, какие LINQ методы поддерживаются асинхронно и для чего.

- **ToListAsync** — Преобразует поток в список асинхронно
- **Where** — Фильтрует элементы по условию
- **Select** — Преобразует элементы в новый вид
- **Take** — Берет первые N элементов последовательности

Шаг 15:

Миссия 15

Расположите шаги правильной асинхронной обработки данных с IEnumerableAsync:

1. Создание источника данных с IEnumerableAsync
2. Применение фильтрации (Where)
3. Применение проекции (Select)
4. Асинхронный перебор элементов с await foreach
5. Преобразование результата в список сToListAsync

Подсказка: Сначала создаем поток, затем фильтруем, преобразуем, перебираем и собираем результат.

Ответ:

- Создание источника данных с IEnumerableAsync
- Применение фильтрации (Where)
- Применение проекции (Select)
- Асинхронный перебор элементов с await foreach
- Преобразование результата в список сToListAsync

Шаг 16:

Миссия 16

Что происходит, если отменяется CancellationToken во время await foreach?

Подсказка: Подумайте о прерывании асинхронного перебора.

Ответ:

- **Перебор элементов прерывается и выбрасывается OperationCanceledException**
- Перебор продолжается до конца без изменений
- Все элементы игнорируются, и возвращается пустой список
- Поток данных блокируется до завершения CancellationToken

Шаг 17:

Миссия 17

 Сопоставьте ключевые слова с их функцией в асинхронных потоках:

Ответ:

- await foreach
 - Yield
 - ToListAsync
 - CancellationToken
1. Асинхронный цикл перебора элементов
 2. Генерация элементов по одному в асинхронном методе
 3. Сбор элементов потока в список
 4. Средство отмены асинхронных операций

Подсказка: Подумайте, какие ключевые слова отвечают за перебор, генерацию, сбор и отмену.

- **await foreach** — Асинхронный цикл перебора элементов
- **Yield** — Генерация элементов по одному в асинхронном методе
- **ToListAsync** — Сбор элементов потока в список
- **CancellationToken** — Средство отмены асинхронных операций

Урок 3 - Parallel LINQ (PLINQ)

Шаг 01:

Миссия 1

Разработчик хочет ускорить обработку большого массива данных. Какой инструмент PLINQ использовать?

Подсказка: Подумайте о параллельной обработке.

Ответ:

- **AsParallel()**
- AsSequential()
- Task.Run()
- ThreadPool.QueueUserWorkItem()

Шаг 02:

Миссия 2

Вы хотите сохранить порядок элементов при параллельной обработке. Какой метод PLINQ подходит?

Подсказка: Обратите внимание на методы, контролирующие порядок.

Ответ:

- **AsOrdered()**
- AsUnordered()
- WithDegreeOfParallelism()
- ForAll()

Шаг 03:

Миссия 3

Какой оператор PLINQ позволяет ограничить количество потоков для параллельной обработки?

Подсказка: Подумайте о контроле степени параллелизма.

Ответ:

- **WithDegreeOfParallelism()**
- AsParallel()
- ForAll()
- AsOrdered()

Шаг 04:

Миссия 4

Какой метод PLINQ применяют для одновременного выполнения действия над каждым элементом без возврата нового потока?

Подсказка: Смотрите на методы, которые не создают коллекцию.

Ответ:

- **ForAll()**
- Select()
- Where()
- ToList()

Шаг 05:

Миссия 5

Вы обрабатываете большой набор данных и хотите игнорировать порядок элементов ради скорости. Какой метод применить?

Подсказка: Подумайте о методе, который снимает ограничения порядка.

Ответ:

- **AsUnordered()**
- AsOrdered()
- WithDegreeOfParallelism()
- ForAll()

Шаг 06:

Миссия 6

 Сопоставьте оператор PLINQ с его назначением:

Ответ:

- AsParallel()
- AsOrdered()
- AsUnordered()
- WithDegreeOfParallelism()

1. Создает параллельный поток для обработки
2. Сохраняет порядок элементов
3. Игнорирует порядок элементов
4. Ограничивает количество потоков

 **Подсказка:**

Смотрите на назначение каждого метода: создание потока, порядок, ограничение потоков.

- **AsParallel()** — Создает параллельный поток для обработки
- **AsOrdered()** — Сохраняет порядок элементов
- **AsUnordered()** — Игнорирует порядок элементов
- **WithDegreeOfParallelism()** — Ограничивает количество потоков

Шаг 07:

Миссия 7

 Сопоставьте метод PLINQ с типичной ситуацией:

Ответ:

- ForAll()
- Select()
- Where()
- ToList()

1. Выполнить действие над каждым элементом без создания нового потока
2. Проецирует элементы в новый поток
3. Фильтрует элементы по условию
4. Собирает результаты в коллекцию

 **Подсказка:**

Подумайте, какие методы создают новые коллекции, а какие нет.

- **ForAll()** — Выполнить действие над каждым элементом без создания нового потока
- **Select()** — Проецирует элементы в новый поток
- **Where()** — Фильтрует элементы по условию
- **ToList()** — Собирает результаты в коллекцию

🔗 Шаг 08:

Миссия 8

📄 Сопоставьте свойства PLINQ с их эффектом:

Ответ:

- Порядок выполнения
 - Степень параллелизма
 - Фильтрация
 - Проекция
1. Контролируется AsOrdered/AsUnordered
 2. Контролируется WithDegreeOfParallelism()
 3. Используется Where()
 4. Используется Select()

🔗 Подсказка:

Подумайте, какие методы управляют порядком, а какие трансформацией элементов.

- **Порядок выполнения** — Контролируется AsOrdered/AsUnordered
- **Степень параллелизма** — Контролируется WithDegreeOfParallelism()
- **Фильтрация** — Используется Where()
- **Проекция** — Используется Select()

🔗 Шаг 09:

Миссия 9

Расположите шаги для безопасного применения PLINQ к большому набору данных:

1. Создать параллельный поток через AsParallel()
2. Применить фильтры и проекции (Where, Select)
3. При необходимости ограничить количество потоков WithDegreeOfParallelism()
4. Собрать результаты в коллекцию через ToList()
5. При необходимости выполнить действие над каждым элементом через ForAll()

🔗 Подсказка:

Сначала создаем поток, затем обрабатываем элементы, управляем потоками и собираем результаты.

Ответ:

- Создать параллельный поток через AsParallel()
- Применить фильтры и проекции (Where, Select)
- При необходимости ограничить количество потоков WithDegreeOfParallelism()
- Собрать результаты в коллекцию через ToList()
- При необходимости выполнить действие над каждым элементом через ForAll()

🔮 Шаг 10:

Миссия 10

Что такое PLINQ в C#?

Подсказка: Подумайте о параллельной обработке коллекций.

Ответ:

- **Parallel LINQ для параллельной обработки последовательностей**
- Библиотека синхронной обработки данных
- Асинхронная коллекция Task
- Метод для объединения потоков данных

🔮 Шаг 11:

Миссия 11

Как включить параллельную обработку в LINQ-запросе?

Подсказка: Вспомните метод, который превращает обычный запрос в параллельный.

Ответ:

- **AsParallel()**
- Parallelize()
- ToParallel()
- RunParallel()

🔮 Шаг 12:

Миссия 12

📄 Сопоставьте ключевые методы PLINQ с их функцией:

Ответ:

- AsParallel()
- WithDegreeOfParallelism
- WithExecutionMode
- ForAll

1. Включает параллельную обработку для последовательности
2. Задаёт количество одновременно выполняемых задач
3. Определяет стратегию выполнения (параллельный или последовательный)
4. Применяет действие к каждому элементу параллельно

Подсказка: Подумайте, какие методы управляют параллельностью, стратегией и применением действий.

- **AsParallel()** — Включает параллельную обработку для последовательности
- **WithDegreeOfParallelism** — Задаёт количество одновременно выполняемых задач
- **WithExecutionMode** — Определяет стратегию выполнения (параллельный или последовательный)
- **ForAll** — Применяет действие к каждому элементу параллельно

🔮 Шаг 13:

Миссия 13

В чём преимущество использования PLINQ для больших коллекций?

Подсказка: Подумайте о многопоточности и времени выполнения.

Ответ:

- **Ускоряет обработку за счет распределения работы между потоками**
- Гарантирует порядок элементов без дополнительных настроек
- Уменьшает использование памяти
- Обеспечивает синхронное выполнение для всех коллекций

Шаг 14:

Миссия 14

 Сопоставьте эффекты оптимизации PLINQ с их методами:

Ответ:

- `AsOrdered()`
 - `WithMergeOptions`
 - `ForAll`
 - `WithCancellation`
1. Сохраняет порядок исходной последовательности при параллельной обработке
 2. Контролирует стратегию объединения результатов параллельного запроса
 3. Выполняет действие для каждого элемента параллельно без ожидания результатов
 4. Позволяет отменять выполнение параллельного запроса

Подсказка: Подумайте о порядке, объединении, параллельных действиях и отмене.

- **`AsOrdered()`** — Сохраняет порядок исходной последовательности при параллельной обработке
- **`WithMergeOptions`** — Контролирует стратегию объединения результатов параллельного запроса
- **`ForAll`** — Выполняет действие для каждого элемента параллельно без ожидания результатов
- **`WithCancellation`** — Позволяет отменять выполнение параллельного запроса

Шаг 15:

Миссия 15

Расположите шаги параллельной обработки коллекции с PLINQ:

1. Создание исходной последовательности
2. Включение параллельной обработки через `AsParallel()`
3. Применение фильтров и проекций (`Where`, `Select`)
4. Выполнение действий с `ForAll`
5. Опционально: сохранение порядка с `AsOrdered()`

Подсказка: Сначала делаем коллекцию параллельной, затем фильтруем и выполняем действия, порядок можно сохранить на конце.

Ответ:

- Создание исходной последовательности
- Включение параллельной обработки через `AsParallel()`
- Применение фильтров и проекций (`Where`, `Select`)
- Выполнение действий с `ForAll`
- Опционально: сохранение порядка с `AsOrdered()`

Шаг 16:

Миссия 16

Что произойдет, если вызвать ForAll на очень большой коллекции без AsOrdered?

Подсказка: Подумайте о порядке обработки элементов и параллельных потоках.

Ответ:

- **Элементы будут обработаны параллельно, порядок может отличаться от исходного**
- Элементы будут обработаны строго в исходном порядке
- Запрос блокируется до завершения всех потоков
- Элементы преобразуются в синхронную коллекцию

Урок 4 - Dynamic LINQ и Expression Trees

Шаг 01:

Миссия 1

Разработчик хочет динамически фильтровать коллекцию по имени свойства. Какой инструмент C# лучше использовать?

Подсказка: Динамический LINQ позволяет строить запросы на лету.

Ответ:

- **Dynamic LINQ**
- LINQ to SQL
- PLINQ
- Entity Framework Core

Шаг 02:

Миссия 2

Какой класс позволяет строить выражения на лету в Expression Trees?

Подсказка: Смотрите на типы, которые создают лямбда-выражения программно.

Ответ:

- **System.Linq.Expressions.Expression**
- Func
- IQueryable
- Task

Шаг 03:

Миссия 3

Вы хотите динамически создавать условие фильтрации, которое может изменяться во время выполнения. Что использовать?

Подсказка: Подумайте о возможностях Expression Trees и Dynamic LINQ.

Ответ:

- **Expression Trees**
- PLINQ
- Task
- ThreadPool

Шаг 04:

Миссия 4

Для чего нужен метод `Compile()` в Expression Trees?

Подсказка: Он превращает выражение в выполняемый код.

Ответ:

- Создает исполняемый делегат из выражения
- Создает `IQueryable` объект
- Фильтрует коллекцию синхронно
- Отправляет запрос в базу данных

Шаг 05:

Миссия 5

Какой синтаксис Dynamic LINQ позволяет фильтровать по строковому выражению?

Подсказка: В Dynamic LINQ условия передаются как строки.

Ответ:

- **`Where("Property == @0", value)`**
- `Where(x => x.Property == value)`
- `Select(x => x.Property)`
- `OrderBy(x => x.Property)`

Шаг 06:

Миссия 6

 Сопоставьте компоненты Expression Trees с их назначением:

Ответ:

- `ParameterExpression`
 - `ConstantExpression`
 - `BinaryExpression`
 - `LambdaExpression`
1. Представляет входной параметр функции
 2. Представляет константу в выражении
 3. Представляет бинарную операцию, например `==` или `+`
 4. Представляет лямбда-выражение

 **Подсказка:**

Каждый тип Expression отвечает за определенный элемент кода.

- **`ParameterExpression`** — Представляет входной параметр функции
- **`ConstantExpression`** — Представляет константу в выражении
- **`BinaryExpression`** — Представляет бинарную операцию, например `==` или `+`
- **`LambdaExpression`** — Представляет лямбда-выражение

Шаг 07:

Миссия 7

 Сопоставьте методы Dynamic LINQ с их назначением:

Ответ:

- Where
 - Select
 - OrderBy
 - GroupBy
1. Фильтрует элементы по условию
 2. Проецирует элементы в новый вид
 3. Сортирует элементы
 4. Группирует элементы по ключу

Подсказка:

Смотрите на стандартные операции LINQ, применимые динамически.

- **Where** — Фильтрует элементы по условию
- **Select** — Проецирует элементы в новый вид
- **OrderBy** — Сортирует элементы
- **GroupBy** — Группирует элементы по ключу

Шаг 08:

Миссия 8

 Сопоставьте преимущества Expression Trees с их применением:

Ответ:

- Отложенное выполнение
 - Динамическая генерация кода
 - Улучшенная производительность запросов
 - Совместимость с LINQ провайдерами
1. Код выполняется только при вызове Compile()
 2. Можно строить выражения во время выполнения программы
 3. Оптимизация запросов к базе данных
 4. Позволяет работать с IQueryable и ORM

Подсказка:

Каждое преимущество отвечает за конкретную пользу Expression Trees.

- **Отложенное выполнение** — Код выполняется только при вызове Compile()
- **Динамическая генерация кода** — Можно строить выражения во время выполнения программы
- **Улучшенная производительность запросов** — Оптимизация запросов к базе данных
- **Совместимость с LINQ провайдерами** — Позволяет работать с IQueryable и ORM

Шаг 09:

Миссия 9

Расположите шаги создания динамического фильтра с Expression Trees:

1. Создать ParameterExpression для входного объекта
2. Создать ConstantExpression для значения фильтра
3. Создать BinaryExpression для сравнения
4. Создать LambdaExpression из BinaryExpression и ParameterExpression
5. Скомпилировать LambdaExpression в делегат

6. Вызвать делегат для фильтрации коллекции

Подсказка:

Подумайте о порядке создания элементов: сначала параметры и константы, потом выражения, потом компиляция и вызов.

Ответ:

- Создать `ParameterExpression` для входного объекта
- Создать `ConstantExpression` для значения фильтра
- Создать `BinaryExpression` для сравнения
- Создать `LambdaExpression` из `BinaryExpression` и `ParameterExpression`
- Скомпилировать `LambdaExpression` в делегат
- Вызвать делегат для фильтрации коллекции

Шаг 10:

Миссия 10

Что позволяет делать `Dynamic LINQ` в `C#`?

Подсказка: Подумайте о возможности строить запросы во время выполнения программы.

Ответ:

- **Строить `LINQ`-запросы динамически во время выполнения**
- Компилировать код на лету
- Создавать асинхронные потоки данных
- Выполнять параллельные запросы без `LINQ`

Шаг 11:

Миссия 11

Какой класс используется для построения `Expression Tree`?

Подсказка: Подумайте о структуре выражений, которая описывает код.

Ответ:

- **`System.Linq.Expressions.Expression`**
- `System.Dynamic.DynamicObject`
- `System.Threading.Tasks.Task`
- `System.Linq.Enumerable`

Шаг 12:

Миссия 12

 Сопоставьте типы `Expression Tree` с их назначением:

Ответ:

- `Expression.Constant`
 - `Expression.Parameter`
 - `Expression.Add`
 - `Expression.Lambda`
1. Представляет константное значение
 2. Определяет параметр функции или лямбды
 3. Создает выражение сложения двух выражений
 4. Создает лямбда-выражение на основе `Expression`

Подсказка: Подумайте, какие типы выражений используются для значений, параметров, операций и функций.

- **Expression.Constant** — Представляет константное значение
- **Expression.Parameter** — Определяет параметр функции или лямбды
- **Expression.Add** — Создает выражение сложения двух выражений
- **Expression.Lambda** — Создает лямбда-выражение на основе Expression

🔗 Шаг 13:

Миссия 13

Какой метод позволяет выполнить Expression Tree?

Подсказка: Подумайте, какой шаг превращает дерево в выполняемый код.

Ответ:

- **Compile()**
- Execute()
- Run()
- Invoke()

🔗 Шаг 14:

Миссия 14

📄 Сопоставьте Dynamic LINQ методы с их функционалом:

Ответ:

- Where(string predicate)
 - Select(string selector)
 - OrderBy(string property)
 - GroupBy(string key)
1. Фильтрует элементы по строковому условию
 2. Проецирует элементы в новое представление по строке
 3. Сортирует элементы по свойству в строке
 4. Группирует элементы по ключу, указанному строкой

Подсказка: Подумайте, как строковые параметры управляют фильтрацией, проекцией, сортировкой и группировкой.

- **Where(string predicate)** — Фильтрует элементы по строковому условию
- **Select(string selector)** — Проецирует элементы в новое представление по строке
- **OrderBy(string property)** — Сортирует элементы по свойству в строке
- **GroupBy(string key)** — Группирует элементы по ключу, указанному строкой

🔗 Шаг 15:

Миссия 15

Расположите шаги построения и выполнения Expression Tree:

1. Создание Expression для каждого элемента (Constant, Parameter, Add)
2. Построение дерева с помощью Expression.Lambda
3. Компиляция дерева с Compile()
4. Вызов с результатом Execute или Invoke

Подсказка: Сначала создаются элементы дерева, затем дерево, потом компиляция и выполнение.

Ответ:

- Создание Expression для каждого элемента (Constant, Parameter, Add)
- Построение дерева с помощью Expression.Lambda
- Компиляция дерева с Compile()
- Вызов с результатом Execute или Invoke

 **Шаг 16:**

Миссия 16

В чем преимущество Dynamic LINQ перед обычным LINQ?

Подсказка: Подумайте о гибкости условий запросов.

Ответ:

- **Можно строить запросы на основе строковых выражений во время выполнения**
- Запросы всегда компилируются в IL код заранее
- Обеспечивает автоматическую параллельную обработку
- Всегда поддерживает только статически типизированные свойства

Урок 5 - LINQ с потоками данных

 **Шаг 01:**

Миссия 1

Разработчик хочет обработать поток данных и получить первые 10 элементов, удовлетворяющих условию. Что лучше использовать?

Подсказка: LINQ умеет работать с IEnumerable и потоками данных.

Ответ:

- **LINQ с Take() и Where()**
- Parallel LINQ
- Task.Run
- ThreadPool.QueueUserWorkItem

 **Шаг 02:**

Миссия 2

Какой метод позволяет обрабатывать данные по мере их поступления без полной загрузки в память?

Подсказка: Потокковое выполнение помогает экономить память.

Ответ:

- **IEnumerable с deferred execution**
- List.ToArray()
- Task.WaitAll
- ConcurrentQueue.Dequeue

 **Шаг 03:**

Миссия 3

Вы хотите параллельно обработать поток данных. Что выбрать?

Подсказка: LINQ поддерживает параллельное выполнение.

Ответ:

- **AsParallel()**
- IEnumerable
- ThreadPool.QueueUserWorkItem
- Task.Delay

Шаг 04:

Миссия 4

Какая особенность потокового LINQ важна для обработки больших данных?

Подсказка: Думайте о памяти и скорости выполнения.

Ответ:

- **Отложенное выполнение (deferred execution)**
- Сразу все элементы в массив
- Фильтрация через Task.Wait
- Построение полного дерева Expression сразу

Шаг 05:

Миссия 5

Какой метод позволяет преобразовать поток данных по мере поступления?

Подсказка: LINQ умеет проецировать элементы без полной загрузки.

Ответ:

- **Select()**
- ToList()
- Task.WaitAll
- GroupBy()

Шаг 06:

Миссия 6

 Сопоставьте методы LINQ с их действием для потоков данных:

Ответ:

- Where
 - Select
 - Take
 - Skip
1. Фильтрует элементы по условию
 2. Проецирует элементы в новый вид
 3. Берет первые N элементов
 4. Пропускает первые N элементов

Подсказка:

Каждый метод работает на последовательностях, поддерживая deferred execution.

- **Where** — Фильтрует элементы по условию
- **Select** — Проецирует элементы в новый вид

- **Take** — Берет первые N элементов
- **Skip** — Пропускает первые N элементов

Шаг 07:

Миссия 7

 Сопоставьте типы потоковых коллекций с их характеристикой:

Ответ:

- `IEnumerable`
 - `IAsyncEnumerable`
 - `ParallelQuery`
 - `List`
1. Последовательная обработка с deferred execution
 2. Асинхронная обработка элементов по мере поступления
 3. Параллельная обработка коллекции
 4. Полная коллекция в памяти

Подсказка:

Подумайте о том, как каждый тип работает с потоками и памятью.

- **`IEnumerable`** — Последовательная обработка с deferred execution
- **`IAsyncEnumerable`** — Асинхронная обработка элементов по мере поступления
- **`ParallelQuery`** — Параллельная обработка коллекции
- **`List`** — Полная коллекция в памяти

Шаг 08:

Миссия 8

 Сопоставьте методы LINQ с их параллельным поведением:

Ответ:

- `AsParallel`
 - `WithDegreeOfParallelism`
 - `ForAll`
 - `AsSequential`
1. Преобразует коллекцию для параллельной обработки
 2. Ограничивает число параллельных потоков
 3. Применяет действие ко всем элементам параллельно
 4. Возвращает последовательное выполнение

Подсказка:

Подумайте, как можно управлять параллельной обработкой данных.

- **`AsParallel`** — Преобразует коллекцию для параллельной обработки
- **`WithDegreeOfParallelism`** — Ограничивает число параллельных потоков
- **`ForAll`** — Применяет действие ко всем элементам параллельно
- **`AsSequential`** — Возвращает последовательное выполнение

Шаг 09:

Миссия 9

Расположите шаги обработки потока данных LINQ в правильном порядке:

1. Создать источник данных (IEnumerable/IAsyncEnumerable)
2. Применить Where() для фильтрации
3. Применить Select() для проекции
4. Применить Take() или Skip() при необходимости
5. Выполнить запрос (например, ToList или foreach)

Подсказка:

Сначала создаем источник, потом фильтруем и проецируем, затем берем нужные элементы и наконец выполняем запрос.

Ответ:

- Создать источник данных (IEnumerable/IAsyncEnumerable)
- Применить Where() для фильтрации
- Применить Select() для проекции
- Применить Take() или Skip() при необходимости
- Выполнить запрос (например, ToList или foreach)

Шаг 10:

Миссия 10

Что такое LINQ to Streams в C#?

Подсказка: Подумайте о последовательной обработке элементов данных в реальном времени.

Ответ:

- **LINQ для работы с потоками данных по мере их поступления**
- Метод для параллельной обработки коллекций
- Библиотека для синхронного доступа к базе данных
- Асинхронная очередь сообщений

Шаг 11:

Миссия 11

Какой метод позволяет преобразовать поток в LINQ-запрос?

Подсказка: Подумайте о том, как сделать поток совместимым с LINQ.

Ответ:

- **AsEnumerable()**
- ToStream()
- AsParallel()
- Streamify()

Шаг 12:

Миссия 12

 Сопоставьте ключевые элементы работы с потоками данных с их назначением:

Ответ:

- Buffer
- Throttle
- Select

- Where

1. Собирает элементы потока в пакеты фиксированного размера
2. Ограничивает скорость обработки элементов
3. Проецирует элементы потока в новое представление
4. Фильтрует элементы по условию

Подсказка: Подумайте о буфере, фильтрации, проекции и контроле скорости потока.

- **Buffer** — Собирает элементы потока в пакеты фиксированного размера
- **Throttle** — Ограничивает скорость обработки элементов
- **Select** — Проецирует элементы потока в новое представление
- **Where** — Фильтрует элементы по условию

Шаг 13:

Миссия 13

В чем преимущество LINQ с потоками данных по сравнению с обычным LINQ?

Подсказка: Подумайте о обработке данных по мере их поступления.

Ответ:

- **Обработка данных по мере поступления, без ожидания полной коллекции**
- Гарантированная последовательная обработка элементов
- Сокращение времени компиляции программы
- Автоматическое сохранение данных в базе

Шаг 14:

Миссия 14

 Сопоставьте методы LINQ для потоков с их функционалом:

Ответ:

- Take
 - Skip
 - Distinct
 - GroupBy
1. Берет первые N элементов потока
 2. Пропускает первые N элементов потока
 3. Удаляет дублирующиеся элементы
 4. Группирует элементы по ключу

Подсказка: Подумайте о том, как управлять последовательностью, уникальностью и группировкой в потоке.

- **Take** — Берет первые N элементов потока
- **Skip** — Пропускает первые N элементов потока
- **Distinct** — Удаляет дублирующиеся элементы
- **GroupBy** — Группирует элементы по ключу

Шаг 15:

Миссия 15

Расположите шаги обработки потока данных с LINQ в правильном порядке:

1. Подключение источника данных
2. Преобразование в поток с AsEnumerable()
3. Фильтрация элементов с Where
4. Проекция элементов с Select
5. Группировка или сортировка результатов
6. Подписка на поток для получения данных

Подсказка: Сначала делаем поток, потом фильтруем и проецируем, далее группируем и подписываемся на получение данных.

Ответ:

- Подключение источника данных
- Преобразование в поток с AsEnumerable()
- Фильтрация элементов с Where
- Проекция элементов с Select
- Группировка или сортировка результатов
- Подписка на поток для получения данных

Шаг 16:

Миссия 16

Что произойдет, если обработка потока данных выполняется медленно?

Подсказка: Подумайте о том, как это влияет на элементы, поступающие в поток.

Ответ:

- **Элементы будут накапливаться в буфере до обработки**
- Программа автоматически ускорит обработку
- Элементы будут теряться, если поток слишком длинный
- LINQ преобразует поток в синхронную коллекцию

Шаг 17:

Миссия 17

 Сопоставьте типичные проблемы при работе с потоками данных с их решениями:

Ответ:

- Перегрузка потока
 - Несоблюдение порядка элементов
 - Повторная обработка данных
 - Ошибка отмены обработки
1. Использовать буферизацию и Throttle
 2. Применять AsOrdered() или порядок вручную
 3. Использовать Distinct и Take/Skip
 4. Применять CancellationToken

Подсказка: Подумайте о контроле скорости, сохранении порядка, фильтрации и отмене потока.

- **Перегрузка потока** — Использовать буферизацию и Throttle
- **Несоблюдение порядка элементов** — Применять AsOrdered() или порядок вручную
- **Повторная обработка данных** — Использовать Distinct и Take/Skip
- **Ошибка отмены обработки** — Применять CancellationToken

Модуль 4 - Асинхронность и многопоточность

Урок 1 - asyncawait и Tasks

Шаг 01:

Миссия 1

Что делает ключевое слово async в методе C#?

Подсказка: Думайте о методах, которые возвращают задачи и могут выполняться асинхронно.

Ответ:

- **Позволяет использовать await внутри метода**
- Принудительно выполняет метод в отдельном потоке
- Синхронно блокирует поток до завершения метода
- Автоматически кэширует результаты метода

Шаг 02:

Миссия 2

Что возвращает метод, помеченный async и возвращающий Task<T>?

Подсказка: Подумайте, как можно получить результат асинхронной операции.

Ответ:

- **Объект Task, который завершится с результатом типа T**
- Сразу результат типа T
- Void
- Task без типа результата

Шаг 03:

Миссия 3

Что делает оператор await?

Подсказка: Он не блокирует поток, но приостанавливает выполнение до результата.

Ответ:

- **Приостанавливает выполнение метода до завершения Task**
- Синхронно блокирует текущий поток
- Запускает Task в новом процессе
- Автоматически отменяет Task

Шаг 04:

Миссия 4

Какая ошибка часто встречается при использовании async/await?

Подсказка: Подумайте о методах, возвращающих void.

Ответ:

- **Async метод возвращает void вместо Task или Task<T>**
- Метод объявлен как async без использования await
- Использование await вне async метода
- Синхронный метод вызывает Task.Run

Шаг 05:

Миссия 5

Для чего используется Task.WhenAll?

Подсказка: Подумайте о запуске нескольких задач одновременно и ожидании их завершения.

Ответ:

- **Ожидает завершения всех переданных задач**
- Запускает задачи последовательно
- Автоматически отменяет задачи при ошибке
- Создает новый поток для каждой задачи

Шаг 06:

Миссия 6

 Сопоставьте понятия async/await с их описанием:

Ответ:

- async
 - await
 - Task
 - Task<T>
1. Позволяет использовать await внутри метода
 2. Приостанавливает выполнение метода до завершения Task
 3. Представляет асинхронную операцию без результата
 4. Представляет асинхронную операцию с результатом типа T

 **Подсказка:**

Подумайте, что делает ключевое слово async и оператор await, и как Task и Task<T> связаны с результатами.

- **async** — Позволяет использовать await внутри метода
- **await** — Приостанавливает выполнение метода до завершения Task
- **Task** — Представляет асинхронную операцию без результата
- **Task** — Представляет асинхронную операцию с результатом типа T

Шаг 07:

Миссия 7

 Сопоставьте методы Task с их поведением:

Ответ:

- Task.Run
 - Task.Delay
 - Task.WhenAll
 - Task.WhenAny
1. Запускает работу в отдельном потоке
 2. Асинхронная задержка на заданное время
 3. Ожидает завершения всех задач
 4. Возвращает первую завершившуюся задачу

 **Подсказка:**

Подумайте о различии между запуском, ожиданием всех и ожиданием первой задачи.

- **Task.Run** — Запускает работу в отдельном потоке
- **Task.Delay** — Асинхронная задержка на заданное время
- **Task.WhenAll** — Ожидает завершения всех задач
- **Task.WhenAny** — Возвращает первую завершившуюся задачу

Шаг 08:

Миссия 8

 Сопоставьте типичные ошибки async/await с их объяснением:

Ответ:

- Async void
 - Deadlock
 - Игнорирование Task
 - Синхронный блок await
1. Метод не возвращает Task, что усложняет обработку ошибок
 2. Ожидание Task в синхронном контексте, блокирующее поток UI
 3. Не обрабатывается результат или исключение Task
 4. Использование async void вместо async Task для методов-обработчиков

Подсказка:

Подумайте о проблемах с блокировкой и обработкой ошибок в асинхронных методах.

- **Async void** — Метод не возвращает Task, что усложняет обработку ошибок
- **Deadlock** — Ожидание Task в синхронном контексте, блокирующее поток UI
- **Игнорирование Task** — Не обрабатывается результат или исключение Task
- **Синхронный блок await** — Использование async void вместо async Task для методов-обработчиков

Шаг 09:

Миссия 9

Расположите шаги выполнения асинхронного метода с Task в правильном порядке:

1. Метод объявляется как async
2. Внутри метода вызывается await для Task
3. Task запускается (например, Task.Run)
4. Основной поток продолжает выполнение
5. После завершения Task результат возвращается или обрабатывается

Подсказка:

Подумайте, как async/await управляет выполнением метода и потоком.

Ответ:

- Метод объявляется как async
- Внутри метода вызывается await для Task
- Task запускается (например, Task.Run)
- Основной поток продолжает выполнение
- После завершения Task результат возвращается или обрабатывается

Шаг 10:

Миссия 10

Что делает ключевое слово `async` в C#?

Подсказка: Подумайте о том, как метод становится асинхронным и возвращает задачу.

Ответ:

- **Позволяет методу выполняться асинхронно и возвращать `Task` или `Task`**
- Останавливает выполнение метода до завершения всех операций
- Автоматически создаёт новый поток для метода
- Позволяет методу выполняться синхронно быстрее

Шаг 11:

Миссия 11

Что делает оператор `await`?

Подсказка: Подумайте о том, как ожидание влияет на поток выполнения и UI.

Ответ:

- **Приостанавливает выполнение метода до завершения `Task` без блокировки потока**
- Блокирует текущий поток до завершения `Task`
- Создает новый поток для выполнения `Task`
- Прерывает выполнение `Task` при ошибке

Шаг 12:

Миссия 12

 Сопоставьте понятия `async/await` с их определениями:

Ответ:

- `async`
 - `await`
 - `Task`
 - `Task`
1. Позволяет объявить метод как асинхронный
 2. Приостанавливает выполнение метода до завершения `Task`
 3. Представляет асинхронную операцию без возвращаемого значения
 4. Представляет асинхронную операцию с возвращаемым значением

Подсказка: Подумайте, как объявляются асинхронные методы, как они ожидаются и что возвращают.

- **`async`** — Позволяет объявить метод как асинхронный
- **`await`** — Приостанавливает выполнение метода до завершения `Task`
- **`Task`** — Представляет асинхронную операцию без возвращаемого значения
- **`Task`** — Представляет асинхронную операцию с возвращаемым значением

Шаг 13:

Миссия 13

Почему стоит использовать `async/await` вместо `Task.Wait()` или `Task.Result`?

Подсказка: Подумайте о блокировке потоков и отзывчивости UI.

Ответ:

- Чтобы не блокировать поток и сохранить отзывчивость приложения
- Чтобы ускорить выполнение синхронных операций
- Чтобы автоматически перехватывать все исключения
- Чтобы использовать меньше памяти для Task

Шаг 14:

Миссия 14

 Сопоставьте типичные проблемы работы с Task и их решения:

Ответ:

- Deadlock
 - Unobserved Exception
 - Cancellation
 - Continuations порядок
1. Использовать await вместо Task.Wait()
 2. Использовать try/catch или TaskScheduler.UnobservedTaskException
 3. Применять CancellationToken
 4. Использовать ConfigureAwait и правильную последовательность continuations

Подсказка: Подумайте о правильном ожидании, обработке ошибок и отмене задач.

- **Deadlock** — Использовать await вместо Task.Wait()
- **Unobserved Exception** — Использовать try/catch или TaskScheduler.UnobservedTaskException
- **Cancellation** — Применять CancellationToken
- **Continuations порядок** — Использовать ConfigureAwait и правильную последовательность continuations

Шаг 15:

Миссия 15

Расположите шаги асинхронного метода в правильном порядке:

1. Определение метода с ключевым словом async
2. Вызов асинхронной операции с await
3. Обработка результата после await
4. Возврат Task или Task

Подсказка: Сначала объявляем метод, затем ожидаем задачу, потом обрабатываем результат и возвращаем Task.

Ответ:

- Определение метода с ключевым словом async
- Вызов асинхронной операции с await
- Обработка результата после await
- Возврат Task или Task

Шаг 16:

Миссия 16

Что произойдет, если забыть await перед Task?

Подсказка: Подумайте о том, как это влияет на выполнение метода и обработку ошибок.

Ответ:

- **Метод продолжит выполнение без ожидания Task, возможны пропущенные результаты и необработанные исключения**
- Task будет автоматически выполнен синхронно
- Компилятор выдаст ошибку
- Task завершится сразу с null

Урок 2 - Task Parallel Library

Шаг 01:

Миссия 1

Что делает метод Task.Run в C#?

Подсказка: Подумайте, как можно запустить работу в отдельном потоке.

Ответ:

- **Запускает указанную функцию в пуле потоков**
- Создает новый процесс
- Синхронно выполняет метод в текущем потоке
- Автоматически отменяет задачу при исключении

Шаг 02:

Миссия 2

Что делает Parallel.For в .NET?

Подсказка: Подумайте о запуске нескольких итераций цикла одновременно.

Ответ:

- **Параллельно выполняет цикл с множеством итераций**
- Последовательно выполняет цикл
- Создает Task для каждой итерации, но последовательно запускает их
- Запускает цикл в отдельном процессе

Шаг 03:

Миссия 3

Для чего используется Parallel.Invoke?

Подсказка: Думайте о параллельном запуске нескольких действий одновременно.

Ответ:

- **Запускает несколько делегатов одновременно и ожидает их завершения**
- Запускает делегаты последовательно
- Создает для каждого делегата новый процесс
- Автоматически разбивает делегаты на потоки, но выполняет один за другим

Шаг 04:

Миссия 4

Что делает метод Task.WaitAll?

Подсказка: Подумайте о синхронизации завершения нескольких задач.

Ответ:

- **Блокирует выполнение до завершения всех переданных задач**
- Запускает задачи последовательно
- Автоматически отменяет все задачи при ошибке
- Возвращает первую завершившуюся задачу

Шаг 05:

Миссия 5

Какая задача выполняется быстрее: последовательный цикл или `Parallel.For` с 1000 итераций на многоядерной машине?

Подсказка: Подумайте о преимуществах многопоточности и параллельного выполнения.

Ответ:

- **`Parallel.For`, если задачи независимы и ядра свободны**
- Последовательный цикл всегда быстрее
- Оба выполняются одинаково быстро
- Зависит только от объема памяти

Шаг 06:

Миссия 6

 Сопоставьте элементы Task Parallel Library с их описанием:

Ответ:

- `Task.Run`
 - `Parallel.For`
 - `Parallel.Invoke`
 - `Task.WaitAll`
1. Запускает работу в пуле потоков
 2. Параллельно выполняет цикл
 3. Параллельно выполняет несколько действий
 4. Ожидает завершения всех задач

Подсказка:

Подумайте о различиях между запуском отдельного `Task`, циклов и нескольких действий.

- **`Task.Run`** — Запускает работу в пуле потоков
- **`Parallel.For`** — Параллельно выполняет цикл
- **`Parallel.Invoke`** — Параллельно выполняет несколько действий
- **`Task.WaitAll`** — Ожидает завершения всех задач

Шаг 07:

Миссия 7

 Сопоставьте типичные проблемы параллельного выполнения:

Ответ:

- Состояние гонки
- Deadlock
- Oversubscription

- Исключение в Task

1. Одновременный доступ к ресурсу из нескольких потоков без синхронизации
2. Зависание потоков из-за ожидания блокировок
3. Слишком много параллельных задач для количества ядер
4. Необработанное исключение в параллельной задаче

Подсказка:

Подумайте о типичных ловушках при параллельной обработке.

- **Состояние гонки** — Одновременный доступ к ресурсу из нескольких потоков без синхронизации
- **Deadlock** — Зависание потоков из-за ожидания блокировок
- **Oversubscription** — Слишком много параллельных задач для количества ядер
- **Исключение в Task** — Необработанное исключение в параллельной задаче

Шаг 08:

Миссия 8

 Сопоставьте свойства Task с их описанием:

Ответ:

- IsCompleted
 - IsFaulted
 - IsCanceled
 - Status
1. Возвращает true, если задача завершилась успешно или с ошибкой
 2. Возвращает true, если задача завершилась с исключением
 3. Возвращает true, если задача была отменена
 4. Текущее состояние задачи

Подсказка:

Подумайте, как Task сообщает о своем состоянии и ошибках.

- **IsCompleted** — Возвращает true, если задача завершилась успешно или с ошибкой
- **IsFaulted** — Возвращает true, если задача завершилась с исключением
- **IsCanceled** — Возвращает true, если задача была отменена
- **Status** — Текущее состояние задачи

Шаг 09:

Миссия 9

Расположите этапы выполнения Parallel.For в правильном порядке:

1. Инициализация цикла и разбиение итераций на диапазоны
2. Запуск диапазонов на доступных потоках
3. Выполнение каждой итерации на своем потоке
4. Сбор результатов и завершение цикла

Подсказка:

Подумайте, как TPL раздает итерации и объединяет результаты.

Ответ:

- Инициализация цикла и разбиение итераций на диапазоны

- Запуск диапазонов на доступных потоках
- Выполнение каждой итерации на своем потоке
- Сбор результатов и завершение цикла

🔗 Шаг 10:

Миссия 10

Что делает Task Parallel Library (TPL) в C#?

Подсказка: Подумайте, как можно упростить запуск и управление множеством задач одновременно.

Ответ:

- **Обеспечивает простой запуск, управление и синхронизацию параллельных задач**
- Автоматически распределяет задачи только на один поток
- Используется только для синхронного выполнения кода
- Заменяет async/await во всех сценариях

🔗 Шаг 11:

Миссия 11

Что делает метод Task.Run()?

Подсказка: Подумайте, как можно запустить код в отдельном потоке без явного создания Thread.

Ответ:

- **Запускает задачу в пуле потоков и возвращает Task для отслеживания выполнения**
- Создает новый процесс для выполнения задачи
- Синхронно выполняет метод на текущем потоке
- Гарантирует выполнение задачи на отдельном ядре процессора

🔗 Шаг 12:

Миссия 12

📄 Сопоставьте понятия TPL с их определениями:

Ответ:

- Task
- Task
- Parallel.For
- Parallel.ForEach

1. Асинхронная операция без возвращаемого значения
2. Асинхронная операция с возвращаемым значением
3. Выполняет цикл с параллельными итерациями по индексам
4. Выполняет цикл с параллельными итерациями по коллекции

Подсказка: Подумайте, чем отличаются Task и Task, а также два варианта Parallel.

- **Task** — Асинхронная операция без возвращаемого значения
- **Task** — Асинхронная операция с возвращаемым значением
- **Parallel.For** — Выполняет цикл с параллельными итерациями по индексам
- **Parallel.ForEach** — Выполняет цикл с параллельными итерациями по коллекции

Шаг 13:

Миссия 13

Почему стоит использовать `Parallel.For` вместо обычного цикла `for` для больших вычислений?

Подсказка: Подумайте о распределении работы между потоками для ускорения выполнения.

Ответ:

- **Позволяет выполнять итерации одновременно на нескольких потоках, ускоряя выполнение**
- Гарантирует последовательное выполнение всех итераций
- Всегда экономит память за счет уменьшения `Task`
- Автоматически улучшает точность вычислений

Шаг 14:

Миссия 14

 Сопоставьте типичные проблемы TPL с их решениями:

Ответ:

- Deadlock
 - Необработанное исключение
 - Race Condition
 - Overhead
1. Использовать правильный синхронизатор и избегать `Task.Wait()`
 2. Применять `try/catch` или `TaskScheduler.UnobservedTaskException`
 3. Применять блокировки или `Concurrent` коллекции
 4. Не распараллеливать слишком мелкие задачи

Подсказка: Подумайте о блокировках, обработке ошибок, синхронизации и размере задач.

- **Deadlock** — Использовать правильный синхронизатор и избегать `Task.Wait()`
- **Необработанное исключение** — Применять `try/catch` или `TaskScheduler.UnobservedTaskException`
- **Race Condition** — Применять блокировки или `Concurrent` коллекции
- **Overhead** — Не распараллеливать слишком мелкие задачи

Шаг 15:

Миссия 15

Расположите шаги использования `Parallel.ForEach` в правильном порядке:

1. Создание коллекции данных
2. Определение действия для каждой итерации
3. Вызов `Parallel.ForEach` с коллекцией и действием
4. Обработка результатов после завершения всех итераций

Подсказка: Сначала подготавливаем данные, потом определяем действие, запускаем `Parallel.ForEach` и затем обрабатываем результаты.

Ответ:

- Создание коллекции данных
- Определение действия для каждой итерации
- Вызов `Parallel.ForEach` с коллекцией и действием

- Обработка результатов после завершения всех итераций

Шаг 16:

Миссия 16

Что такое `Parallel.Invoke()`?

Подсказка: Подумайте о запуске нескольких действий одновременно без использования циклов.

Ответ:

- **Метод позволяет выполнить несколько действий одновременно параллельно**
- Создает новый поток для каждого действия и ждёт их завершения синхронно
- Используется для последовательного выполнения делегатов
- Автоматически объединяет результаты всех действий в одну коллекцию

Шаг 17:

Миссия 17

 Сопоставьте методы TPL с их основным назначением:

Ответ:

- `Task.Run()`
 - `Parallel.For`
 - `Parallel.ForEach`
 - `Parallel.Invoke`
1. Запуск асинхронной задачи в пуле потоков
 2. Параллельное выполнение цикла по индексам
 3. Параллельное выполнение цикла по коллекции
 4. Параллельный запуск нескольких действий одновременно

Подсказка: Подумайте, какой метод удобен для задач, циклов и одновременного выполнения действий.

- **`Task.Run()`** — Запуск асинхронной задачи в пуле потоков
- **`Parallel.For`** — Параллельное выполнение цикла по индексам
- **`Parallel.ForEach`** — Параллельное выполнение цикла по коллекции
- **`Parallel.Invoke`** — Параллельный запуск нескольких действий одновременно

Урок 3 - Concurrent Collections

Шаг 01:

Миссия 1

Что такое `ConcurrentDictionary` в C#?

Подсказка: Подумайте о словаре, безопасном для многопоточности.

Ответ:

- **Словарь, поддерживающий безопасный доступ из нескольких потоков**
- Простой `Dictionary` без синхронизации
- Словарь, который автоматически сортирует элементы
- Словарь, который хранит только уникальные ключи

Шаг 02:

Миссия 2

Какой класс лучше использовать для потокобезопасной очереди FIFO?

Подсказка: Думайте о ConcurrentQueue.

Ответ:

- **ConcurrentQueue**
- Queue
- Stack
- ConcurrentStack

Шаг 03:

Миссия 3

Для чего используется ConcurrentBag?

Подсказка: Подумайте о коллекции, где порядок элементов не важен.

Ответ:

- **Хранение элементов без учета порядка с потокобезопасным доступом**
- Сортированное хранилище элементов
- Очередь с доступом только из одного потока
- Словарь ключ-значение с блокировкой

Шаг 04:

Миссия 4

Что делает метод TryAdd в ConcurrentDictionary?

Подсказка: Попробуйте вспомнить, как добавить ключ-значение, если ключ еще не существует.

Ответ:

- **Пытается добавить пару ключ-значение, возвращает false, если ключ уже существует**
- Добавляет ключ-значение и заменяет существующий ключ
- Всегда выбрасывает исключение, если ключ существует
- Просто обновляет значение по ключу

Шаг 05:

Миссия 5

 Сопоставьте коллекции с их особенностями:

Ответ:

- ConcurrentQueue
 - ConcurrentStack
 - ConcurrentBag
 - ConcurrentDictionary
1. FIFO, безопасная для потоков очередь
 2. LIFO, безопасный стек
 3. Неупорядоченное хранение элементов
 4. Потокобезопасный словарь ключ-значение

Подсказка:

Подумайте о назначении каждой коллекции.

- **ConcurrentQueue** — FIFO, безопасная для потоков очередь
- **ConcurrentStack** — LIFO, безопасный стек
- **ConcurrentBag** — Неупорядоченное хранение элементов
- **ConcurrentDictionary** — Потокобезопасный словарь ключ-значение

Шаг 06:

Миссия 6

 Сопоставьте методы ConcurrentDictionary с их назначением:

Ответ:

- TryAdd
 - TryRemove
 - TryUpdate
 - GetOrAdd
1. Добавляет пару ключ-значение, если ключ отсутствует
 2. Удаляет пару ключ-значение, если ключ существует
 3. Обновляет значение по ключу, если старое значение совпадает с ожидаемым
 4. Возвращает существующее значение или добавляет новое

Подсказка:

Подумайте о стандартных операциях над словарем в многопоточном режиме.

- **TryAdd** — Добавляет пару ключ-значение, если ключ отсутствует
- **TryRemove** — Удаляет пару ключ-значение, если ключ существует
- **TryUpdate** — Обновляет значение по ключу, если старое значение совпадает с ожидаемым
- **GetOrAdd** — Возвращает существующее значение или добавляет новое

Шаг 07:

Миссия 7

 Сопоставьте потокобезопасные коллекции с их использованием:

Ответ:

- ConcurrentQueue
 - ConcurrentStack
 - ConcurrentBag
 - ConcurrentDictionary
1. Обработка задач в порядке поступления
 2. Обработка последних добавленных элементов первыми
 3. Хранение временных объектов без порядка
 4. Счетчики или кэш с потокобезопасным доступом

Подсказка:

Подумайте о логике FIFO, LIFO и неупорядоченного хранения.

- **ConcurrentQueue** — Обработка задач в порядке поступления
- **ConcurrentStack** — Обработка последних добавленных элементов первыми

- **ConcurrentBag** — Хранение временных объектов без порядка
- **ConcurrentDictionary** — Счетчики или кэш с потокобезопасным доступом

Шаг 08:

Миссия 8

Расположите шаги использования ConcurrentQueue:

1. Создание ConcurrentQueue
2. Добавление элементов с помощью Enqueue
3. Обработка элементов с помощью TryDequeue
4. Проверка наличия элементов и завершение обработки

Подсказка:

Подумайте о логике работы очереди FIFO.

Ответ:

- Создание ConcurrentQueue
- Добавление элементов с помощью Enqueue
- Обработка элементов с помощью TryDequeue
- Проверка наличия элементов и завершение обработки

Шаг 09:

Миссия 9

Расположите этапы работы с ConcurrentDictionary:

1. Создание ConcurrentDictionary
2. Добавление или обновление элементов с помощью TryAdd или GetOrAdd
3. Удаление элементов с помощью TryRemove
4. Обход элементов для чтения

Подсказка:

Подумайте о безопасном добавлении, удалении и чтении.

Ответ:

- Создание ConcurrentDictionary
- Добавление или обновление элементов с помощью TryAdd или GetOrAdd
- Удаление элементов с помощью TryRemove
- Обход элементов для чтения

Шаг 10:

Миссия 10

Что отличает коллекции из пространства имен System.Collections.Concurrent от обычных коллекций?

Подсказка: Подумайте о потокобезопасности и возможности работы нескольких потоков одновременно.

Ответ:

- **Они потокобезопасны и позволяют одновременный доступ из нескольких потоков**
- Работают только в одном потоке без блокировок
- Автоматически сортируют элементы при добавлении
- Используются только для хранения строковых значений

🔗 Шаг 11:

Миссия 11

Какой из вариантов коллекций лучше всего подходит для частого добавления и удаления элементов в многопоточной среде?

Подсказка: Подумайте о структуре, которая поддерживает быстрый доступ и потокобезопасность.

Ответ:

- **ConcurrentBag**
- List
- Dictionary
- Stack

🔗 Шаг 12:

Миссия 12

📖 Сопоставьте коллекции Concurrent Collections с их особенностями:

Ответ:

- ConcurrentBag
 - ConcurrentQueue
 - ConcurrentStack
 - ConcurrentDictionary
1. Коллекция для работы с ключ-значение, потокобезопасная
 2. Очередь с потокобезопасными операциями FIFO
 3. Стек с потокобезопасными операциями LIFO
 4. Набор элементов без порядка, оптимизированный для добавления/удаления

Подсказка: Подумайте о поведении FIFO и LIFO, а также о работе с ключами.

- **ConcurrentBag** — Набор элементов без порядка, оптимизированный для добавления/удаления
- **ConcurrentQueue** — Очередь с потокобезопасными операциями FIFO
- **ConcurrentStack** — Стек с потокобезопасными операциями LIFO
- **ConcurrentDictionary** — Коллекция для работы с ключ-значение, потокобезопасная

🔗 Шаг 13:

Миссия 13

Как безопасно получить элемент из ConcurrentBag?

Подсказка: Подумайте, какие методы потокобезопасны для выборки элементов.

Ответ:

- **Использовать метод TryTake()**
- Прямой доступ по индексу
- Использовать foreach без блокировки
- Использовать метод ElementAt()

🔗 Шаг 14:

Миссия 14

📖 Сопоставьте методы ConcurrentDictionary с их функциями:

Ответ:

- TryAdd
- TryRemove
- GetOrAdd
- AddOrUpdate

1. Попытка добавить ключ-значение, возвращает false если ключ существует
2. Попытка удалить элемент по ключу
3. Возвращает существующее значение или добавляет новое, если ключ отсутствует
4. Добавляет элемент или обновляет значение, если ключ уже существует

Подсказка: Подумайте о безопасных операциях добавления, удаления и обновления.

- **TryAdd** — Попытка добавить ключ-значение, возвращает false если ключ существует
- **TryRemove** — Попытка удалить элемент по ключу
- **GetOrAdd** — Возвращает существующее значение или добавляет новое, если ключ отсутствует
- **AddOrUpdate** — Добавляет элемент или обновляет значение, если ключ уже существует

 Шаг 15:

Миссия 15

Расположите шаги безопасного использования ConcurrentQueue:

1. Создать очередь
2. Добавлять элементы с помощью Enqueue()
3. Извлекать элементы с помощью TryDequeue()
4. Обработать извлечённые элементы

Подсказка: Сначала создаём очередь, потом добавляем, извлекаем и обрабатываем элементы.

Ответ:

- Создать очередь
- Добавлять элементы с помощью Enqueue()
- Извлекать элементы с помощью TryDequeue()
- Обработать извлечённые элементы

 Шаг 16:

Миссия 16

Почему лучше использовать ConcurrentDictionary вместо Dictionary в многопоточной среде?

Подсказка: Подумайте о синхронизации и безопасной работе с ключами.

Ответ:

- **Он потокобезопасен и предотвращает ошибки при одновременном доступе из нескольких потоков**
- Он быстрее для однопоточной работы
- Он автоматически сортирует элементы по ключу
- Он поддерживает только строковые ключи

Урок 4 - SynchronizationContext и UI-потoki

Шаг 01:

Миссия 1

Что делает SynchronizationContext в C#?

Подсказка: Подумайте, как асинхронные операции возвращаются в нужный поток.

Ответ:

- **Определяет контекст, в котором выполняются продолжения async-методов**
- Создает новый поток для каждой задачи
- Блокирует UI-поток на время выполнения задачи
- Синхронизирует работу Task и ThreadPool автоматически

Шаг 02:

Миссия 2

Почему важно использовать SynchronizationContext в UI-приложениях?

Подсказка: Подумайте, что произойдет, если изменить элементы UI из другого потока.

Ответ:

- **Чтобы избежать ошибок при доступе к элементам UI из другого потока**
- Чтобы ускорить выполнение задач в ThreadPool
- Чтобы синхронизировать сетевые запросы
- Чтобы Task автоматически завершались в основном потоке

Шаг 03:

Миссия 3

Какой метод позволяет выполнить код в UI-потоке из асинхронного метода?

Подсказка: Подумайте о методах, которые используют Dispatcher или контекст синхронизации.

Ответ:

- **SynchronizationContext.Post или Dispatcher.Invoke**
- Thread.Sleep
- Task.Run без контекста
- Thread.Abort

Шаг 04:

Миссия 4

Что произойдет, если изменить элементы UI напрямую из фонового потока?

Подсказка: Подумайте о потокобезопасности и исключениях Windows Forms/WPF.

Ответ:

- **Произойдет InvalidOperationException или непредсказуемое поведение**
- Элементы UI изменятся без ошибок
- Задача автоматически выполнится в UI-потоке
- Приложение заблокируется навсегда

Шаг 05:

Миссия 5

 Сопоставьте объекты и их предназначение в контексте UI-потоков:

Ответ:

- SynchronizationContext
 - Dispatcher
 - TaskScheduler.FromCurrentSynchronizationContext
 - ThreadPool
1. Определяет поток, в котором выполняются продолжения async
 2. Позволяет выполнять код в UI-потоке WPF
 3. Позволяет привязать Task к текущему синхронизационному контексту
 4. Используется для фонового выполнения задач в пуле потоков

 **Подсказка:**

Подумайте о контексте и фоновой работе.

- **SynchronizationContext** — Определяет поток, в котором выполняются продолжения async
- **Dispatcher** — Позволяет выполнять код в UI-потоке WPF
- **TaskScheduler.FromCurrentSynchronizationContext** — Позволяет привязать Task к текущему синхронизационному контексту
- **ThreadPool** — Используется для фонового выполнения задач в пуле потоков

 **Шаг 06:**

Миссия 6

 Сопоставьте элементы Windows Forms/WPF и их методы для потокобезопасного обновления UI:

Ответ:

- Control
 - DispatcherObject
 - Button
 - TextBox
1. InvokeRequired и Invoke/BeginInvoke
 2. CheckAccess и Invoke/BeginInvoke
 3. Invoke для изменения состояния кнопки из другого потока
 4. Invoke для изменения текста из другого потока

 **Подсказка:**

Подумайте о базовых методах и наследовании объектов.

- **Control** — InvokeRequired и Invoke/BeginInvoke
- **DispatcherObject** — CheckAccess и Invoke/BeginInvoke
- **Button** — Invoke для изменения состояния кнопки из другого потока
- **TextBox** — Invoke для изменения текста из другого потока

 **Шаг 07:**

Миссия 7

 Сопоставьте понятия с описанием:

Ответ:

- UI-поток

- Фоновый поток
- Deadlock
- Continuation

1. Поток, в котором обновляются элементы интерфейса
2. Поток для выполнения фоновых операций
3. Ситуация, когда потоки блокируют друг друга навсегда
4. Код, выполняющийся после завершения Task

🔔 Подсказка:

Подумайте о роли каждого потока и о продолжениях async.

- **UI-поток** — Поток, в котором обновляются элементы интерфейса
- **Фоновый поток** — Поток для выполнения фоновых операций
- **Deadlock** — Ситуация, когда потоки блокируют друг друга навсегда
- **Continuation** — Код, выполняющийся после завершения Task

🔔 Шаг 08:

Миссия 8

Расположите шаги безопасного обновления UI из асинхронного метода:

1. Вызов асинхронного метода Task.Run
2. Получение SynchronizationContext текущего потока
3. Вызов SynchronizationContext.Post для выполнения кода в UI-потоке
4. Обновление элементов интерфейса внутри делегата

🔔 Подсказка:

Подумайте о последовательности: фон -> контекст -> UI.

Ответ:

- Вызов асинхронного метода Task.Run
- Получение SynchronizationContext текущего потока
- Вызов SynchronizationContext.Post для выполнения кода в UI-потоке
- Обновление элементов интерфейса внутри делегата

🔔 Шаг 09:

Миссия 9

Расположите действия для безопасного использования TaskScheduler.FromCurrentSynchronizationContext:

1. Создать TaskScheduler из текущего SynchronizationContext
2. Запустить Task с указанным TaskScheduler
3. Выполнить код внутри Task, обновляющий UI
4. Завершение Task

🔔 Подсказка:

Подумайте о связывании задачи с UI-поток через TaskScheduler.

Ответ:

- Создать TaskScheduler из текущего SynchronizationContext
- Запустить Task с указанным TaskScheduler
- Выполнить код внутри Task, обновляющий UI

- Завершение Task

Шаг 10:

Миссия 10

Что обеспечивает SynchronizationContext в UI-приложениях?

Подсказка: Подумайте, почему обновление элементов интерфейса должно происходить только из UI-потока.

Ответ:

- **Гарантирует выполнение кода в UI-потоке**
- Разрешает доступ к UI из любых потоков без ограничений
- Автоматически параллелит все задачи в фоне
- Блокирует все асинхронные операции

Шаг 11:

Миссия 11

Как правильно обновить элемент UI из асинхронного метода?

Подсказка: Подумайте о методах, которые используют контекст синхронизации для возвращения в UI-поток.

Ответ:

- **Использовать await с текущим SynchronizationContext**
- Прямо изменять элементы UI в Task.Run
- Использовать Thread.Sleep перед обновлением UI
- Создать новый поток для каждого обновления UI

Шаг 12:

Миссия 12

 Сопоставьте термины с их объяснением:

Ответ:

- SynchronizationContext
 - UI-поток
 - ConfigureAwait(false)
 - Dispatcher.Invoke
1. Контекст, обеспечивающий выполнение кода в определённом потоке
 2. Поток, который управляет интерфейсом приложения
 3. Продолжение задачи без возвращения в исходный контекст синхронизации
 4. Метод для синхронного выполнения кода в UI-потоке

Подсказка: Подумайте, что делает каждый термин при работе с асинхронностью и UI.

- **SynchronizationContext** — Контекст, обеспечивающий выполнение кода в определённом потоке
- **UI-поток** — Поток, который управляет интерфейсом приложения
- **ConfigureAwait(false)** — Продолжение задачи без возвращения в исходный контекст синхронизации
- **Dispatcher.Invoke** — Метод для синхронного выполнения кода в UI-потоке

Шаг 13:

Миссия 13

Что происходит, если попытаться обновить UI напрямую из фонового потока без `SynchronizationContext`?

Подсказка: Подумайте о типичных ошибках доступа к UI в WPF или WinForms.

Ответ:

- **Бросается исключение или возникает непредсказуемое поведение**
- UI обновляется корректно без проблем
- Задача автоматически переносится в UI-поток
- Приложение игнорирует обновление UI

Шаг 14:

Миссия 14

 Сопоставьте способы безопасного обновления UI с их описанием:

Ответ:

- `await Task.Run(...)`
 - `Dispatcher.Invoke(...)`
 - `Control.Invoke(...)`
 - `SynchronizationContext.Post(...)`
1. Асинхронный запуск кода в фоновой задаче
 2. Синхронное выполнение кода в UI-поток WPF
 3. Синхронное выполнение кода в UI-поток WinForms
 4. Асинхронное выполнение кода в UI-поток через контекст синхронизации

Подсказка: Подумайте, какие методы работают для WPF, WinForms и общих задач с `SynchronizationContext`.

- **`await Task.Run(...)`** — Асинхронный запуск кода в фоновой задаче
- **`Dispatcher.Invoke(...)`** — Синхронное выполнение кода в UI-поток WPF
- **`Control.Invoke(...)`** — Синхронное выполнение кода в UI-поток WinForms
- **`SynchronizationContext.Post(...)`** — Асинхронное выполнение кода в UI-поток через контекст синхронизации

Шаг 15:

Миссия 15

Расположите шаги безопасного обновления UI из асинхронного метода:

1. Создать асинхронную задачу
2. Выполнить тяжёлую работу в `Task.Run()`
3. Вернуться в UI-поток с помощью `SynchronizationContext` или `Dispatcher`
4. Обновить элементы интерфейса

Подсказка: Сначала выполняем задачу в фоне, затем безопасно возвращаемся в UI для обновления.

Ответ:

- Создать асинхронную задачу
- Выполнить тяжёлую работу в `Task.Run()`
- Вернуться в UI-поток с помощью `SynchronizationContext` или `Dispatcher`

- Обновить элементы интерфейса

Шаг 16:

Миссия 16

Зачем в UI-приложениях используют `ConfigureAwait(false)` в фоновых задачах?

Подсказка: Подумайте, когда контекст синхронизации не нужен и можно ускорить выполнение.

Ответ:

- **Чтобы продолжение задачи выполнялось в любом потоке без возврата в UI-поток**
- Чтобы принудительно вернуться в UI-поток
- Чтобы блокировать выполнение фоновой задачи
- Чтобы ускорить рендеринг UI автоматически

Шаг 17:

Миссия 17

 Сопоставьте типичные ошибки с правильным решением:

Ответ:

- Попытка обновления UI из `Task.Run` без контекста
 - Долгие операции в UI-потоке
 - Игнорирование `SynchronizationContext` в WinForms
 - Неиспользование `ConfigureAwait(false)` для фоновой работы
1. Использовать `SynchronizationContext.Post` или `Dispatcher.Invoke`
 2. Выполнять тяжёлую работу в `Task.Run()` или фоновых потоках
 3. Вернуться в UI-поток через `Control.Invoke`
 4. Ускоряет выполнение фоновой задачи и предотвращает блокировку UI

Подсказка: Сопоставляйте каждую ошибку с подходящим способом решения.

- **Попытка обновления UI из `Task.Run` без контекста** — Использовать `SynchronizationContext.Post` или `Dispatcher.Invoke`
- **Долгие операции в UI-потоке** — Выполнять тяжёлую работу в `Task.Run()` или фоновых потоках
- **Игнорирование `SynchronizationContext` в WinForms** — Вернуться в UI-поток через `Control.Invoke`
- **Неиспользование `ConfigureAwait(false)` для фоновой работы** — Ускоряет выполнение фоновой задачи и предотвращает блокировку UI

Урок 5 - Lock-free и CAS (Compare-And-Swap)

Шаг 01:

Миссия 1

Что такое Lock-free алгоритм?

Подсказка: Подумайте, что означает отсутствие блокировок в многопоточном коде.

Ответ:

- **Алгоритм, который позволяет нескольким потокам работать с данными без блокировок**
- Алгоритм, использующий только один поток
- Алгоритм, который всегда использует mutex для синхронизации

- Алгоритм, блокирующий потоки до завершения операции

Шаг 02:

Миссия 2

Что делает операция Compare-And-Swap (CAS)?

Подсказка: Подумайте, как атомарно изменить значение только при совпадении.

Ответ:

- **Сравнивает текущее значение с ожидаемым и, если совпадает, заменяет его новым**
- Всегда заменяет значение без проверки
- Блокирует поток до завершения операции
- Создает копию объекта для каждого потока

Шаг 03:

Миссия 3

Почему Lock-free структуры данных полезны в многопоточных приложениях?

Подсказка: Подумайте о производительности и блокировках.

Ответ:

- **Они повышают производительность и уменьшают вероятность блокировок**
- Они всегда создают новые потоки для каждой операции
- Они гарантируют последовательное выполнение потоков
- Они используют стандартные lock-объекты для синхронизации

Шаг 04:

Миссия 4

Что происходит, если CAS неудачен?

Подсказка: Подумайте, что делать, если значение изменилось другим потоком.

Ответ:

- **Операция повторяется до успеха или отбрасывается**
- CAS блокирует все потоки на время изменения
- CAS выбрасывает исключение и завершает программу
- Значение автоматически заменяется новым независимо от старого

Шаг 05:

Миссия 5

 Сопоставьте понятия Lock-free и CAS с их описанием:

Ответ:

- Lock-free
 - Wait-free
 - CAS
 - ABA проблема
1. Алгоритм, в котором хотя бы один поток делает прогресс
 2. Алгоритм, в котором все потоки гарантированно завершат операцию за конечное время
 3. Атомарная операция сравнения и замены значения

4. Ситуация, когда значение изменилось и вернулось обратно, маскируя изменение

Подсказка:

Подумайте о типах алгоритмов и особенностях CAS.

- **Lock-free** — Алгоритм, в котором хотя бы один поток делает прогресс
- **Wait-free** — Алгоритм, в котором все потоки гарантированно завершат операцию за конечное время
- **CAS** — Атомарная операция сравнения и замены значения
- **ABA проблема** — Ситуация, когда значение изменилось и вернулось обратно, маскируя изменение

Шаг 06:

Миссия 6

 Сопоставьте структуры данных и их особенности:

Ответ:

- ConcurrentStack
 - ConcurrentQueue
 - ConcurrentDictionary
 - AtomicInteger
1. Lock-free стек с поддержкой нескольких потоков
 2. Lock-free очередь с поддержкой нескольких потоков
 3. Потокбезопасный словарь без использования блокировок
 4. Атомарный счетчик, поддерживающий CAS

Подсказка:

Подумайте о типичных lock-free структурах в .NET.

- **ConcurrentStack** — Lock-free стек с поддержкой нескольких потоков
- **ConcurrentQueue** — Lock-free очередь с поддержкой нескольких потоков
- **ConcurrentDictionary** — Потокбезопасный словарь без использования блокировок
- **AtomicInteger** — Атомарный счетчик, поддерживающий CAS

Шаг 07:

Миссия 7

 Сопоставьте методы с их применением в CAS:

Ответ:

- Interlocked.CompareExchange
 - Volatile.Read
 - Volatile.Write
 - SpinWait
1. Атомарная операция сравнения и замены
 2. Чтение значения без оптимизаций компилятора и кэша
 3. Запись значения с немедленной видимостью для других потоков
 4. Ждем повторную попытку CAS без блокировок

Подсказка:

Подумайте об атомарных операциях и синхронизации.

- **Interlocked.CompareExchange** — Атомарная операция сравнения и замены
- **Volatile.Read** — Чтение значения без оптимизаций компилятора и кэша
- **Volatile.Write** — Запись значения с немедленной видимостью для других потоков
- **SpinWait** — Ждем повторную попытку CAS без блокировок

Шаг 08:

Миссия 8

Расположите шаги выполнения CAS:

1. Чтение текущего значения
2. Сравнение с ожидаемым значением
3. Попытка замены значения новым
4. Повтор операции при неудаче

Подсказка:

Подумайте о логике атомарной замены значения.

Ответ:

- Чтение текущего значения
- Сравнение с ожидаемым значением
- Попытка замены значения новым
- Повтор операции при неудаче

Шаг 09:

Миссия 9

Что является ключевым принципом lock-free программирования?

Подсказка: Подумайте о том, как можно избежать блокировок при работе с общими ресурсами.

Ответ:

- **Обеспечение корректности без блокировок**
- Использование монополий и mutex на каждый объект
- Полная сериализация всех потоков
- Запрещение доступа к общим ресурсам из нескольких потоков

Шаг 10:

Миссия 10

Для чего используется операция Compare-And-Swap (CAS)?

Подсказка: Подумайте о том, как безопасно обновлять переменные в многопоточной среде без блокировок.

Ответ:

- **Атомарное обновление значения при совпадении с ожидаемым**
- Блокировка потока до завершения другой операции
- Создание копии переменной для каждого потока
- Использование событий для синхронизации потоков

Шаг 11:

Миссия 11

 Сопоставьте термины с их объяснением:

Ответ:

- Lock-free
 - Wait-free
 - CAS (Compare-And-Swap)
 - ABA-проблема
1. Алгоритм, где операции могут завершиться за конечное время, не блокируя другие потоки
 2. Алгоритм, где каждая операция гарантированно завершится за конечное время
 3. Атомарная операция сравнения и замены значения
 4. Ситуация, когда значение меняется и возвращается к исходному, вводя в заблуждение CAS

Подсказка: Подумайте о том, как CAS используется в lock-free структурах и какие проблемы могут возникнуть.

- **Lock-free** — Алгоритм, где операции могут завершиться за конечное время, не блокируя другие потоки
- **Wait-free** — Алгоритм, где каждая операция гарантированно завершится за конечное время
- **CAS (Compare-And-Swap)** — Атомарная операция сравнения и замены значения
- **ABA-проблема** — Ситуация, когда значение меняется и возвращается к исходному, вводя в заблуждение CAS

 Шаг 12:

Миссия 12

Почему стандартный lock может быть медленнее lock-free решения?

Подсказка: Подумайте о том, что блокировки делают с потоками, которые хотят выполнить операцию.

Ответ:

- **Потоки блокируются и ждут освобождения ресурса, создавая задержки**
- Lock использует CAS и выполняется быстрее
- Lock автоматически параллелизирует доступ к ресурсу
- Lock отключает все потоки кроме одного

 Шаг 13:

Миссия 13

 Сопоставьте структуры данных с их характеристикой в lock-free реализации:

Ответ:

- ConcurrentStack
 - ConcurrentQueue
 - ConcurrentDictionary
 - AtomicInteger
1. Lock-free стек с атомарными операциями Push и Pop
 2. Lock-free очередь для многопоточного доступа
 3. Словарь с атомарными операциями добавления и обновления
 4. Атомарный счетчик с CAS

Подсказка: Подумайте, какие структуры используют атомарные операции для потокобезопасности.

- **ConcurrentStack** — Lock-free стек с атомарными операциями Push и Pop
- **ConcurrentQueue** — Lock-free очередь для многопоточного доступа
- **ConcurrentDictionary** — Словарь с атомарными операциями добавления и обновления
- **AtomicInteger** — Атомарный счетчик с CAS

Шаг 14:

Миссия 14

Расположите шаги безопасного обновления переменной с помощью CAS:

1. Прочитать текущее значение переменной
2. Вычислить новое значение на основе прочитанного
3. Попытаться атомарно заменить старое значение на новое с помощью CAS
4. Если CAS неудачен, повторить процесс

Подсказка: Сначала получаем текущее значение, затем пробуем заменить, повторяя при конфликте.

Ответ:

- Прочитать текущее значение переменной
- Вычислить новое значение на основе прочитанного
- Попытаться атомарно заменить старое значение на новое с помощью CAS
- Если CAS неудачен, повторить процесс

Шаг 15:

Миссия 15

Что делает ABA-проблема сложной для lock-free алгоритмов?

Подсказка: Подумайте, что может случиться, если значение изменилось и вернулось к исходному без уведомления об этом.

Ответ:

- **CAS может считать, что значение не менялось, хотя оно изменялось и вернулось обратно**
- ABA предотвращает любые блокировки
- ABA автоматически исправляется через Task.Run
- ABA делает алгоритм wait-free по определению

Модуль 5 - Паттерны проектирования и архитектура

Урок 1 - Основные паттерны

Шаг 01:

Миссия 1

Какой паттерн подходит для создания единственного объекта в приложении?

Подсказка: Подумайте о глобальном доступе к одному ресурсу.

Ответ:

- **Singleton**
- Factory Method

- Observer
- Strategy

Шаг 02:

Миссия 2

Какой паттерн позволяет создавать семейство взаимосвязанных объектов без указания их конкретных классов?

Подсказка: Подумайте о фабриках для группы объектов.

Ответ:

- **Abstract Factory**
- Decorator
- Adapter
- Prototype

Шаг 03:

Миссия 3

Какой паттерн помогает изменять поведение объекта без изменения его кода?

Подсказка: Подумайте о стратегиях и замене алгоритмов на лету.

Ответ:

- **Strategy**
- Singleton
- Observer
- Factory Method

Шаг 04:

Миссия 4

Какой паттерн позволяет объектам подписываться на события других объектов?

Подсказка: Подумайте о механизме уведомлений.

Ответ:

- **Observer**
- Decorator
- Adapter
- Facade

Шаг 05:

Миссия 5

 Сопоставьте паттерны с их описанием:

Ответ:

- Singleton
- Factory Method
- Abstract Factory
- Builder

1. Создание одного уникального объекта
2. Создание объектов через наследование

3. Создание семейства связанных объектов
4. Пошаговое построение сложного объекта

 Подсказка:

Подумайте о различии между уникальным объектом, фабрикой и построением сложных объектов.

- **Singleton** — Создание одного уникального объекта
- **Factory Method** — Создание объектов через наследование
- **Abstract Factory** — Создание семейства связанных объектов
- **Builder** — Пошаговое построение сложного объекта

 Шаг 06:

Миссия 6

 Сопоставьте паттерны структурного уровня с их целью:

Ответ:

- Adapter
 - Decorator
 - Facade
 - Composite
1. Приведение интерфейсов к совместимому виду
 2. Динамическое добавление функциональности объекту
 3. Упрощение сложной подсистемы
 4. Компоновка объектов в древовидную структуру

 Подсказка:

Подумайте о назначении каждого структурного паттерна.

- **Adapter** — Приведение интерфейсов к совместимому виду
- **Decorator** — Динамическое добавление функциональности объекту
- **Facade** — Упрощение сложной подсистемы
- **Composite** — Компоновка объектов в древовидную структуру

 Шаг 07:

Миссия 7

 Сопоставьте поведенческие паттерны с их функцией:

Ответ:

- Observer
 - Strategy
 - Command
 - Memento
1. Подписка на события и уведомления
 2. Инкапсуляция алгоритма для гибкой замены
 3. Инкапсуляция команды для отложенного выполнения
 4. Сохранение состояния объекта для последующего восстановления

 Подсказка:

Подумайте, какие паттерны помогают управлять поведением и состоянием объектов.

- **Observer** — Подписка на события и уведомления
- **Strategy** — Инкапсуляция алгоритма для гибкой замены
- **Command** — Инкапсуляция команды для отложенного выполнения
- **Memento** — Сохранение состояния объекта для последующего восстановления

🔮 Шаг 08:

Миссия 8

Расположите шаги применения паттерна Builder:

1. Создание объекта Director
2. Инициализация Builder с параметрами
3. Пошаговое построение частей объекта
4. Получение готового объекта

🔮 Подсказка:

Подумайте о последовательности построения сложного объекта шаг за шагом.

Ответ:

- Создание объекта Director
- Инициализация Builder с параметрами
- Пошаговое построение частей объекта
- Получение готового объекта

🔮 Шаг 09:

Миссия 9

Расположите шаги использования паттерна Observer:

1. Создание Subject
2. Подписка Observers на Subject
3. Изменение состояния Subject
4. Уведомление всех Observers

🔮 Подсказка:

Подумайте о логике уведомления подписчиков при изменении состояния.

Ответ:

- Создание Subject
- Подписка Observers на Subject
- Изменение состояния Subject
- Уведомление всех Observers

🔮 Шаг 10:

Миссия 10

Какой паттерн проектирования подходит для создания одного глобального объекта, доступного во всей программе?

Подсказка: Подумайте о том, как гарантировать, что объект создается только один раз.

Ответ:

- **Singleton**
- Factory Method

- Observer
- Strategy

Шаг 11:

Миссия 11

Какой паттерн позволяет менять поведение объекта во время выполнения, не изменяя его класс?

Подсказка: Подумайте о том, как объект может делегировать действия различным алгоритмам.

Ответ:

- **Strategy**
- Singleton
- Adapter
- Decorator

Шаг 12:

Миссия 12

 Сопоставьте паттерны с их назначением:

Ответ:

- Singleton
 - Observer
 - Factory Method
 - Decorator
1. Создание единственного глобального объекта
 2. Уведомление зависимых объектов при изменении состояния
 3. Создание объектов через наследуемый метод
 4. Динамическое расширение функциональности объекта

Подсказка: Подумайте, какой паттерн отвечает за создание, а какой — за поведение и расширение.

- **Singleton** — Создание единственного глобального объекта
- **Observer** — Уведомление зависимых объектов при изменении состояния
- **Factory Method** — Создание объектов через наследуемый метод
- **Decorator** — Динамическое расширение функциональности объекта

Шаг 13:

Миссия 13

Какой паттерн позволяет объекту принимать различные представления, не изменяя его логику?

Подсказка: Подумайте о разделении модели и представления.

Ответ:

- **MVC (Model-View-Controller)**
- Strategy
- Observer
- Adapter

Шаг 14:

Миссия 14

 Сопоставьте паттерны с их преимуществами:

Ответ:

- Adapter
- Facade
- Composite
- Proxy

1. Позволяет использовать несовместимые интерфейсы
2. Упрощает доступ к сложной системе
3. Объединяет объекты в древовидную структуру
4. Контролирует доступ к объекту и добавляет дополнительную логику

Подсказка: Подумайте, какие паттерны облегчают интеграцию и управление сложными структурами.

- **Adapter** — Позволяет использовать несовместимые интерфейсы
- **Facade** — Упрощает доступ к сложной системе
- **Composite** — Объединяет объекты в древовидную структуру
- **Proxy** — Контролирует доступ к объекту и добавляет дополнительную логику

Шаг 15:

Миссия 15

Расположите шаги применения паттерна Builder для создания сложного объекта:

1. Определение структуры объекта и необходимых компонентов
2. Создание класса Builder с методами для каждого компонента
3. Пошаговое создание объекта через Builder
4. Получение готового объекта

Подсказка: Сначала определяем структуру, затем строим объект шаг за шагом.

Ответ:

- Определение структуры объекта и необходимых компонентов
- Создание класса Builder с методами для каждого компонента
- Пошаговое создание объекта через Builder
- Получение готового объекта

Шаг 16:

Миссия 16

Какой паттерн полезен для обеспечения расширяемости функциональности объекта без изменения его кода?

Подсказка: Подумайте о динамическом добавлении поведения объекту.

Ответ:

- **Decorator**
- Singleton
- Observer
- Builder

Шаг 17:

Миссия 17

Расположите шаги реализации паттерна Observer для уведомления подписчиков:

1. Определение субъекта и списка подписчиков
2. Создание методов для добавления и удаления подписчиков
3. Определение метода уведомления всех подписчиков о событии
4. Вызов метода уведомления при изменении состояния субъекта

Подсказка: Сначала создаем структуру, потом методы управления подписчиками, затем уведомляем при изменении.

Ответ:

- Определение субъекта и списка подписчиков
- Создание методов для добавления и удаления подписчиков
- Определение метода уведомления всех подписчиков о событии
- Вызов метода уведомления при изменении состояния субъекта

Урок 2 - Dependency Injection

Шаг 01:

Миссия 1

Что является основной целью Dependency Injection?

Подсказка: Подумайте о том, как уменьшить зависимость классов друг от друга.

Ответ:

- **Ослабление зависимостей между классами**
- Ускорение выполнения программы
- Создание синглтона для всех сервисов
- Добавление глобальных переменных

Шаг 02:

Миссия 2

Какой способ внедрения зависимостей является самым гибким?

Подсказка: Подумайте о том, что позволяет легко подменять реализации.

Ответ:

- **Через конструктор**
- Через глобальные переменные
- Через статические методы
- Через приватные поля без сеттеров

Шаг 03:

Миссия 3

Какой паттерн тесно связан с Dependency Injection?

Подсказка: Подумайте о контейнерах и управлении зависимостями.

Ответ:

- **Inversion of Control (IoC)**
- Singleton

- Observer
- Builder

Шаг 04:

Миссия 4

Что чаще всего внедряется через Dependency Injection?

Подсказка: Подумайте о сервисах и компонентах приложения.

Ответ:

- **Сервисы и репозитории**
- Локальные переменные метода
- Константы
- UI-элементы

Шаг 05:

Миссия 5

 Сопоставьте типы внедрения зависимостей с их характеристикой:

Ответ:

- Через конструктор
 - Через сеттер
 - Через интерфейс
 - Через метод
1. Обеспечивает неизменяемость зависимостей после создания объекта
 2. Позволяет менять зависимости после создания объекта
 3. Обеспечивает слабую связность через контракт
 4. Передача зависимости на момент вызова метода

 **Подсказка:**

Подумайте о времени и способе передачи зависимости объекту.

- **Через конструктор** — Обеспечивает неизменяемость зависимостей после создания объекта
- **Через сеттер** — Позволяет менять зависимости после создания объекта
- **Через интерфейс** — Обеспечивает слабую связность через контракт
- **Через метод** — Передача зависимости на момент вызова метода

Шаг 06:

Миссия 6

 Сопоставьте преимущества DI с описанием:

Ответ:

- Слабая связность
 - Тестируемость
 - Поддерживаемость
 - Гибкость
1. Объекты легко заменять и модифицировать
 2. Облегчает написание unit-тестов
 3. Модули слабо зависят друг от друга

4. Можно менять реализации без изменения клиента

 Подсказка:

Подумайте, как DI влияет на качество кода и тестирование.

- **Слабая связность** — Модули слабо зависят друг от друга
- **Тестируемость** — Облегчает написание unit-тестов
- **Поддерживаемость** — Объекты легко заменять и модифицировать
- **Гибкость** — Можно менять реализации без изменения клиента

 Шаг 07:

Миссия 7

 Сопоставьте типичные ошибки при использовании DI с их описанием:

Ответ:

- Слишком много зависимостей в конструкторе
 - Жесткая привязка к конкретной реализации
 - Неинициализированные зависимости
 - Круговые зависимости
1. Создает сложности при тестировании и поддержке
 2. Трудно подменять объекты на моки
 3. Приводит к `NullReferenceException`
 4. Вызывает `StackOverflow` или ошибки сборки

 Подсказка:

Подумайте о том, как DI может быть неправильно применен.

- **Слишком много зависимостей в конструкторе** — Создает сложности при тестировании и поддержке
- **Жесткая привязка к конкретной реализации** — Трудно подменять объекты на моки
- **Неинициализированные зависимости** — Приводит к `NullReferenceException`
- **Круговые зависимости** — Вызывает `StackOverflow` или ошибки сборки

 Шаг 08:

Миссия 8

Расположите шаги внедрения зависимости через конструктор:

1. Определение интерфейса зависимости
2. Создание реализации интерфейса
3. Добавление параметра в конструктор класса клиента
4. Передача зависимости при создании объекта клиента

 Подсказка:

Подумайте о порядке подготовки интерфейса, реализации и передачи зависимостей.

Ответ:

- Определение интерфейса зависимости
- Создание реализации интерфейса
- Добавление параметра в конструктор класса клиента
- Передача зависимости при создании объекта клиента

Шаг 09:

Миссия 9

Что является основной целью Dependency Injection?

Подсказка: Подумайте, как уменьшить зависимость классов друг от друга.

Ответ:

- **Ослабление зависимостей между компонентами**
- Ускорение работы программы
- Увеличение количества статических методов
- Использование только глобальных переменных

Шаг 10:

Миссия 10

Какая форма внедрения зависимостей подразумевает передачу объекта через конструктор?

Подсказка: Подумайте, как объект получает зависимости сразу при создании.

Ответ:

- **Constructor Injection**
- Property Injection
- Method Injection
- Service Locator

Шаг 11:

Миссия 11

 Сопоставьте типы внедрения зависимостей с их описанием:

Ответ:

- Constructor Injection
 - Property Injection
 - Method Injection
 - Service Locator
1. Передача зависимостей через конструктор класса
 2. Передача зависимостей через свойства (сеттеры) объекта
 3. Передача зависимостей через параметры метода
 4. Использование глобального реестра для получения зависимостей

Подсказка: Обратите внимание, как зависимости поступают к объекту — сразу при создании, через свойства, методы или через сервис-локатор.

- **Constructor Injection** — Передача зависимостей через конструктор класса
- **Property Injection** — Передача зависимостей через свойства (сеттеры) объекта
- **Method Injection** — Передача зависимостей через параметры метода
- **Service Locator** — Использование глобального реестра для получения зависимостей

Шаг 12:

Миссия 12

Почему использование DI повышает тестируемость кода?

Подсказка: Подумайте, как можно подменять реальные зависимости на заглушки или моки.

Ответ:

- **Потому что зависимости можно легко подменить моками или заглушками**
- Потому что код становится быстрее
- Потому что уменьшается количество классов
- Потому что DI автоматически генерирует тесты

Шаг 13:

Миссия 13

 Сопоставьте преимущества DI с объяснением:

Ответ:

- Ослабление связности
 - Повышение тестируемости
 - Упрощение поддержки
 - Гибкость конфигурации
1. Классы меньше зависят друг от друга
 2. Легко подставлять моки для тестов
 3. Проще изменять и расширять код без изменений зависимых классов
 4. Легко менять реализации зависимостей без изменения кода клиента

Подсказка: Вспомните, как DI влияет на структуру и конфигурацию приложения.

- **Ослабление связности** — Классы меньше зависят друг от друга
- **Повышение тестируемости** — Легко подставлять моки для тестов
- **Упрощение поддержки** — Проще изменять и расширять код без изменений зависимых классов
- **Гибкость конфигурации** — Легко менять реализации зависимостей без изменения кода клиента

Шаг 14:

Миссия 14

Расположите шаги внедрения Dependency Injection через конструктор:

1. Определение зависимостей класса
2. Создание интерфейсов для зависимостей
3. Передача зависимостей через конструктор при создании объекта
4. Использование объекта с внедренными зависимостями

Подсказка: Сначала определяем что нужно классу, затем создаем интерфейсы, передаем зависимости, и используем объект.

Ответ:

- Определение зависимостей класса
- Создание интерфейсов для зависимостей
- Передача зависимостей через конструктор при создании объекта
- Использование объекта с внедренными зависимостями

Шаг 15:

Миссия 15

Что из нижеперечисленного является недостатком DI при неправильном использовании?

Подсказка: Подумайте, как слишком сложная конфигурация может повлиять на проект.

Ответ:

- **Сложная конфигурация и возможная путаница в контейнерах**
- Невозможность подменять зависимости
- Отсутствие интерфейсов
- Автоматическое уменьшение связности

🔮 **Шаг 16:**

Миссия 16

Расположите порядок действий для внедрения DI через свойство (Property Injection):

1. Определение свойств, которые будут получать зависимости
2. Создание объектов зависимостей
3. Установка зависимостей через свойства после создания объекта
4. Использование объекта с установленными зависимостями

Подсказка: Сначала объявляем свойства, затем создаем зависимости, присваиваем их и используем объект.

Ответ:

- Определение свойств, которые будут получать зависимости
- Создание объектов зависимостей
- Установка зависимостей через свойства после создания объекта
- Использование объекта с установленными зависимостями

📖 **Урок 3 - SOLID и Clean Architecture**

🔮 **Шаг 01:**

Миссия 1

Какой принцип SOLID отвечает за то, что класс должен иметь только одну причину для изменения?

Подсказка: Подумайте о разделении ответственности внутри класса.

Ответ:

- **Single Responsibility Principle (SRP)**
- Open-Closed Principle (OCP)
- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)

🔮 **Шаг 02:**

Миссия 2

Какой принцип SOLID говорит, что объекты должны быть открыты для расширения, но закрыты для изменения?

Подсказка: Подумайте о возможности добавления функционала без редактирования существующего кода.

Ответ:

- **Open-Closed Principle (OCP)**
- Dependency Inversion Principle (DIP)
- Single Responsibility Principle (SRP)

- Liskov Substitution Principle (LSP)

Шаг 03:

Миссия 3

Какой принцип SOLID помогает обеспечивать заменяемость объектов наследниками без поломки работы программы?

Подсказка: Подумайте о поведении наследников относительно базового класса.

Ответ:

- **Liskov Substitution Principle (LSP)**
- Interface Segregation Principle (ISP)
- Dependency Inversion Principle (DIP)
- Single Responsibility Principle (SRP)

Шаг 04:

Миссия 4

Какой принцип SOLID говорит о том, что клиенты не должны зависеть от интерфейсов, которые они не используют?

Подсказка: Подумайте о разделении больших интерфейсов на более узкие.

Ответ:

- **Interface Segregation Principle (ISP)**
- Single Responsibility Principle (SRP)
- Open-Closed Principle (OCP)
- Dependency Inversion Principle (DIP)

Шаг 05:

Миссия 5

 Сопоставьте принципы SOLID с их описанием:

Ответ:

- SRP
- OCP
- LSP
- ISP
- DIP

1. Класс должен иметь только одну причину для изменения
2. Объекты должны быть открыты для расширения, но закрыты для изменения
3. Наследники должны заменять базовые классы без нарушения работы
4. Клиенты не должны зависеть от интерфейсов, которые они не используют
5. Модули высокого уровня не должны зависеть от модулей низкого уровня

Подсказка:

Подумайте о сути каждого принципа и о том, на что он влияет в архитектуре приложения.

- **SRP** — Класс должен иметь только одну причину для изменения
- **OCP** — Объекты должны быть открыты для расширения, но закрыты для изменения
- **LSP** — Наследники должны заменять базовые классы без нарушения работы
- **ISP** — Клиенты не должны зависеть от интерфейсов, которые они не используют

- **DIP** — Модули высокого уровня не должны зависеть от модулей низкого уровня

Шаг 06:

Миссия 6

 Сопоставьте компоненты Clean Architecture с их уровнем:

Ответ:

- Entities
 - Use Cases
 - Interface Adapters
 - Frameworks & Drivers
1. Бизнес-объекты с правилами
 2. Бизнес-логика приложения
 3. Презентация и преобразование данных
 4. Внешние системы и инструменты

Подсказка:

Подумайте, какой уровень ближе к ядру приложения, а какой — к внешним системам.

- **Entities** — Бизнес-объекты с правилами
- **Use Cases** — Бизнес-логика приложения
- **Interface Adapters** — Презентация и преобразование данных
- **Frameworks & Drivers** — Внешние системы и инструменты

Шаг 07:

Миссия 7

 Сопоставьте преимущества Clean Architecture с их описанием:

Ответ:

- Тестируемость
 - Гибкость
 - Поддерживаемость
 - Изоляция бизнес-логики
1. Можно легко писать unit-тесты
 2. Можно менять внешние системы без изменения бизнес-логики
 3. Облегчает внесение изменений в код
 4. Бизнес-правила не зависят от фреймворков

Подсказка:

Подумайте, как архитектура влияет на качество и масштабируемость кода.

- **Тестируемость** — Можно легко писать unit-тесты
- **Гибкость** — Можно менять внешние системы без изменения бизнес-логики
- **Поддерживаемость** — Облегчает внесение изменений в код
- **Изоляция бизнес-логики** — Бизнес-правила не зависят от фреймворков

Шаг 08:

Миссия 8

Расположите уровни Clean Architecture от ядра до внешних слоев:

1. Entities
2. Use Cases
3. Interface Adapters
4. Frameworks & Drivers

💡 **Подсказка:**

Подумайте о том, какие компоненты ближе к бизнес-логике, а какие к внешним системам.

Ответ:

- Entities
- Use Cases
- Interface Adapters
- Frameworks & Drivers

💡 **Шаг 09:**

Миссия 9

Какой принцип SOLID отвечает за то, что класс должен иметь только одну ответственность?

Подсказка: Подумайте, что произойдет, если класс делает слишком много разных вещей.

Ответ:

- **Single Responsibility Principle (SRP)**
- Open/Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle

💡 **Шаг 10:**

Миссия 10

Какой принцип SOLID позволяет расширять функциональность класса без изменения его исходного кода?

Подсказка: Подумайте, какой принцип помогает добавлять новые возможности, не ломая старые.

Ответ:

- **Open/Closed Principle (OCP)**
- Dependency Inversion Principle
- Liskov Substitution Principle
- Interface Segregation Principle

💡 **Шаг 11:**

Миссия 11

📄 Сопоставьте принципы SOLID с их описанием:

Ответ:

- SRP
- OCP
- LSP
- ISP

- DIP

1. Класс должен иметь только одну причину для изменения
2. Код должен быть открыт для расширения и закрыт для модификации
3. Подклассы должны быть заменяемы своими базовыми классами
4. Интерфейсы должны быть специфичными и не перегружены лишними методами
5. Модули высокого уровня не должны зависеть от модулей низкого уровня напрямую

Подсказка: Вспомните назначение каждого принципа и как он влияет на архитектуру приложения.

- **SRP** — Класс должен иметь только одну причину для изменения
- **OCP** — Код должен быть открыт для расширения и закрыт для модификации
- **LSP** — Подклассы должны быть заменяемы своими базовыми классами
- **ISP** — Интерфейсы должны быть специфичными и не перегружены лишними методами
- **DIP** — Модули высокого уровня не должны зависеть от модулей низкого уровня напрямую

🔗 Шаг 12:

Миссия 12

Как Clean Architecture помогает управлять зависимостями между слоями приложения?

Подсказка: Подумайте о направлении зависимостей от деталей к абстракциям.

Ответ:

- **Зависимости направляются внутрь, к бизнес-логике**
- Все слои напрямую зависят от UI
- Слой данных зависит от слоя UI
- UI напрямую зависит от бизнес-логики

🔗 Шаг 13:

Миссия 13

📄 Сопоставьте слои Clean Architecture с их ролью:

Ответ:

- Entities
 - Use Cases
 - Interface Adapters
 - Frameworks & Drivers
1. Бизнес-объекты и правила
 2. Бизнес-логика приложения и сценарии использования
 3. Перевод данных между слоями, UI и внешними интерфейсами
 4. Внешние фреймворки, базы данных, UI

Подсказка: Вспомните, что внутренние слои не должны зависеть от внешних.

- **Entities** — Бизнес-объекты и правила
- **Use Cases** — Бизнес-логика приложения и сценарии использования
- **Interface Adapters** — Перевод данных между слоями, UI и внешними интерфейсами
- **Frameworks & Drivers** — Внешние фреймворки, базы данных, UI

🔗 Шаг 14:

Миссия 14

Какой принцип SOLID нарушается, если подкласс изменяет поведение базового класса так, что клиентский код ломается?

Подсказка: Подумайте о замещении подклассов базовыми классами.

Ответ:

- **Liskov Substitution Principle (LSP)**
- Single Responsibility Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Шаг 15:

Миссия 15

Расположите этапы построения Clean Architecture проекта:

1. Определение бизнес-объектов и правил (Entities)
2. Определение сценариев использования (Use Cases)
3. Создание интерфейсов и адаптеров (Interface Adapters)
4. Подключение внешних фреймворков и драйверов (Frameworks & Drivers)

Подсказка: Начинаем с ядра, затем сценарии, адаптеры и внешние технологии.

Ответ:

- Определение бизнес-объектов и правил (Entities)
- Определение сценариев использования (Use Cases)
- Создание интерфейсов и адаптеров (Interface Adapters)
- Подключение внешних фреймворков и драйверов (Frameworks & Drivers)

Шаг 16:

Миссия 16

Почему важно разделять интерфейсы в соответствии с ISP?

Подсказка: Подумайте, как клиентские классы используют только те методы, которые им действительно нужны.

Ответ:

- **Чтобы клиенты не были вынуждены реализовывать лишние методы**
- Чтобы уменьшить количество классов
- Чтобы автоматически генерировать тесты
- Чтобы увеличить связность кода

Урок 4 - Event Sourcing и CQRS

Шаг 01:

Миссия 1

Что является основной идеей Event Sourcing?

Подсказка: Подумайте о хранении событий вместо текущего состояния.

Ответ:

- **Хранение всех изменений состояния как последовательности событий**
- Сохранение только текущего состояния объекта
- Хранение данных только в кэше
- Игнорирование истории изменений объекта

Шаг 02:

Миссия 2

Какой основной плюс использования CQRS?

Подсказка: Подумайте о разделении операций чтения и записи для масштабирования.

Ответ:

- **Разделение команд и запросов для упрощения масштабирования**
- Объединение всех операций в один поток
- Игнорирование событий для оптимизации производительности
- Хранение только агрегатов без истории изменений

Шаг 03:

Миссия 3

Что является преимуществом использования Event Sourcing?

Подсказка: Подумайте о том, как восстановить состояние объекта в любой момент времени.

Ответ:

- **Можно восстановить состояние системы на любую точку времени**
- Не требует хранения событий
- Сразу упрощает все запросы к базе
- Отменяет необходимость в транзакциях

Шаг 04:

Миссия 4

Какое ограничение важно учитывать при применении CQRS?

Подсказка: Подумайте о сложности синхронизации данных между чтением и записью.

Ответ:

- **Необходимость синхронизации проекций и агрегатов**
- Невозможность использования событий
- Обязательное хранение только текущего состояния
- Запрет на масштабирование чтения

Шаг 05:

Миссия 5

 Сопоставьте понятия Event Sourcing и CQRS с их описанием:

Ответ:

- Event Sourcing
 - CQRS
 - Агрегат
 - Проекция
1. Хранение всех изменений состояния как последовательности событий
 2. Разделение команд и запросов для упрощения архитектуры
 3. Объект, который инкапсулирует состояние и бизнес-правила
 4. Представление состояния системы для операций чтения

 **Подсказка:**

Подумайте о ролях каждого элемента: что хранит историю, что отвечает за чтение, а что за бизнес-логику.

- **Event Sourcing** — Хранение всех изменений состояния как последовательности событий
- **CQRS** — Разделение команд и запросов для упрощения архитектуры
- **Агрегат** — Объект, который инкапсулирует состояние и бизнес-правила
- **Проекция** — Представление состояния системы для операций чтения

Шаг 06:

Миссия 6

 Сопоставьте типы событий с их примером:

Ответ:

- Событие домена
 - Командное событие
 - Интеграционное событие
 - Событие системы
1. Пользователь зарегистрирован
 2. Запрос на изменение адреса
 3. Заказ отправлен в платежную систему
 4. Ошибка базы данных

Подсказка:

Обратите внимание, где событие относится к бизнес-логике, а где — к внешним системам или внутренним процессам.

- **Событие домена** — Пользователь зарегистрирован
- **Командное событие** — Запрос на изменение адреса
- **Интеграционное событие** — Заказ отправлен в платежную систему
- **Событие системы** — Ошибка базы данных

Шаг 07:

Миссия 7

 Сопоставьте преимущества Event Sourcing с их объяснением:

Ответ:

- Полная история изменений
 - Возможность отката
 - Повышенная прозрачность
 - Совместимость с CQRS
1. Можно восстановить состояние системы на любой момент времени
 2. Можно отменить ошибочные операции
 3. Легко отслеживать, что происходило с системой
 4. Проекция легко строится из событий

Подсказка:

Подумайте, какие качества системы напрямую зависят от хранения последовательности событий.

- **Полная история изменений** — Можно восстановить состояние системы на любой момент времени

- **Возможность отката** — Можно отменить ошибочные операции
- **Повышенная прозрачность** — Легко отслеживать, что происходило с системой
- **Совместимость с CQRS** — Проекция легко строится из событий

🔗 Шаг 08:

Миссия 8

Расположите шаги обработки команды в системе CQRS с Event Sourcing:

1. Команда отправлена
2. Агрегат обрабатывает команду и генерирует событие
3. Событие сохраняется в Event Store
4. Проекция обновляется для операций чтения

🔗 Подсказка:

Подумайте о порядке: сначала команда, потом обработка, затем сохранение и обновление представлений.

Ответ:

- Команда отправлена
- Агрегат обрабатывает команду и генерирует событие
- Событие сохраняется в Event Store
- Проекция обновляется для операций чтения

🔗 Шаг 09:

Миссия 9

Что является ключевой идеей Event Sourcing?

Подсказка: Подумайте, что хранится вместо текущего состояния объекта.

Ответ:

- **Сохраняются все события, изменяющие состояние объекта**
- Хранится только текущее состояние объекта
- Хранятся только ошибки приложения
- Хранятся только агрегаты без истории изменений

🔗 Шаг 10:

Миссия 10

Какая основная цель CQRS (Command Query Responsibility Segregation)?

Подсказка: Подумайте о разделении операций изменения данных и получения данных.

Ответ:

- **Разделять команды и запросы для оптимизации и безопасности**
- Объединять все операции в одном методе
- Хранить все данные в одной таблице
- Использовать только SQL для всех операций

🔗 Шаг 11:

Миссия 11

📄 Сопоставьте термины Event Sourcing и CQRS с их определениями:

Ответ:

- Event Sourcing
- CQRS
- Агрегат
- Событие
- Проекция

1. Хранение последовательности изменений состояния вместо текущего состояния
2. Разделение операций чтения и записи для повышения производительности
3. Корневой объект, контролирующий изменения связанных сущностей
4. Изменение состояния, записанное в системе
5. Слой для быстрого чтения данных на основе событий

Подсказка: Вспомните назначение каждого элемента в архитектуре Event Sourcing и CQRS.

- **Event Sourcing** — Хранение последовательности изменений состояния вместо текущего состояния
- **CQRS** — Разделение операций чтения и записи для повышения производительности
- **Агрегат** — Корневой объект, контролирующий изменения связанных сущностей
- **Событие** — Изменение состояния, записанное в системе
- **Проекция** — Слой для быстрого чтения данных на основе событий

Шаг 12:

Миссия 12

Почему важно использовать неизменяемые события в Event Sourcing?

Подсказка: Подумайте о том, что произойдет, если событие изменится после записи.

Ответ:

- **Чтобы гарантировать корректное восстановление состояния системы**
- Чтобы ускорить выполнение команд
- Чтобы уменьшить объем базы данных
- Чтобы события можно было удалять по желанию

Шаг 13:

Миссия 13

 Сопоставьте термины CQRS с их примерами:

Ответ:

- Command
- Query
- Write Model
- Read Model
- Проекция

1. Создать новый заказ в системе
2. Получить список всех заказов пользователя
3. Модель, используемая для внесения изменений в данные
4. Модель, используемая для быстрого чтения данных
5. Адаптированная структура данных для конкретного запроса

Подсказка: Подумайте, какие элементы отвечают за запись, а какие — за чтение.

- **Command** — Создать новый заказ в системе
- **Query** — Получить список всех заказов пользователя
- **Write Model** — Модель, используемая для внесения изменений в данные
- **Read Model** — Модель, используемая для быстрого чтения данных
- **Проекция** — Адаптированная структура данных для конкретного запроса

🔮 Шаг 14:

Миссия 14

Какая проблема решается с помощью проекций в CQRS?

Подсказка: Подумайте о производительности чтения данных для разных сценариев.

Ответ:

- **Быстрое чтение данных без нагрузки на основной агрегат**
- Сохранение всех команд в одном месте
- Автоматическое удаление событий
- Объединение всех моделей в одну таблицу

🔮 Шаг 15:

Миссия 15

Расположите этапы обработки команды с Event Sourcing и CQRS в правильном порядке:

1. Приход команды (Command) от пользователя
2. Проверка и выполнение бизнес-логики в агрегате
3. Генерация событий (Event) о совершенных изменениях
4. Сохранение событий в хранилище событий
5. Обновление проекций для быстрого чтения (Read Model)

Подсказка: Начинаем с команды, затем бизнес-логика, запись событий и обновление проекций.

Ответ:

- Приход команды (Command) от пользователя
- Проверка и выполнение бизнес-логики в агрегате
- Генерация событий (Event) о совершенных изменениях
- Сохранение событий в хранилище событий
- Обновление проекций для быстрого чтения (Read Model)

📖 Урок 5 - Модульное проектирование и Microservices

🔮 Шаг 01:

Миссия 1

Что является основной целью модульного проектирования?

Подсказка: Подумайте о том, как сделать код более управляемым и масштабируемым.

Ответ:

- **Разделение приложения на независимые модули с четкой ответственностью**
- Объединение всех функций в один большой класс
- Игнорирование зависимостей между компонентами
- Создание монолитного кода без модулей

Шаг 02:

Миссия 2

Какой принцип помогает уменьшить зависимость между модулями?

Подсказка: Подумайте о том, как модули общаются друг с другом.

Ответ:

- **Использование интерфейсов и инверсии зависимостей**
- Жесткое связывание модулей
- Прямой вызов функций из других модулей
- Сохранение всех данных в глобальных переменных

Шаг 03:

Миссия 3

Что является преимуществом микросервисной архитектуры?

Подсказка: Подумайте о независимом развертывании и масштабировании.

Ответ:

- **Независимое развертывание и масштабирование сервисов**
- Все сервисы обязательно должны развертываться вместе
- Сервисам нельзя иметь собственные базы данных
- Микросервисы уменьшают производительность на всех уровнях

Шаг 04:

Миссия 4

Что важно учитывать при построении микросервисов?

Подсказка: Подумайте о коммуникации и отказоустойчивости.

Ответ:

- **Межсервисная коммуникация и обработка отказов**
- Все сервисы должны использовать одну общую базу данных
- Игнорирование логирования и мониторинга
- Объединение всех сервисов в один процесс

Шаг 05:

Миссия 5

 Сопоставьте понятия модульного проектирования и микросервисов с их описанием:

Ответ:

- Модуль
 - Сервис
 - Интерфейс
 - Деплоймент
1. Самостоятельная единица функциональности с четкой ответственностью
 2. Независимая часть системы, которую можно развернуть отдельно
 3. Контракт взаимодействия между модулями или сервисами
 4. Процесс развертывания модуля или сервиса

 **Подсказка:**

Подумайте о ролях компонентов: что выполняет функциональность, что описывает взаимодействие, а что управляет развертыванием.

- **Модуль** — Самостоятельная единица функциональности с четкой ответственностью
- **Сервис** — Независимая часть системы, которую можно развернуть отдельно
- **Интерфейс** — Контракт взаимодействия между модулями или сервисами
- **Деплоймент** — Процесс развертывания модуля или сервиса

Шаг 06:

Миссия 6

 Сопоставьте типы зависимостей с примерами:

Ответ:

- Сильная зависимость
 - Слабая зависимость
 - Интермодульная зависимость
 - Внешняя зависимость
1. Модуль напрямую вызывает методы другого модуля
 2. Модуль взаимодействует через интерфейс
 3. Модули одного приложения зависят друг от друга
 4. Сервис зависит от внешнего API

Подсказка:

Подумайте, какие зависимости сильнее связывают компоненты, а какие позволяют гибкость.

- **Сильная зависимость** — Модуль напрямую вызывает методы другого модуля
- **Слабая зависимость** — Модуль взаимодействует через интерфейс
- **Интермодульная зависимость** — Модули одного приложения зависят друг от друга
- **Внешняя зависимость** — Сервис зависит от внешнего API

Шаг 07:

Миссия 7

 Сопоставьте преимущества микросервисов с их объяснением:

Ответ:

- Масштабируемость
 - Независимость развертывания
 - Устойчивость к сбоям
 - Гибкость технологий
1. Можно увеличивать ресурсы только для нужных сервисов
 2. Каждый сервис можно развернуть отдельно
 3. Отказ одного сервиса не рушит всю систему
 4. Разные сервисы могут использовать разные технологии

Подсказка:

Подумайте о практических преимуществах разделения системы на независимые сервисы.

- **Масштабируемость** — Можно увеличивать ресурсы только для нужных сервисов
- **Независимость развертывания** — Каждый сервис можно развернуть отдельно
- **Устойчивость к сбоям** — Отказ одного сервиса не рушит всю систему

- **Гибкость технологий** — Разные сервисы могут использовать разные технологии

Шаг 08:

Миссия 8

Расположите шаги создания микросервиса от идеи до деплоя:

1. Проектирование интерфейсов и API
2. Разработка сервиса
3. Тестирование функциональности
4. Развертывание на сервере или облаке

Подсказка:

Подумайте о логической последовательности: сначала проектируем, потом разрабатываем, тестируем и разворачиваем.

Ответ:

- Проектирование интерфейсов и API
- Разработка сервиса
- Тестирование функциональности
- Развертывание на сервере или облаке

Шаг 09:

Миссия 9

Какое основное преимущество модульного проектирования?

Подсказка: Подумайте, как разделение системы на модули влияет на командную работу и поддержку кода.

Ответ:

- **Упрощает поддержку и тестирование отдельных частей системы**
- Увеличивает зависимость между компонентами
- Позволяет игнорировать ошибки в коде
- Объединяет все функциональности в один монолит

Шаг 10:

Миссия 10

Что является ключевым принципом микросервисной архитектуры?

Подсказка: Подумайте, как отдельные сервисы взаимодействуют друг с другом.

Ответ:

- **Каждый сервис выполняет одну бизнес-функцию и взаимодействует через API**
- Все сервисы используют одну общую базу данных и логику
- Сервисы не должны быть независимыми
- Каждый сервис должен дублировать функциональность других

Шаг 11:

Миссия 11

 Сопоставьте термины модульного проектирования и микросервисов с их определениями:

Ответ:

- Модуль

- Микросервис
- Интерфейс
- Зависимость
- Контейнеризация

1. Отдельная часть системы с определенной функциональностью
2. Независимый сервис, выполняющий одну бизнес-задачу
3. Способ взаимодействия между модулями или сервисами
4. Связь одного модуля или сервиса от другого
5. Среда для изоляции и запуска микросервисов

Подсказка: Подумайте, как каждый термин помогает управлять сложной системой.

- **Модуль** — Отдельная часть системы с определенной функциональностью
- **Микросервис** — Независимый сервис, выполняющий одну бизнес-задачу
- **Интерфейс** — Способ взаимодействия между модулями или сервисами
- **Зависимость** — Связь одного модуля или сервиса от другого
- **Контейнеризация** — Среда для изоляции и запуска микросервисов

Шаг 12:

Миссия 12

Почему важно, чтобы микросервисы были слабо связанными?

Подсказка: Подумайте о возможности независимого развертывания и обновления.

Ответ:

- **Чтобы изменения в одном сервисе не нарушали работу других**
- Чтобы все сервисы имели одинаковый код
- Чтобы каждый сервис зависел от центрального модуля
- Чтобы уменьшить количество интерфейсов

Шаг 13:

Миссия 13

 Сопоставьте преимущества микросервисов с их объяснениями:

Ответ:

- Масштабируемость
 - Независимое развертывание
 - Устойчивость к сбоям
 - Гибкость технологий
 - Повышенная поддерживаемость
1. Можно увеличивать ресурсы только для нужных сервисов
 2. Обновление одного сервиса не влияет на другие
 3. Проблемы одного сервиса не выводят всю систему из строя
 4. Сервисы могут использовать разные технологии и языки
 5. Малые, изолированные сервисы проще поддерживать и тестировать

Подсказка: Подумайте, какое преимущество связано с масштабированием, а какое — с надежностью.

- **Масштабируемость** — Можно увеличивать ресурсы только для нужных сервисов
- **Независимое развертывание** — Обновление одного сервиса не влияет на другие

- **Устойчивость к сбоям** — Проблемы одного сервиса не выводят всю систему из строя
- **Гибкость технологий** — Сервисы могут использовать разные технологии и языки
- **Повышенная поддерживаемость** — Малые, изолированные сервисы проще поддерживать и тестировать

🔮 Шаг 14:

Миссия 14

Какая архитектурная ошибка чаще всего встречается при внедрении микросервисов?

Подсказка: Подумайте о зависимостях и коммуникации между сервисами.

Ответ:

- **Сильная связанность сервисов, из-за которой изменения нарушают всю систему**
- Использование контейнеров для каждого сервиса
- Автономность сервисов без взаимодействия
- Разделение на модули с четкой функциональностью

🔮 Шаг 15:

Миссия 15

Расположите шаги внедрения микросервисной архитектуры в правильном порядке:

1. Определение бизнес-доменов и границ сервисов
2. Разработка отдельных сервисов с независимым API
3. Тестирование сервисов и взаимодействий между ними
4. Настройка инфраструктуры (контейнеризация, оркестрация)
5. Постепенный переход от монолита к микросервисам

Подсказка: Сначала проектируем, потом создаем сервисы, тестируем, настраиваем инфраструктуру и постепенно мигрируем.

Ответ:

- Определение бизнес-доменов и границ сервисов
- Разработка отдельных сервисов с независимым API
- Тестирование сервисов и взаимодействий между ними
- Настройка инфраструктуры (контейнеризация, оркестрация)
- Постепенный переход от монолита к микросервисам

🔮 Шаг 16:

Миссия 16

Почему модульное проектирование облегчает командную работу?

Подсказка: Подумайте о распределении задач и изоляции изменений.

Ответ:

- **Каждый разработчик может работать над отдельным модулем без конфликтов**
- Все работают над одним большим файлом кода
- Модули не имеют четкой функциональности
- Невозможно тестировать модули по отдельности

Модуль 6 - Продвинутый C#

Урок 1 - Records и init-only

Шаг 01:

Миссия 1

Что отличает record от обычного класса в C#?

Подсказка: Подумайте о сравнении объектов и неизменяемости.

Ответ:

- **Records создают неизменяемые объекты по умолчанию и поддерживают value-based equality**
- Records не могут иметь конструкторы
- Records не поддерживают наследование
- Records автоматически сериализуются в JSON

Шаг 02:

Миссия 2

Какое назначение ключевого слова init в свойствах?

Подсказка: Подумайте, когда свойство можно изменять.

Ответ:

- **Свойство можно установить только при инициализации объекта**
- Свойство можно менять в любой точке кода
- Свойство нельзя ни читать, ни писать
- Свойство автоматически вычисляется на основе других полей

Шаг 03:

Миссия 3

Какая особенность у record при сравнении объектов?

Подсказка: Подумайте, что важнее для сравнения — ссылки или данные.

Ответ:

- **Сравниваются значения свойств (value-based equality)**
- Сравниваются только ссылки на объекты
- Объекты record всегда не равны
- Сравнение зависит от порядка создания объектов

Шаг 04:

Миссия 4

 Сопоставьте понятия с их объяснением:

Ответ:

- Record
- init-only
- Positional record
- with-expression

1. Неизменяемый объект с автоматической поддержкой value-based equality

2. Свойство, которое можно установить только при инициализации объекта
3. Record с компактным синтаксисом для конструктора и свойств
4. Создание нового объекта на основе существующего с изменением некоторых свойств

Подсказка:

Подумайте, какие конструкции помогают создавать неизменяемые объекты и работать с ними удобно.

- **Record** — Неизменяемый объект с автоматической поддержкой value-based equality
- **init-only** — Свойство, которое можно установить только при инициализации объекта
- **Positional record** — Record с компактным синтаксисом для конструктора и свойств
- **with-expression** — Создание нового объекта на основе существующего с изменением некоторых свойств

Шаг 05:

Миссия 5

 Сопоставьте типы record с их особенностями:

Ответ:

- Reference record
 - Value record
 - Immutable record
 - Mutable record
1. Сравнение объектов по ссылке
 2. Сравнение объектов по значениям свойств
 3. Объекты нельзя менять после создания
 4. Объекты можно изменять после создания

Подсказка:

Обратите внимание на поведение record при изменении и сравнении объектов.

- **Reference record** — Сравнение объектов по ссылке
- **Value record** — Сравнение объектов по значениям свойств
- **Immutable record** — Объекты нельзя менять после создания
- **Mutable record** — Объекты можно изменять после создания

Шаг 06:

Миссия 6

Расположите шаги создания нового объекта record с изменением свойства:

1. Создать исходный объект record
2. Использовать выражение with, чтобы создать новый объект с измененным свойством
3. Использовать новый объект в коде

Подсказка:

Сначала нужен исходный объект, затем создается новый через with, и только после этого он используется.

Ответ:

- Создать исходный объект record
- Использовать выражение with, чтобы создать новый объект с измененным свойством
- Использовать новый объект в коде

Шаг 07:

Миссия 7

Можно ли изменить значение свойства init-only после создания объекта?

Подсказка: Подумайте, когда свойства init-only разрешено менять.

Ответ:

- **Нет, можно установить только при инициализации объекта**
- Да, можно менять в любом месте кода
- Да, но только в методах класса
- Да, если используется with-expression

Шаг 08:

Миссия 8

Что произойдет при сравнении двух record с одинаковыми свойствами?

Подсказка: Подумайте о value-based equality.

Ответ:

- **Они будут равны, даже если это разные экземпляры**
- Они всегда не равны, так как это разные объекты
- Равенство зависит от порядка создания объектов
- Сравнение вызывает ошибку компиляции

Шаг 09:

Миссия 9

Что является ключевым преимуществом использования record в C#?

Подсказка: Подумайте о том, как record облегчает работу с неизменяемыми данными.

Ответ:

- **Простое создание неизменяемых объектов с поддержкой сравнения по значению**
- Позволяет изменять любые свойства после создания
- Не поддерживает наследование
- Автоматически сохраняет данные в базу

Шаг 10:

Миссия 10

Для чего используется ключевое слово init в C#?

Подсказка: Подумайте о времени, когда можно изменять значение свойства.

Ответ:

- **Позволяет присваивать значение только при инициализации объекта**
- Разрешает менять значение свойства в любой момент
- Запрещает присваивание значения даже при создании объекта
- Используется только для методов класса

Шаг 11:

Миссия 11

 Сопоставьте термины C# с их правильными объяснениями:

Ответ:

- record
- init-only
- immutable
- with-выражение
- deconstructor

1. Класс, оптимизированный для хранения данных с неизменяемыми свойствами
2. Позволяет присваивать значение свойству только при инициализации объекта
3. Объект, состояние которого нельзя изменить после создания
4. Создание нового объекта с частично измененными свойствами существующего
5. Метод для распаковки значений объекта в отдельные переменные

Подсказка: Подумайте, какие ключевые особенности record и init облегчают работу с данными.

- **record** — Класс, оптимизированный для хранения данных с неизменяемыми свойствами
- **init-only** — Позволяет присваивать значение свойству только при инициализации объекта
- **immutable** — Объект, состояние которого нельзя изменить после создания
- **with-выражение** — Создание нового объекта с частично измененными свойствами существующего
- **deconstructor** — Метод для распаковки значений объекта в отдельные переменные

 Шаг 12:

Миссия 12

Что произойдет, если попытаться изменить свойство record после инициализации, если оно объявлено с init?

Подсказка: Подумайте о правилах неизменяемости init-свойств.

Ответ:

- **Компилятор выдаст ошибку, изменение запрещено**
- Свойство изменится без проблем
- Произойдет предупреждение во время выполнения
- Свойство удалится из объекта

 Шаг 13:

Миссия 13

 Сопоставьте преимущества record и init-only с их описанием:

Ответ:

- Сравнение по значению
 - Легкость создания копий с изменениями
 - Безопасность неизменяемости
 - Ясный и краткий синтаксис
 - Поддержка деструктуризации
1. Объекты считаются равными, если значения всех свойств совпадают
 2. Можно создавать новые объекты на основе существующих с изменением некоторых свойств
 3. Свойства не могут быть изменены после инициализации
 4. Уменьшает количество шаблонного кода
 5. Позволяет распаковывать объект в отдельные переменные

Подсказка: Подумайте, как record облегчает работу с данными, а init обеспечивает неизменяемость.

- **Сравнение по значению** — Объекты считаются равными, если значения всех свойств совпадают
- **Легкость создания копий с изменениями** — Можно создавать новые объекты на основе существующих с изменением некоторых свойств
- **Безопасность неизменяемости** — Свойства не могут быть изменены после инициализации
- **Ясный и краткий синтаксис** — Уменьшает количество шаблонного кода
- **Поддержка деструктуризации** — Позволяет распаковывать объект в отдельные переменные

Шаг 14:

Миссия 14

Какой синтаксис позволяет создать копию record с изменением некоторых свойств?

Подсказка: Используйте ключевое слово, позволяющее частично обновлять объект.

Ответ:

- `var newObj = oldObj with { Property = value }`
- `var newObj = oldObj.assign { Property = value }`
- `var newObj = oldObj.update(Property, value)`
- `var newObj = oldObj.modify { Property = value }`

Шаг 15:

Миссия 15

Расположите шаги создания и использования record с init-only свойствами в правильном порядке:

1. Определение record с init-only свойствами
2. Создание объекта с инициализацией свойств
3. Использование объекта в коде без изменения init-свойств
4. Создание копии с помощью with-выражения при необходимости изменений
5. Использование деструктуризации для распаковки значений

Подсказка: Сначала определяем, потом создаем объект, используем, при необходимости создаем копии и распаковываем.

Ответ:

- Определение record с init-only свойствами
- Создание объекта с инициализацией свойств
- Использование объекта в коде без изменения init-свойств
- Создание копии с помощью with-выражения при необходимости изменений
- Использование деструктуризации для распаковки значений

Урок 2 - Reflection и Expression Trees

Шаг 01:

Миссия 1

Что позволяет сделать Reflection в C#?

Подсказка: Подумайте, как можно исследовать объекты во время выполнения.

Ответ:

- **Изучать типы, свойства, методы и атрибуты объектов во время выполнения**
- Изменять исходный код программы на лету
- Компилировать программу без Visual Studio
- Автоматически оптимизировать память для объектов

Шаг 02:

Миссия 2

Что позволяет делать Expression Tree в C#?

Подсказка: Подумайте, как можно работать с кодом как с данными.

Ответ:

- **Представлять код в виде объектов и строить дерево выражений для анализа и компиляции**
- Автоматически выполнять рефакторинг кода
- Определять типы объектов только на этапе компиляции
- Создавать интерфейсы без реализации методов

Шаг 03:

Миссия 3

Какой метод класса Type используется для получения информации о свойствах объекта через Reflection?

Подсказка: Он возвращает массив PropertyInfo.

Ответ:

- **GetProperties()**
- GetFields()
- GetMethods()
- GetAttributes()

Шаг 04:

Миссия 4

 Сопоставьте понятия с их описанием:

Ответ:

- Reflection
 - Expression Tree
 - PropertyInfo
 - MethodInfo
1. Механизм для получения информации о типах и членах объектов во время выполнения
 2. Структура, представляющая код программы в виде дерева выражений
 3. Представление свойства объекта, его типа и методов доступа
 4. Представление метода объекта с возможностью вызова

Подсказка:

Подумайте, какие объекты Reflection помогают исследовать и вызывать.

- **Reflection** — Механизм для получения информации о типах и членах объектов во время выполнения

- **Expression Tree** — Структура, представляющая код программы в виде дерева выражений
- **PropertyInfo** — Представление свойства объекта, его типа и методов доступа
- **MethodInfo** — Представление метода объекта с возможностью вызова

🔗 Шаг 05:

Миссия 5

📄 Сопоставьте методы Expression с их назначением:

Ответ:

- Expression.Lambda
 - Expression.Call
 - Expression.Constant
 - Expression.Parameter
1. Создает лямбда-выражение на основе дерева выражений
 2. Создает вызов метода в дереве выражений
 3. Представляет константу в дереве выражений
 4. Определяет параметр для лямбда-выражения

💡 **Подсказка:**

Обратите внимание, какие элементы необходимы для построения дерева выражений.

- **Expression.Lambda** — Создает лямбда-выражение на основе дерева выражений
- **Expression.Call** — Создает вызов метода в дереве выражений
- **Expression.Constant** — Представляет константу в дереве выражений
- **Expression.Parameter** — Определяет параметр для лямбда-выражения

🔗 Шаг 06:

Миссия 6

Можно ли через Reflection вызвать приватный метод объекта?

Подсказка: Подумайте о правах доступа и BindingFlags.

Ответ:

- **Да, если использовать BindingFlags.NonPublic**
- Нет, приватные методы недоступны всегда
- Да, но только в том же классе
- Да, через Expression Tree без Reflection

🔗 Шаг 07:

Миссия 7

Что нужно для компиляции Expression Tree в делегат?

Подсказка: Подумайте о связке между деревом выражений и исполняемым кодом.

Ответ:

- **Вызвать метод Compile() у LambdaExpression**
- Использовать Reflection для вызова метода Compile()
- Сохранить Expression Tree в XML
- Передать дерево выражений в конструктор делегата напрямую

🔧 Шаг 08:

Миссия 8

📖 Сопоставьте виды BindingFlags с их значением:

Ответ:

- Public
 - NonPublic
 - Static
 - Instance
1. Доступ к публичным членам класса
 2. Доступ к приватным и защищенным членам класса
 3. Доступ к статическим членам класса
 4. Доступ к нестатическим (экземплярным) членам класса

💡 Подсказка:

BindingFlags помогают Reflection понимать, какие члены класса искать.

- **Public** — Доступ к публичным членам класса
- **NonPublic** — Доступ к приватным и защищенным членам класса
- **Static** — Доступ к статическим членам класса
- **Instance** — Доступ к нестатическим (экземплярным) членам класса

🔧 Шаг 09:

Миссия 9

Расположите шаги использования Expression Tree для вызова метода:

1. Создать ParameterExpression для аргументов метода
2. Построить Call Expression для метода
3. Создать Lambda Expression, объединяющий Call Expression и параметры
4. Скомпилировать Lambda Expression и вызвать делегат

💡 Подсказка:

Сначала определяем параметры, потом создаем вызов метода, затем лямбду, и только в конце компилируем и вызываем.

Ответ:

- Создать ParameterExpression для аргументов метода
- Построить Call Expression для метода
- Создать Lambda Expression, объединяющий Call Expression и параметры
- Скомпилировать Lambda Expression и вызвать делегат

🔧 Шаг 10:

Миссия 10

Можно ли изменить значение приватного поля объекта через Reflection?

Подсказка: Подумайте о доступе к полям и свойствам.

Ответ:

- **Да, используя FieldInfo.SetValue и BindingFlags.NonPublic**
- Нет, приватные поля нельзя менять через Reflection
- Да, но только через Expression Tree

- Да, через обычное присваивание

Шаг 11:

Миссия 11

Что является преимуществом Expression Trees при построении LINQ провайдеров?

Подсказка: Подумайте о динамическом построении запросов.

Ответ:

- **Позволяют строить запросы на лету и преобразовывать их в SQL или другие форматы**
- Автоматически кешируют результаты всех методов
- Гарантируют отсутствие исключений во время выполнения
- Заменяют Reflection для получения информации о типах

Шаг 12:

Миссия 12

Что позволяет делать Reflection в C#?

Подсказка: Подумайте о возможности изучать и изменять типы во время выполнения.

Ответ:

- **Получать информацию о типах, методах и свойствах во время выполнения**
- Компилировать код без использования сборки
- Обеспечивать только статическую проверку типов
- Автоматически оптимизировать память объекта

Шаг 13:

Миссия 13

Какая основная цель Expression Trees?

Подсказка: Подумайте о представлении кода как данных.

Ответ:

- **Представление кода в виде структуры данных для анализа и модификации**
- Компиляция кода в машинные инструкции на лету
- Создание графического интерфейса для методов
- Автоматическое тестирование функций

Шаг 14:

Миссия 14

 Сопоставьте понятия Reflection и Expression Trees с их описаниями:

Ответ:

- Reflection
- Expression Trees
- PropertyInfo
- MethodInfo
- LambdaExpression

1. Позволяет изучать свойства и методы объектов во время выполнения
2. Позволяет представлять код как структуру данных

3. Представляет информацию о свойстве типа
4. Представляет информацию о методе типа
5. Представляет выражение lambda как объект для анализа

Подсказка: Обратите внимание, что Reflection работает с типами, а Expression Trees — с кодом.

- **Reflection** — Позволяет изучать свойства и методы объектов во время выполнения
- **Expression Trees** — Позволяет представлять код как структуру данных
- **PropertyInfo** — Представляет информацию о свойстве типа
- **MethodInfo** — Представляет информацию о методе типа
- **LambdaExpression** — Представляет выражение lambda как объект для анализа

Шаг 15:

Миссия 15

Что позволяет делать MethodInfo в Reflection?

Подсказка: Подумайте о способах взаимодействия с методами во время выполнения.

Ответ:

- **Вызывать метод объекта и получать информацию о его параметрах**
- Создавать новые методы на лету
- Менять тело метода в памяти без перекомпиляции
- Автоматически тестировать метод

Шаг 16:

Миссия 16

 Сопоставьте типы Expression Trees с их назначением:

Ответ:

- BinaryExpression
- ParameterExpression
- ConstantExpression
- MemberExpression
- MethodCallExpression

1. Представляет бинарные операции, например + или *
2. Представляет параметры метода или lambda
3. Представляет константное значение
4. Представляет доступ к свойству или полю объекта
5. Представляет вызов метода

Подсказка: Обратите внимание, какие элементы выражения соответствуют операциям, параметрам и вызовам.

- **BinaryExpression** — Представляет бинарные операции, например + или *
- **ParameterExpression** — Представляет параметры метода или lambda
- **ConstantExpression** — Представляет константное значение
- **MemberExpression** — Представляет доступ к свойству или полю объекта
- **MethodCallExpression** — Представляет вызов метода

Шаг 17:

Миссия 17

Какой способ безопасного вызова метода через Reflection без потери производительности?

Подсказка: Подумайте о кешировании MethodInfo и делегатах.

Ответ:

- **Создать делегат через MethodInfo и использовать его повторно**
- Каждый раз вызывать MethodInfo.Invoke
- Переписывать метод вручную для каждого вызова
- Использовать Reflection.Emit без делегатов

Шаг 18:

Миссия 18

Расположите шаги использования Expression Tree для вызова метода с анализом параметров:

1. Создание параметров через ParameterExpression
2. Создание тела метода через MethodCallExpression
3. Создание lambda-выражения через Expression.Lambda
4. Компиляция lambda в делегат
5. Вызов метода через делегат

Подсказка: Сначала определяем параметры, затем тело, создаем lambda, компилируем и вызываем.

Ответ:

- Создание параметров через ParameterExpression
- Создание тела метода через MethodCallExpression
- Создание lambda-выражения через Expression.Lambda
- Компиляция lambda в делегат
- Вызов метода через делегат

Шаг 19:

Миссия 19

Почему Reflection и Expression Trees полезны при разработке ORM или динамических фреймворков?

Подсказка: Подумайте о том, как можно работать с кодом и объектами без жесткой привязки к типам во время компиляции.

Ответ:

- **Позволяют динамически анализировать и вызывать методы и свойства без жесткой привязки к типам**
- Автоматически генерируют графический интерфейс
- Позволяют хранить объекты только в памяти
- Заменяют статический типизированный код полностью

Урок 3 - Source Generators и Nullable Reference Types

Шаг 01:

Миссия 1

Что позволяет делать Source Generators в C#?

Подсказка: Подумайте, как можно автоматически создавать код во время компиляции.

Ответ:

- **Автоматически генерировать код на этапе компиляции**
- Выполнять код динамически во время выполнения
- Изменять IL код без компиляции
- Автоматически оптимизировать память

 **Шаг 02:**

Миссия 2

Что означает Nullable Reference Types в C#?

Подсказка: Подумайте о возможности указания, что ссылка может быть null.

Ответ:

- **Позволяет явно указывать, какие ссылки могут быть null**
- Запрещает использовать null для всех типов ссылок
- Автоматически инициализирует все ссылки значениями по умолчанию
- Позволяет null только для значимых типов

 **Шаг 03:**

Миссия 3

Как можно применить Source Generator для DTO-классов?

Подсказка: Подумайте о создании стандартного кода для сериализации.

Ответ:

- **Автоматически создавать свойства и методы для сериализации и копирования**
- Динамически менять значения свойств во время выполнения
- Создавать базу данных из DTO
- Выполнять проверку ссылок null во время исполнения

 **Шаг 04:**

Миссия 4

 Сопоставьте понятия с их объяснением:

Ответ:

- Source Generator
 - Nullable Reference Type
 - Non-Nullable Reference
 - Partial Class
1. Механизм генерации кода на этапе компиляции
 2. Ссылка, которая может быть null и требует проверки
 3. Ссылка, которая не может быть null
 4. Позволяет разделять определение класса на несколько файлов

 **Подсказка:**

Обратите внимание на различие между ссылками, которые могут содержать null и которые не могут.

- **Source Generator** — Механизм генерации кода на этапе компиляции
- **Nullable Reference Type** — Ссылка, которая может быть null и требует проверки
- **Non-Nullable Reference** — Ссылка, которая не может быть null

- **Partial Class** — Позволяет разделять определение класса на несколько файлов

Шаг 05:

Миссия 5

 Сопоставьте атрибуты C# с их применением:

Ответ:

- [Nullable]
 - [MaybeNull]
 - [NotNull]
 - [GeneratedCode]
1. Указывает, что ссылка может быть null
 2. Указывает, что метод может вернуть null
 3. Указывает, что параметр или поле не может быть null
 4. Отмечает код, сгенерированный Source Generator

Подсказка:

Подумайте, как C# помогает проверять ссылки на null статически.

- **[Nullable]** — Указывает, что ссылка может быть null
- **[MaybeNull]** — Указывает, что метод может вернуть null
- **[NotNull]** — Указывает, что параметр или поле не может быть null
- **[GeneratedCode]** — Отмечает код, сгенерированный Source Generator

Шаг 06:

Миссия 6

Можно ли использовать Source Generator для валидации null-ссылок?

Подсказка: Подумайте, что можно сгенерировать во время компиляции.

Ответ:

- **Да, можно генерировать проверки null при создании классов или методов**
- Нет, Source Generator не имеет доступа к null-значениям во время выполнения
- Да, но только для интерфейсов
- Нет, это можно делать только вручную

Шаг 07:

Миссия 7

Что происходит при отключении Nullable Reference Types?

Подсказка: Подумайте о компиляции и проверках на null.

Ответ:

- **Компилятор перестает предупреждать о возможных null-ссылках**
- Все ссылки становятся Nullable автоматически
- Source Generator перестает работать
- Программа не компилируется

Шаг 08:

Миссия 8

Что позволяет делать Source Generators в C#?

Подсказка: Подумайте о генерации кода во время компиляции.

Ответ:

- **Автоматически генерировать код во время компиляции**
- Выполнять код во время выполнения программы
- Проверять null-ссылки только в рантайме
- Рефакторить код без участия компилятора

Шаг 09:

Миссия 9

Что делает Nullable Reference Types в C#?

Подсказка: Подумайте о безопасности ссылок и предупреждениях компилятора.

Ответ:

- **Помогает определить, может ли ссылка быть null, и выдаёт предупреждения компилятора**
- Автоматически инициализирует все ссылки значением по умолчанию
- Удаляет необходимость использовать try/catch
- Компилирует код быстрее за счёт оптимизации null

Шаг 10:

Миссия 10

 Сопоставьте понятия с их определениями:

Ответ:

- Source Generator
 - Nullable Reference Type
 - Partial Class
 - #nullable enable
 - CodeContext
1. Позволяет генерировать код во время компиляции
 2. Определяет, может ли ссылка быть null и даёт предупреждения компилятора
 3. Позволяет разбивать класс на несколько файлов для генерации кода
 4. Включает анализ nullable ссылок в проекте
 5. Содержит контекст, используемый генератором для создания кода

Подсказка: Обратите внимание на разницу между анализом null и генерацией кода.

- **Source Generator** — Позволяет генерировать код во время компиляции
- **Nullable Reference Type** — Определяет, может ли ссылка быть null и даёт предупреждения компилятора
- **Partial Class** — Позволяет разбивать класс на несколько файлов для генерации кода
- **#nullable enable** — Включает анализ nullable ссылок в проекте
- **CodeContext** — Содержит контекст, используемый генератором для создания кода

Шаг 11:

Миссия 11

Почему Source Generators полезны для больших проектов?

Подсказка: Подумайте о повторяемом коде и снижении ручного труда.

Ответ:

- **Позволяют автоматически генерировать повторяющийся код, снижая вероятность ошибок и ускоряя разработку**
- Ускоряют выполнение программы в рантайме
- Заменяют необходимость в юнит-тестах
- Автоматически исправляют ошибки null-ссылок во время выполнения

Шаг 12:

Миссия 12

 Сопоставьте элементы Nullable Reference Types с их назначением:

Ответ:

- string?
 - string
 - #nullable disable
 - ! (null-forgiving)
 - ? after value type
1. Разрешает null для строки
 2. Запрещает null для строки, выдаёт предупреждение при попытке присвоить null
 3. Отключает анализ nullable ссылок
 4. Убирает предупреждение компилятора о потенциальном null
 5. Позволяет использовать null для значимых типов

Подсказка: Обратите внимание на синтаксис и правила компилятора.

- **string?** — Разрешает null для строки
- **string** — Запрещает null для строки, выдаёт предупреждение при попытке присвоить null
- **#nullable disable** — Отключает анализ nullable ссылок
- **! (null-forgiving)** — Убирает предупреждение компилятора о потенциальном null
- **? after value type** — Позволяет использовать null для значимых типов

Шаг 13:

Миссия 13

Как правильно использовать Source Generators для валидации DTO объектов?

Подсказка: Подумайте о генерации кода, который проверяет свойства автоматически.

Ответ:

- **Генерировать классы с методами проверки всех свойств DTO на null и корректность данных**
- Менять исходный код DTO вручную
- Вставлять проверки через Reflection во время выполнения
- Игнорировать null, полагаясь на runtime ошибки

Шаг 14:

Миссия 14

Расположите шаги работы Source Generator для генерации кода DTO с nullable проверками:

1. Определение шаблона кода для генерации
2. Создание класса генератора, реализующего `ISourceGenerator`

3. Анализ исходных файлов и поиск DTO классов
4. Генерация кода с проверками на null для свойств
5. Добавление сгенерированного кода в проект

Подсказка: Сначала создаём шаблон и генератор, потом анализируем код, генерируем и добавляем его.

Ответ:

- Определение шаблона кода для генерации
- Создание класса генератора, реализующего `ISourceGenerator`
- Анализ исходных файлов и поиск DTO классов
- Генерация кода с проверками на null для свойств
- Добавление сгенерированного кода в проект

Шаг 15:

Миссия 15

Почему включение `Nullable Reference Types` полезно вместе с `Source Generators`?

Подсказка: Подумайте о комбинации компиляции и анализа ссылок для повышения безопасности.

Ответ:

- **Source Generators могут создавать код с проверками null, а компилятор предупреждает о потенциальных ошибках ссылок**
- `Source Generators` автоматически отключают `Nullable Reference Types`
- `Nullable Reference Types` заменяют `Source Generators` для проверки DTO
- Обеспечивают ускорение выполнения программы на рантайме

Урок 4 - Pattern Matching в C# 10-1

Шаг 01:

Миссия 1

Что такое `pattern matching` в C#?

Подсказка: Подумайте о способе проверки структуры данных вместо простого сравнения.

Ответ:

- **Механизм проверки типа и структуры объектов с возможностью извлечения данных**
- Сравнение объектов по ссылке
- Проверка только на null
- Автоматическая сериализация объектов

Шаг 02:

Миссия 2

Какие виды `pattern matching` доступны в C# 10-11?

Подсказка: Подумайте о различных способах проверки данных: по типу, значению, структуре.

Ответ:

- **Type patterns**
- **Relational patterns**
- **Logical patterns**

- Arithmetic patterns

🔗 Шаг 03:

Миссия 3

Что делает конструкция ``x is int i``?

Подсказка: Подумайте, как можно одновременно проверить тип и получить значение.

Ответ:

- **Проверяет, что `x` — `int`**, и присваивает его значение переменной `i`
- Просто проверяет тип `x`, не создавая переменной
- Создает переменную `i` без проверки типа
- Сравнивает `x` с `i`

🔗 Шаг 04:

Миссия 4

📄 Сопоставьте типы `pattern matching` с их назначением:

Ответ:

- Type pattern
 - Relational pattern
 - Logical pattern
 - Property pattern
1. Проверяет соответствие типа и позволяет извлечь значение
 2. Сравнивает значение с использованием `<`, `>`, `<=`, `>=`
 3. Объединяет несколько условий с помощью `&&`, `||`, `!`
 4. Сравнивает свойства объекта с конкретными значениями

🔗 **Подсказка:**

Подумайте, какие паттерны проверяют тип, значения, комбинации и свойства объекта.

- **Type pattern** — Проверяет соответствие типа и позволяет извлечь значение
- **Relational pattern** — Сравнивает значение с использованием `<`, `>`, `<=`, `>=`
- **Logical pattern** — Объединяет несколько условий с помощью `&&`, `||`, `!`
- **Property pattern** — Сравнивает свойства объекта с конкретными значениями

🔗 Шаг 05:

Миссия 5

Что делает `pattern matching` с tuple ``var (x, y) = point;``?

Подсказка: Подумайте, как можно деструктурировать объект.

Ответ:

- **Позволяет извлечь элементы tuple в отдельные переменные**
- Создает новый tuple без проверки типа
- Проверяет только тип `point`
- Присваивает `point` другой переменной

🔗 Шаг 06:

Миссия 6

Когда используется `pattern matching` с `switch expression`?

Подсказка: Подумайте о замене длинных if-else цепочек.

Ответ:

- **Для краткой и наглядной обработки разных случаев объекта**
- Для генерации кода во время компиляции
- Для проверки null-ссылок
- Для сравнения только примитивных типов

🔮 **Шаг 07:**

Миссия 7

📄 Сопоставьте сценарий с подходящим pattern:

Ответ:

- Проверка типа объекта и извлечение значения
- Сравнение чисел на больше/меньше
- Проверка нескольких условий одновременно
- Сравнение свойств объекта с конкретными значениями

1. Type pattern
2. Relational pattern
3. Logical pattern
4. Property pattern

🔮 **Подсказка:**

Подумайте, какой паттерн соответствует проверке типа, значений, комбинации условий или свойств объекта.

- **Проверка типа объекта и извлечение значения** — Type pattern
- **Сравнение чисел на больше/меньше** — Relational pattern
- **Проверка нескольких условий одновременно** — Logical pattern
- **Сравнение свойств объекта с конкретными значениями** — Property pattern

🔮 **Шаг 08:**

Миссия 8

Что делает pattern ``x is not null``?

Подсказка: Подумайте о проверке ссылки перед использованием.

Ответ:

- **Проверяет, что объект x не равен null**
- Создает новый объект x
- Присваивает x значение по умолчанию
- Проверяет только тип x

🔮 **Шаг 09:**

Миссия 9

Какие новые возможности появились в pattern matching в C# 11?

Подсказка: Подумайте о расширении логики и проверки значений.

Ответ:

- **List patterns (сравнение элементов коллекции)**
- **Required patterns (обязательные проверки свойств)**

- Tuple patterns были удалены
- **Enhanced logical patterns** с поддержкой **or, and, not**

Шаг 10:

Миссия 10

Расположите шаги применения pattern matching для объекта:

1. Определить тип объекта с помощью Type pattern
2. Проверить свойства объекта с помощью Property pattern
3. Объединить условия через Logical pattern
4. Обработать результат или выполнить действие

Подсказка:

Сначала проверяем тип, потом свойства, затем объединяем условия и наконец выполняем действия.

Ответ:

- Определить тип объекта с помощью Type pattern
- Проверить свойства объекта с помощью Property pattern
- Объединить условия через Logical pattern
- Обработать результат или выполнить действие

Шаг 11:

Миссия 11

Что нового привнес pattern matching в C# 10?

Подсказка: Подумайте о более лаконичном способе проверки типа и значений одновременно.

Ответ:

- **Позволяет проверять тип и условия в одном выражении**
- Автоматически конвертирует все типы в object
- Заменяет if/else на switch без условий
- Убирает необходимость в компиляции типов

Шаг 12:

Миссия 12

Какие конструкции pattern matching позволяют использовать C# 11?

Подсказка: Подумайте о новых видах шаблонов для проверки свойств объектов.

Ответ:

- **List patterns** для проверки элементов коллекций
- **Raw string patterns** для строковых литералов
- Dynamic typing patterns для всех объектов
- Auto null check patterns без компилятора

Шаг 13:

Миссия 13

 Сопоставьте виды pattern matching с их описанием:

Ответ:

- Type Pattern

- Property Pattern
- Positional Pattern
- Relational Pattern
- Logical Pattern

1. Проверяет тип объекта и при совпадении создаёт переменную указанного типа
2. Проверяет значения свойств объекта
3. Сопоставляет объект с кортежем или record по позициям
4. Сравнивает значения с помощью `<`, `>`, `<=`, `>=`
5. Комбинирует несколько шаблонов с помощью `and`, `or`, `not`

Подсказка: Смотрите на синтаксис и на то, как можно проверить составные условия.

- **Type Pattern** — Проверяет тип объекта и при совпадении создаёт переменную указанного типа
- **Property Pattern** — Проверяет значения свойств объекта
- **Positional Pattern** — Сопоставляет объект с кортежем или record по позициям
- **Relational Pattern** — Сравнивает значения с помощью `<`, `>`, `<=`, `>=`
- **Logical Pattern** — Комбинирует несколько шаблонов с помощью `and`, `or`, `not`

Шаг 14:

Миссия 14

Почему pattern matching помогает улучшить читаемость кода?

Подсказка: Подумайте о сокращении вложенных if и увеличении наглядности условий.

Ответ:

- **Снижает количество вложенных if и делает код более декларативным**
- Полностью исключает необходимость использовать switch
- Автоматически исправляет ошибки типов во время выполнения
- Ускоряет выполнение кода за счёт компилятора

Шаг 15:

Миссия 15

 Сопоставьте примеры pattern matching с их типом:

Ответ:

- `x is int i`
- `point is { X: 0, Y: 0 }`
- `recordPoint is (var x, var y)`
- `age is >= 18 and <= 65`
- `value is not null`

1. Type Pattern
2. Property Pattern
3. Positional Pattern
4. Relational Pattern combined
5. Logical Pattern

Подсказка: Посмотрите на синтаксис и ключевые слова `is`, `not`, `var`.

- **`x is int i`** — Type Pattern
- **`point is { X: 0, Y: 0 }`** — Property Pattern

- **recordPoint is (var x, var y)** — Positional Pattern
- **age is >= 18 and <= 65** — Relational Pattern combined
- **value is not null** — Logical Pattern

🔗 Шаг 16:

Миссия 16

Как pattern matching упрощает обработку различных типов в коллекции объектов?

Подсказка: Подумайте о switch expressions и проверке типов одновременно.

Ответ:

- **Позволяет легко распределять объекты по типам и условиям без длинных if/else**
- Автоматически удаляет ненужные элементы коллекции
- Меняет тип объектов в коллекции на object
- Компилирует коллекцию в отдельный assembly

🔗 Шаг 17:

Миссия 17

Расположите шаги выполнения pattern matching при анализе объекта record:

1. Определение шаблона (pattern) для проверки объекта
2. Сопоставление объекта с шаблоном
3. Выполнение действий при совпадении шаблона
4. Обработка случая, когда шаблон не совпал (default)

Подсказка: Сначала определяем шаблон, затем проверяем, потом выполняем код и обрабатываем default.

Ответ:

- Определение шаблона (pattern) для проверки объекта
- Сопоставление объекта с шаблоном
- Выполнение действий при совпадении шаблона
- Обработка случая, когда шаблон не совпал (default)

🔗 Шаг 18:

Миссия 18

Какая комбинация pattern matching является наиболее универсальной для сложных условий?

Подсказка: Подумайте о логических операторах и проверках свойств одновременно.

Ответ:

- **Сочетание property, relational и logical pattern**
- Только type pattern
- Только positional pattern
- Использование switch без шаблонов

📖 Урок 5 - Функциональные возможности C#

🔗 Шаг 01:

Миссия 1

Что такое LINQ в C#?

Подсказка: Подумайте о способе удобной работы с коллекциями.

Ответ:

- **Язык запросов для работы с коллекциями**
- Фреймворк для UI
- Механизм сериализации объектов
- Система управления потоками

Шаг 02:

Миссия 2

Что делает метод `Select` в LINQ?

Подсказка: Подумайте о преобразовании элементов коллекции.

Ответ:

- **Преобразует каждый элемент коллекции в новый вид**
- Фильтрует элементы по условию
- Сортирует элементы коллекции
- Объединяет несколько коллекций

Шаг 03:

Миссия 3

Что делает метод `Where` в LINQ?

Подсказка: Подумайте о выборе элементов коллекции по условию.

Ответ:

- **Фильтрует элементы коллекции по условию**
- Преобразует элементы в другой вид
- Сортирует коллекцию
- Объединяет коллекции

Шаг 04:

Миссия 4

 Сопоставьте функциональные методы LINQ с их назначением:

Ответ:

- Select
- Where
- OrderBy
- GroupBy

1. Преобразует каждый элемент коллекции в другой вид
2. Фильтрует элементы по условию
3. Сортирует элементы по ключу
4. Группирует элементы по ключу

Подсказка:

Подумайте, какой метод изменяет элементы, какой фильтрует, сортирует или группирует.

- **Select** — Преобразует каждый элемент коллекции в другой вид
- **Where** — Фильтрует элементы по условию
- **OrderBy** — Сортирует элементы по ключу
- **GroupBy** — Группирует элементы по ключу

🔗 Шаг 05:

Миссия 5

Что такое ``Func`` в C#?

Подсказка: Подумайте о типе, который представляет функцию.

Ответ:

- **Делегат для методов с входным и выходным параметром**
- Тип коллекции
- Событие C#
- Интерфейс для классов данных

🔗 Шаг 06:

Миссия 6

Что делает метод ``Aggregate`` в LINQ?

Подсказка: Подумайте о способе получения единого значения из коллекции.

Ответ:

- **Применяет функцию к элементам коллекции и возвращает одно итоговое значение**
- Фильтрует элементы коллекции
- Создает новый объект из коллекции
- Сортирует элементы по ключу

🔗 Шаг 07:

Миссия 7

📄 Сопоставьте функциональные возможности с их примером:

Ответ:

- Lazy evaluation
 - Deferred execution
 - Immutable data
 - Pure function
1. Вычисление выполняется только при необходимости
 2. Выражения LINQ выполняются только при обращении к результату
 3. Объект не меняет своё состояние после создания
 4. Функция не имеет побочных эффектов

🔗 **Подсказка:**

Подумайте о поведении коллекций и функций: когда вычисление откладывается, когда данные неизменны.

- **Lazy evaluation** — Вычисление выполняется только при необходимости
- **Deferred execution** — Выражения LINQ выполняются только при обращении к результату
- **Immutable data** — Объект не меняет своё состояние после создания
- **Pure function** — Функция не имеет побочных эффектов

🔗 Шаг 08:

Миссия 8

Что делает метод ``SelectMany`` в LINQ?

Подсказка: Подумайте о преобразовании вложенных коллекций в одну плоскую.

Ответ:

- **Преобразует вложенные коллекции в одну плоскую коллекцию**
- Фильтрует элементы коллекции
- Сортирует коллекцию
- Группирует элементы по ключу

 **Шаг 09:**

Миссия 9

Что такое `Action` в C#?

Подсказка: Подумайте о типе делегата, который выполняет действие без возвращаемого значения.

Ответ:

- **Делегат для методов, не возвращающих значения**
- Делегат для функций с возвращаемым значением
- Событие C#
- Тип коллекции

 **Шаг 10:**

Миссия 10

Аня разрабатывает библиотеку функций и хочет использовать лямбда-выражения для обработки коллекций. Какой синтаксис правильный для лямбда-выражения в C#?

Подсказка: Лямбда-выражения используют стрелку => для определения тела функции.

Ответ:

- **`x => x * 2`**
- `function(x) { return x * 2 }`
- `(x) -> x * 2`
- `def x: x*2`

 **Шаг 11:**

Миссия 11

Иван хочет использовать метод расширения для коллекции `List<int>`. Какой принцип должен соблюдаться при создании метода расширения?

Подсказка: Метод расширения определяется как статический метод в статическом классе и первым параметром принимает `this`-объект.

Ответ:

- **Метод должен быть статическим и первый параметр использовать `this`**
- Метод может быть любым нестатическим методом класса
- Метод должен быть приватным и без параметров
- Метод должен быть виртуальным и наследоваться

 **Шаг 12:**

Миссия 12

 Сопоставьте функциональные концепции с их описанием.

Понятия:

Ответ:

- Лямбда-выражение
- Метод расширения
- Функциональный интерфейс
- Частичное применение

Определения:

1. Функция, передающаяся как объект и используемая для обработки данных
2. Статический метод, который добавляет новые возможности существующим типам
3. Интерфейс с одним методом, который можно реализовать через делегат
4. Создание новой функции с частью заранее зафиксированных параметров

 Подсказка:

Подумайте, какая концепция позволяет писать короткие функции, а какая расширяет существующие типы.

- **Лямбда-выражение** — Функция, передающаяся как объект и используемая для обработки данных
- **Метод расширения** — Статический метод, который добавляет новые возможности существующим типам
- **Функциональный интерфейс** — Интерфейс с одним методом, который можно реализовать через делегат
- **Частичное применение** — Создание новой функции с частью заранее зафиксированных параметров

 Шаг 13:

Миссия 13

 Сопоставьте методы LINQ с их основным действием.

Понятия:

Ответ:

- Select
- Where
- Aggregate
- OrderBy

Определения:

1. Фильтрует элементы коллекции по условию
2. Проецирует каждый элемент в новую форму
3. Сводит коллекцию к одному значению
4. Сортирует элементы по ключу

 Подсказка:

Подумайте, какой метод выбирает элементы, какой фильтрует, а какой комбинирует.

- **Select** — Проецирует каждый элемент в новую форму
- **Where** — Фильтрует элементы коллекции по условию
- **Aggregate** — Сводит коллекцию к одному значению
- **OrderBy** — Сортирует элементы по ключу

🔗 Шаг 14:

Миссия 14

Сергей хочет избежать мутабельности данных и применяет методы функционального программирования. Какой принцип поможет ему?

Подсказка: Функциональные методы часто не изменяют исходные объекты.

Ответ:

- **Использовать неизменяемые коллекции и методы, возвращающие новые объекты**
- Изменять элементы коллекции напрямую в цикле
- Использовать глобальные переменные для хранения состояния
- Применять только объекты класса без методов

🔗 Шаг 15:

Миссия 15

Расположите этапы применения функционального подхода к коллекции:

1. Определение цели трансформации данных
2. Выбор метода LINQ или функциональной операции
3. Применение метода к коллекции
4. Обработка полученного результата

🔗 **Подсказка:**

Сначала нужно понять, что вы хотите сделать, затем выбрать метод, потом применить и обработать результат.

Ответ:

- Определение цели трансформации данных
- Выбор метода LINQ или функциональной операции
- Применение метода к коллекции
- Обработка полученного результата

🔗 Шаг 16:

Миссия 16

Алексей работает с функцией высшего порядка. Что отличает такие функции в C#?

Подсказка: Обратите внимание на то, что функция может принимать другие функции как параметры или возвращать их.

Ответ:

- **Функция может принимать другие функции как аргументы или возвращать их**
- Функция всегда должна быть void и без параметров
- Функция не может использовать лямбда-выражения
- Функция должна быть приватной и статической

🔗 Шаг 17:

Миссия 17

Расположите шаги при комбинировании нескольких функциональных методов LINQ:

1. Начать с исходной коллекции
2. Применить фильтрацию Where
3. Выполнить проекцию Select

4. Свести результаты с помощью Aggregate или Sum

 Подсказка:

Фильтруйте, потом трансформируйте, а в конце агрегируйте.

Ответ:

- Начать с исходной коллекции
- Применить фильтрацию Where
- Выполнить проекцию Select
- Свести результаты с помощью Aggregate или Sum

Модуль 7 - Тестирование и отладка

Урок 1 - Unit-тесты и Mocking

 Шаг 01:

Миссия 1

Разработчик Дима пишет калькулятор и хочет проверить корректность метода Add() . Какой фреймворк чаще всего используют для Unit-тестов в C#?

Подсказка: Этот инструмент идёт рука об руку с Visual Studio.

Ответ:

- **NUnit**
- Mocha
- PyTest
- RSpec

 Шаг 02:

Миссия 2

Программистка Аня заметила, что её метод работает странно. Она пишет тесты. Какой атрибут в NUnit указывает, что метод является тестом?

Подсказка: Вспомни квадратные скобки над методом.

Ответ:

- **[Test]**
- [Fact]
- [Check]
- [Unit]

 Шаг 03:

Миссия 3

Вася тестирует функцию оплаты. Он хочет проверить, что метод бросает исключение при неверных данных. Какой метод проверки в NUnit для этого используется?

Подсказка: Ожидание исключений — особая форма проверки.

Ответ:

- **Assert.Throws**
- Assert.Fail
- Assert.CatchError
- Assert.Check

Шаг 04:

Миссия 4

Катя тестирует сервис авторизации. Ей нужно изолировать тест от базы данных. Какой инструмент используют для имитации зависимостей?

Подсказка: Этот инструмент «притворяется» настоящим объектом.

Ответ:

- **Mocking**
- Logging
- Refactoring
- Casting

Шаг 05:

Миссия 5

Разработчик Саша использует Moq. Как создаётся поддельный объект?

Подсказка: Вспомни конструкцию с ключевым словом new.

Ответ:

- **new Mock()**
- Mock.Create()
- new Fake()
- Interface.Mock()

Шаг 06:

Миссия 6

Игорь пишет тесты и хочет запустить их все разом в Visual Studio. Где это делается?

Подсказка: Вспомни меню, где обычно ищут тесты.

Ответ:

- **Test Explorer**
- Solution Manager
- Code Editor
- Debugger Window

Шаг 07:

Миссия 7

Марина хочет сделать тест, который будет выполняться перед всеми остальными. Какой атрибут NUnit подойдёт?

Подсказка: Подумай о методе инициализации.

Ответ:

- **[SetUp]**
- [Init]
- [Prepare]
- [Before]

Шаг 08:

Миссия 8

Сопоставьте основные элементы тестирования в NUnit с их назначением.

Подсказка: Вспомни, какие атрибуты запускают тесты, а какие настраивают окружение.

Ответ:

- **[Test]** — Обозначает метод теста
- **[SetUp]** — Запускается перед каждым тестом
- **[TearDown]** — Выполняется после каждого теста
- **Assert.AreEqual** — Сравнение ожидаемого и фактического значения

🔮 Шаг 09:

Миссия 9

Артём тестирует магазин. Сопоставьте библиотеку с её назначением.

Подсказка: NUnit для тестов, Moq для подмены.

Ответ:

- **NUnit** — Фреймворк для модульного тестирования
- **Moq** — Создание поддельных зависимостей
- **xUnit** — Альтернативный тестовый фреймворк
- **FluentAssertions** — Более читаемые утверждения

🔮 Шаг 10:

Миссия 10

Сопоставьте термин с его описанием.

Подсказка: Подумайте, чем отличается mock от stub.

Ответ:

- **Mock** — Объект, проверяющий взаимодействие
- **Stub** — Простейшая заглушка, возвращающая фиксированные данные
- **Fake** — Упрощённая рабочая реализация
- **Dummy** — Объект-заглушка без логики

🔮 Шаг 11:

Миссия 11

Программистка Елена использует Moq. Сопоставьте метод с его назначением.

Подсказка: Настройка поведения делается через Setup.

Ответ:

- **Setup()** — Определение поведения метода
- **Returns()** — Указание возвращаемого значения
- **Verify()** — Проверка вызова метода
- **Object** — Получение поддельного объекта

🔮 Шаг 12:

Миссия 12

Сопоставьте принцип тестирования с его кратким описанием.

Подсказка: Вспомни AAA (Arrange, Act, Assert).

Ответ:

- **Arrange** — Подготовка окружения

- **Act** — Выполнение тестируемого действия
- **Assert** — Проверка результата
- **Clean-up** — Очистка после теста

🔗 Шаг 13:

Миссия 13

Сопоставьте ситуацию и соответствующий инструмент.

Подсказка: Для проверки исключений в NUnit есть специальный метод.

Ответ:

- **Проверить равенство двух чисел** — Assert.AreEqual
- **Проверить, что метод выбросил исключение** — Assert.Throws
- **Создать поддельный объект зависимости** — Moq
- **Запустить все тесты** — Test Explorer

🔗 Шаг 14:

Миссия 14

Расположите шаги написания теста в правильном порядке.

Подсказка: Используй правило AAA.

Ответ:

- **Arrange** — подготовка данных
- **Act** — вызов метода
- **Assert** — проверка результата

🔗 Шаг 15:

Миссия 15

Разработчик Даниил пишет юнит-тест. Расставь этапы работы с Moq в правильном порядке.

Подсказка: Сначала создаём mock, затем настраиваем, используем и проверяем.

Ответ:

- Создание Mock объекта
- Настройка поведения методом Setup
- Вызов метода через Object
- Проверка вызова методом Verify

🔗 Шаг 16:

Миссия 16

Упорядочьте этапы отладки теста.

Подсказка: Вначале запускаем, потом анализируем ошибку, исправляем и снова проверяем.

Ответ:

- Запуск теста
- Анализ ошибки
- Исправление кода
- Повторный запуск

Шаг 17:

Миссия 17

Расположите шаги создания тестового проекта в Visual Studio.

Подсказка: Вспомни: новый проект, добавление тестов, настройка зависимостей, запуск.

Ответ:

- Создать новый проект Unit Test
- Добавить ссылки на необходимые библиотеки
- Написать тестовые методы
- Запустить тесты через Test Explorer

Шаг 18:

Миссия 18

Анна разрабатывает сервис и хочет убедиться, что метод AddUser работает корректно. Какой подход правильный для unit-тестирования?

Подсказка: Unit-тест проверяет отдельный метод в изоляции от внешних зависимостей.

Ответ:

- **Написать тест, который проверяет только метод AddUser без внешних зависимостей**
- Проверить весь сервис целиком с базой данных
- Тестировать метод только вручную в продакшн-среде
- Игнорировать метод, так как он прост

Шаг 19:

Миссия 19

Иван хочет протестировать метод, который обращается к внешнему API. Какой подход поможет избежать реальных вызовов API?

Подсказка: Использование Mock позволяет имитировать поведение внешних сервисов.

Ответ:

- **Использовать Mock объекта API для тестирования метода**
- Обращаться к реальному API в каждом тесте
- Игнорировать тестирование методов с внешними API
- Тестировать методом интеграции с базой данных

Урок 2 - Benchmarking и профилирование

Шаг 01:

Миссия 1

Андрей хочет измерить производительность метода сортировки. Какой популярный фреймворк в C# используется для Benchmarking?

Подсказка: Этот инструмент поддерживается командой .NET и активно применяется в сообществе.

Ответ:

- **BenchmarkDotNet**
- PerfTestSharp
- NBench

- SpeedCheck

🔮 Шаг 02:

Миссия 2

Марина запускает BenchmarkDotNet. Какой атрибут используется для пометки метода, который нужно измерить?

Подсказка: Этот атрибут пишется над методом и говорит: «замерь мою скорость».

Ответ:

- **[Benchmark]**
- [Measure]
- [PerfTest]
- [SpeedCheck]

🔮 Шаг 03:

Миссия 3

Олег заметил, что при запуске BenchmarkDotNet результаты показывают среднее время выполнения. В какой единице измерения чаще всего выводятся результаты?

Подсказка: Это очень маленькая единица времени.

Ответ:

- **Наносекунды**
- Минуты
- Секунды
- Часы

🔮 Шаг 04:

Миссия 4

Елена сравнивает два алгоритма поиска. Какой параметр BenchmarkDotNet поможет сравнить результаты между методами?

Подсказка: Это столбец в итоговой таблице.

Ответ:

- **Mean**
- Name
- Method
- RunId

🔮 Шаг 05:

Миссия 5

Дима хочет задать количество итераций теста. Какой атрибут BenchmarkDotNet используется для этого?

Подсказка: Этот атрибут управляет количеством повторений для большей точности.

Ответ:

- **[IterationCount]**
- [LoopCount]
- [Repeat]
- [Count]

🔗 Шаг 06:

Миссия 6

Игорь использует профилировщик для анализа кода. Что чаще всего показывает профилировщик?

Подсказка: Это помогает найти «узкие места» по времени.

Ответ:

- **Где тратится больше всего времени выполнения**
- Только количество строк кода
- Ошибки компиляции
- Названия переменных

🔗 Шаг 07:

Миссия 7

Наташа использует BenchmarkDotNet, но результаты сильно отличаются между запусками. Что может помочь получить стабильные результаты?

Подсказка: Подумай о прогреве JIT-компилятора.

Ответ:

- **[WarmupCount]**
- [TestRepeat]
- [Retry]
- [Stabilize]

🔗 Шаг 08:

Миссия 8

Сопоставьте инструмент с его назначением.

Подсказка: Один для тестов, другой для замеров, третий для профилирования.

Ответ:

- **NUnit** — Фреймворк модульного тестирования
- **BenchmarkDotNet** — Замеры производительности методов
- **dotTrace** — Профилировка времени выполнения
- **PerfView** — Анализ производительности и памяти

🔗 Шаг 09:

Миссия 9

Артём решил понять, какие показатели выводит BenchmarkDotNet. Сопоставьте показатель и его смысл.

Подсказка: Среднее время $\neq$ стандартное отклонение.

Ответ:

- **Mean** — Среднее время выполнения
- **Error** — Приблизительная ошибка измерения
- **StdDev** — Стандартное отклонение
- **Median** — Медианное значение

🔮 Шаг 10:

Миссия 10

Сопоставьте атрибут BenchmarkDotNet и его назначение.

Подсказка: Некоторые управляют временем, другие количеством запусков.

Ответ:

- **[Benchmark]** — Метод, подлежащий измерению
- **[IterationCount]** — Количество итераций
- **[WarmupCount]** — Количество прогревочных запусков
- **[MemoryDiagnoser]** — Измерение потребления памяти

🔮 Шаг 11:

Миссия 11

Сопоставьте профилировщик и его специализацию.

Подсказка: Один анализирует CPU, другой — память.

Ответ:

- **dotTrace** — Анализ нагрузки на процессор
- **dotMemory** — Поиск утечек памяти
- **PerfView** — Сбор ETW-трейсов
- **Visual Studio Profiler** — Встроенный профилировщик IDE

🔮 Шаг 12:

Миссия 12

Сопоставьте метрику с её описанием.

Подсказка: Время и память — разные показатели.

Ответ:

- **Allocated** — Объём выделенной памяти
- **Gen 0** — Количество сборок мусора младшего поколения
- **Gen 2** — Количество сборок мусора старшего поколения
- **Op/s** — Количество операций в секунду

🔮 Шаг 13:

Миссия 13

Расположите шаги подготовки к BenchmarkDotNet в правильном порядке.

Подсказка: Сначала подключение пакета, потом написание кода.

Ответ:

- Установить пакет BenchmarkDotNet
- Создать класс с методами
- Пометить метод атрибутом [Benchmark]
- Запустить BenchmarkRunner

🔮 Шаг 14:

Миссия 14

Разработчик Максим хочет проанализировать производительность. Расставь этапы работы с профилировщиком.

Подсказка: Сначала запусти программу, потом анализируй.

Ответ:

- Запустить приложение в профилировщике
- Собрать данные о выполнении
- Посмотреть отчёт о времени и памяти
- Найти узкие места и оптимизировать

Шаг 15:

Миссия 15

Упорядочьте этапы анализа утечек памяти.

Подсказка: Сначала запускаем, потом ищем подозрительные объекты.

Ответ:

- Запустить приложение под dotMemory
- Собрать snapshot памяти
- Сравнить snapshots
- Выявить объекты, не освобождаемые сборщиком

Шаг 16:

Миссия 16

Расположите шаги оптимизации кода по результатам BenchmarkDotNet.

Подсказка: Сначала измеряем, потом улучшаем.

Ответ:

- Написать бенчмарк
- Запустить BenchmarkDotNet
- Анализировать отчёт
- Внести оптимизации и повторить замеры

Шаг 17:

Миссия 17

Алексей хочет измерить скорость выполнения метода сортировки списка чисел. Какой подход верный?

Подсказка: Benchmarking помогает замерять время работы отдельных методов.

Ответ:

- **Использовать BenchmarkDotNet для измерения времени выполнения метода**
- Вручную ставить таймеры и делать замеры на разных компьютерах
- Пропустить измерения, так как метод простой
- Использовать профайлер для замера только памяти

Шаг 18:

Миссия 18

Марина хочет понять, какой метод занимает больше всего ресурсов CPU. Какой инструмент лучше использовать?

Подсказка: Профилирование помогает выявлять узкие места по использованию ресурсов.

Ответ:

- **Использовать профайлер для измерения использования CPU**
- Считать вызовы методов вручную
- Оценивать нагрузку по времени выполнения на глаз
- Измерять только память, игнорируя CPU

🔗 **Шаг 19:**

Миссия 19

📄 Сопоставьте понятия Benchmarking и профилирования с их описанием.

Понятия:

Ответ:

- Benchmarking
- Профилирование
- CPU-bound процесс
- Memory-bound процесс

Определения:

1. Измерение времени выполнения определённых методов
2. Анализ использования ресурсов программы (CPU, память, I/O)
3. Процесс, ограниченный производительностью процессора
4. Процесс, ограниченный объёмом памяти или её доступностью

🔗 **Подсказка:**

Подумайте, что измеряет Benchmark, а что показывает профилировщик.

- **Benchmarking** — Измерение времени выполнения определённых методов
- **Профилирование** — Анализ использования ресурсов программы (CPU, память, I/O)
- **CPU-bound процесс** — Процесс, ограниченный производительностью процессора
- **Memory-bound процесс** — Процесс, ограниченный объёмом памяти или её доступностью

📖 **Урок 3 - Integration и End-to-End тестирование**

🔗 **Шаг 01:**

Миссия 1

Алексей интегрирует модуль оплаты в интернет-магазин и хочет проверить работу с реальной базой данных. Какой тип теста он должен использовать?

Подсказка: Здесь важна не изолированность, а взаимодействие нескольких компонентов.

Ответ:

- **Интеграционный тест**
- Юнит-тест
- Модульный тест
- Smoke-тест

🔗 **Шаг 02:**

Миссия 2

Мария подключает API геолокации к своему приложению. Что она проверяет с помощью end-to-end теста?

Подсказка: Такой тест должен моделировать действия реального пользователя.

Ответ:

- **Полный сценарий использования от ввода до результата**
- Работу одного метода API в изоляции
- Только корректность SQL-запроса
- Скорость работы одного класса

Шаг 03:

Миссия 3

Дмитрий запустил интеграционный тест и получил ошибку соединения с Redis. Что это чаще всего означает?

Подсказка: Здесь проблема может быть не в логике, а в окружении.

Ответ:

- **Неверно настроено внешнее окружение**
- Ошибка в арифметических операциях
- Неверный юнит-тест
- Пропущен using System

Шаг 04:

Миссия 4

Ольга написала end-to-end тест, который проверяет покупку билета в онлайн-системе. Что будет ключевым результатом такого теста?

Подсказка: Этот результат должен быть максимально приближен к опыту реального пользователя.

Ответ:

- **Подтверждение успешной покупки билета**
- Проверка вызова одного метода
- Вычисление контрольной суммы
- Запуск консольной команды

Шаг 05:

Миссия 5

Сергей тестирует приложение, где участвуют веб-сервер, база данных и очередь сообщений. Что он скорее всего запускает?

Подсказка: Этот тип тестов проверяет взаимодействие сразу нескольких сервисов.

Ответ:

- **Интеграционные тесты**
- Юнит-тесты
- Бенчмаркинг
- Рефакторинг

Шаг 06:

Миссия 6

Сопоставьте виды тестов с их основной целью.

Подсказка: Вспомните различие между проверкой отдельных функций и всей системы.

Ответ:

- **Юнит-тест** — Проверка работы одного метода или класса в изоляции
- **Интеграционный тест** — Проверка взаимодействия нескольких компонентов
- **End-to-End тест** — Моделирование сценария реального пользователя
- **Smoke-тест** — Быстрая проверка, что приложение запускается

Шаг 07:

Миссия 7

Сопоставьте проблему с типом теста, который её быстрее выявит.

Подсказка: Если ошибка в бизнес-логике метода — это один тип, если сбой на уровне сервера — другой.

Ответ:

- **Ошибка в арифметике метода** — Юнит-тест
- **Проблема подключения к базе данных** — Интеграционный тест
- **Неоткрываемая форма на сайте** — End-to-End тест
- **Сервер не запускается** — Smoke-тест

Шаг 08:

Миссия 8

Сопоставьте понятие с его описанием.

Подсказка: Обратите внимание на уровень изоляции и охват системы.

Ответ:

- **Изоляция** — Тестирование без зависимости от внешних сервисов
- **Полный сценарий** — Воспроизведение действий пользователя от начала до конца
- **Контрактный тест** — Проверка корректности взаимодействия API
- **Регрессионный тест** — Проверка, что старый функционал работает после изменений

Шаг 09:

Миссия 9

Сопоставьте роли участников проекта с тем, какие тесты они обычно запускают.

Подсказка: QA чаще проверяют систему целиком, а разработчики — отдельные части.

Ответ:

- **Разработчик** — Юнит-тесты
- **DevOps инженер** — Smoke-тесты при деплое
- **Тестировщик** — End-to-End тесты
- **Архитектор** — Интеграционные тесты

Шаг 10:

Миссия 10

Сопоставьте преимущества тестов с их типом.

Подсказка: Чем шире охват, тем выше доверие к системе, но и тесты дольше.

Ответ:

- **Быстро запускаются** — Юнит-тесты
- **Проверяют критичные сценарии пользователя** — End-to-End тесты

- **Обнаруживают ошибки интеграции сервисов** — Интеграционные тесты
- **Подтверждают запуск приложения** — Smoke-тесты

Шаг 11:

Миссия 11

Расположите шаги написания end-to-end теста в правильном порядке:

1. Проверка ожидаемого результата
2. Запуск приложения
3. Ввод данных пользователем
4. Навигация по интерфейсу

Подсказка: Сначала приложение нужно открыть, потом взаимодействовать, а в конце убедиться в результате.

Ответ:

- Запуск приложения
- Навигация по интерфейсу
- Ввод данных пользователем
- Проверка ожидаемого результата

Шаг 12:

Миссия 12

Расположите уровни тестирования от самого узкого к самому широкому.

Подсказка: Думайте от проверки маленькой функции до проверки всей системы.

Ответ:

- Юнит-тест
- Интеграционный тест
- End-to-End тест
- Системное тестирование

Шаг 13:

Миссия 13

Расположите шаги интеграционного теста:

1. Поднятие необходимых сервисов
2. Вызов взаимодействующих модулей
3. Анализ результата
4. Завершение и очистка окружения

Подсказка: Сначала развернуть окружение, потом проверить, и только в конце закрыть всё.

Ответ:

- Поднятие необходимых сервисов
- Вызов взаимодействующих модулей
- Анализ результата
- Завершение и очистка окружения

Шаг 14:

Миссия 14

Расположите действия тестировщика при нахождении ошибки в E2E тесте.

1. Завести баг-репорт
2. Повторить тест для воспроизведения
3. Сообщить команде
4. Анализ логов

Подсказка: Ошибку нужно воспроизвести, проанализировать и только потом официально оформить.

Ответ:

- Повторить тест для воспроизведения
- Анализ логов
- Завести баг-репорт
- Сообщить команде

Шаг 15:

Миссия 15

Расположите шаги CI/CD пайплайна с тестами:

1. Запуск интеграционных тестов
2. Сборка проекта
3. Деплой на тестовый сервер
4. Запуск end-to-end тестов

Подсказка: Сначала билд, потом интеграция, затем деплой и финальная проверка.

Ответ:

- Сборка проекта
- Запуск интеграционных тестов
- Деплой на тестовый сервер
- Запуск end-to-end тестов

Урок 4 - Logging и Telemetry

Шаг 01:

Миссия 1

Алексей пишет сервис на C#, и его команда решила добавить базовое логирование. Какой минимальный уровень логов стоит включить в production?

Подсказка: Подумайте, что слишком подробное логирование может перегрузить систему, но слишком редкое — лишит вас информации.

Ответ:

- **Error и Warning**
- Debug
- Trace
- Verbose

Шаг 02:

Миссия 2

Марина тестирует новый модуль. Она добавила логирование всех запросов и ответов, включая тела сообщений. Какой риск она создала?

Подсказка: Иногда логирование может раскрывать слишком много информации.

Ответ:

- **Утечка конфиденциальных данных**
- Повышение скорости работы системы
- Улучшение читаемости логов
- Уменьшение затрат на хранение логов

Шаг 03:

Миссия 3

Команда обсуждает, как правильно писать сообщения логов. Какой вариант будет наиболее полезным для последующей диагностики?

Подсказка: Лог должен быть понятен не только автору кода, но и любому члену команды.

Ответ:

- **Сообщения с контекстом (ID запроса, пользователь, параметры)**
- Сообщения вроде «Ошибка здесь!»
- Слишком подробные технические детали без контекста
- Минимум текста, только слово «Ошибка»

Шаг 04:

Миссия 4

Игорь хочет использовать Microsoft.Extensions.Logging. Как правильно подключить логгер к своему сервису?

Подсказка: Вспомните про dependency injection.

Ответ:

- **Внедрить ILogger через конструктор**
- Создать новый Logger в каждом методе
- Использовать Console.WriteLine
- Сохранять логи в статический List

Шаг 05:

Миссия 5

Ольга хочет измерять метрики приложения: время отклика, количество ошибок. Что для этого лучше использовать?

Подсказка: Это часть телеметрии, а не обычного логирования.

Ответ:

- **Telemetry (например, Application Insights)**
- Debug.WriteLine
- Комментарии в коде
- Файл README.md

Шаг 06:

Миссия 6

Компания решила собирать централизованные логи. Какой инструмент часто используется в связке для этого?

Подсказка: Популярный стек: Elasticsearch + ...

Ответ:

- **Kibana**
- MS Paint
- Notepad
- Excel

🔗 **Шаг 07:**

Миссия 7

Евгений анализирует логи и видит, что система перегружена большим количеством Debug-сообщений. Что стоит сделать?

Подсказка: Уровень логирования можно менять.

Ответ:

- **Отключить или понизить уровень Debug в production**
- Добавить ещё больше Debug
- Удалить все логи
- Заменить Debug на Console.WriteLine

🔗 **Шаг 08:**

Миссия 8

Сопоставьте уровни логирования с их назначением.

Подсказка: Представьте, что у вас разные фильтры: от самых критичных событий до самых подробных.

Ответ:

- **Error** — Серьезная проблема, требующая немедленного внимания.
- **Warning** — Что-то пошло не так, но система может продолжить работу.
- **Info** — Информация о нормальной работе приложения.
- **Debug** — Подробные сведения для разработчиков.

🔗 **Шаг 09:**

Миссия 9

Свяжите инструмент с его основной задачей в логировании и телеметрии.

Подсказка: Вспомните известные связи Elastic и Application Insights.

Ответ:

- **ElasticSearch** — Хранение и поиск логов.
- **Kibana** — Визуализация логов и метрик.
- **Application Insights** — Сбор телеметрии и мониторинг приложений.
- **Seq** — Централизованный просмотр структурированных логов.

🔗 **Шаг 10:**

Миссия 10

Сопоставьте элементы хорошего лог-сообщения с их примером.

Подсказка: Думайте о контексте: время, место, участник события.

Ответ:

- **Timestamp** — 2025-09-26 12:34:56

- **CorrelationId** — Request-12345
- **User** — Ivan.Petrov
- **Message** — Ошибка при сохранении заказа.

🔗 Шаг 11:

Миссия 11

Сопоставьте тип телеметрии с примером данных.

Подсказка: Ошибки, метрики и трассировки — это разные сущности.

Ответ:

- **Metrics** — Среднее время ответа сервиса — 250 мс.
- **Traces** — Путь запроса через 3 микросервиса.
- **Logs** — Ошибка при подключении к базе данных.
- **Events** — Пользователь завершил регистрацию.

🔗 Шаг 12:

Миссия 12

Сопоставьте понятие с примером из реальной жизни разработчика.

Подсказка: Представьте, что вы диагностируете систему.

Ответ:

- **Verbose logging** — Содержит каждый шаг выполнения, включая внутренние детали.
- **Structured logging** — Лог хранится в формате JSON с ключами и значениями.
- **Correlation Id** — Помогает связать все запросы одного пользователя.
- **Sampling** — Сбор только части запросов для уменьшения нагрузки.

🔗 Шаг 13:

Миссия 13

Расположите шаги настройки Application Insights для C# приложения.

Подсказка: Подумайте: сначала нужно добавить библиотеку, потом настроить, затем инициализировать, а потом использовать.

Ответ:

- Установить пакет NuGet Application Insights
- Добавить настройки подключения (Instrumentation Key)
- Инициализировать TelemetryClient
- Отправлять метрики и логи

🔗 Шаг 14:

Миссия 14

Расставьте шаги обработки ошибки с логированием.

Подсказка: Сначала фиксируем ошибку, потом записываем, затем уведомляем и анализируем.

Ответ:

- Поймать исключение
- Записать в лог с уровнем Error
- Отправить уведомление ответственному
- Проанализировать причину

Шаг 15:

Миссия 15

Расположите шаги настройки централизованного логирования с использованием ElasticSearch.

Подсказка: Сначала собираем, потом передаем, затем сохраняем и визуализируем.

Ответ:

- Собрать логи с микросервисов
- Передать их через Logstash
- Сохранить в ElasticSearch
- Отобразить в Kibana

Шаг 16:

Миссия 16

Расположите шаги анализа телеметрии при расследовании инцидента.

Подсказка: Сначала собираем данные, потом фильтруем, ищем корреляции и только после этого делаем выводы.

Ответ:

- Собрать данные логов и метрик
- Отфильтровать по времени инцидента
- Найти запросы с одинаковым CorrelationId
- Сделать выводы о причине сбоя

Шаг 17:

Миссия 17

Расставьте шаги добавления структурированного логирования в .NET проект.

Подсказка: Вспомните: сначала библиотека, потом настройка, потом внедрение и использование.

Ответ:

- Подключить библиотеку Serilog
- Настроить sinks (например, в файл или ElasticSearch)
- Внедрить ILogger в сервисы
- Писать структурированные логи с контекстом